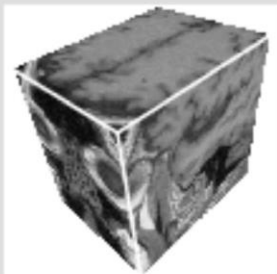
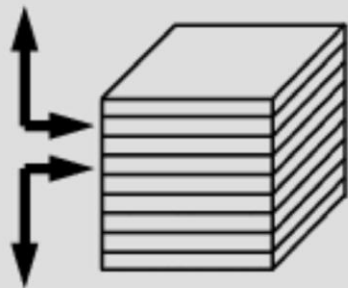
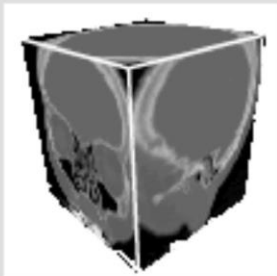
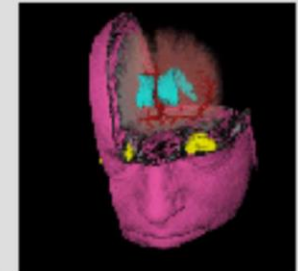
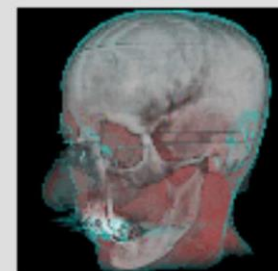
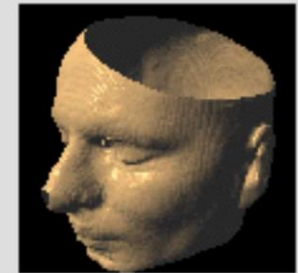
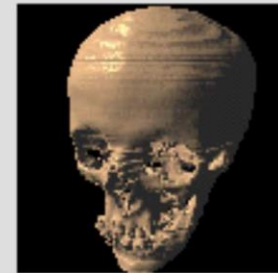
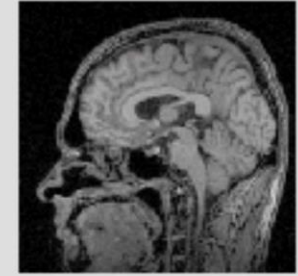


Rendu volumique

HA19111

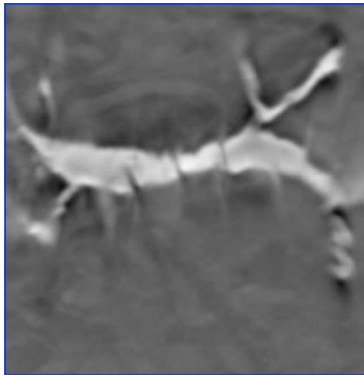


- 2D visualization slice images (or multi-planar reformatting MPR)
- *Indirect* 3D visualization isosurfaces (or surface-shaded display SSD)
- *Direct* 3D visualization (direct volume rendering DVR)



Extraction d'isosurface

- Extraire une isosurface
 - « binaire »
 - Manque d'information de l'intérieur
 - Comment les voxels contribuent à l'image ?
 - Est-ce qu'une frontière est nécessaire pour la tâche de visualisation ?



Slice



Isosurface

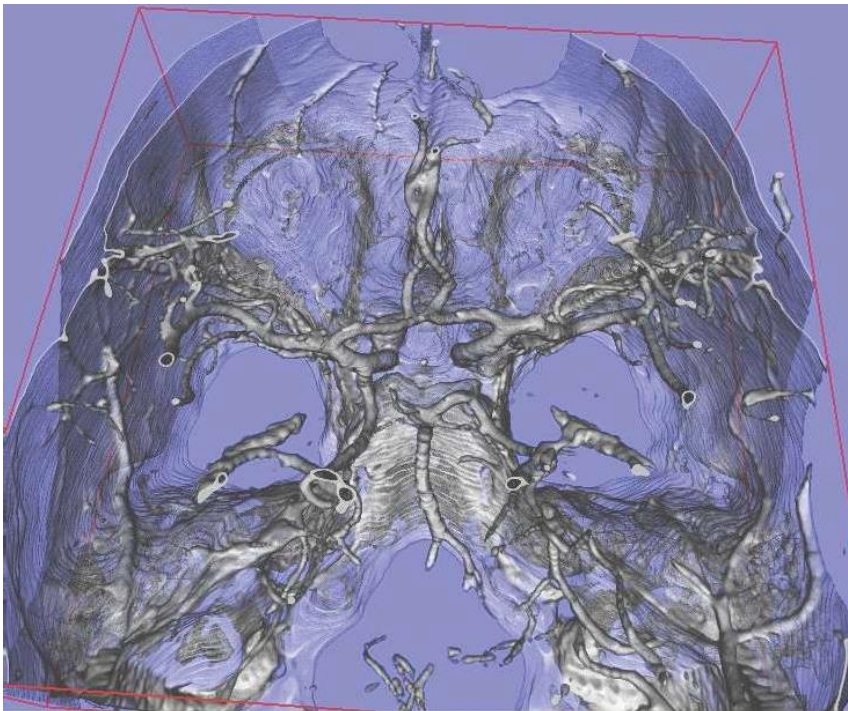


Volume Rendering

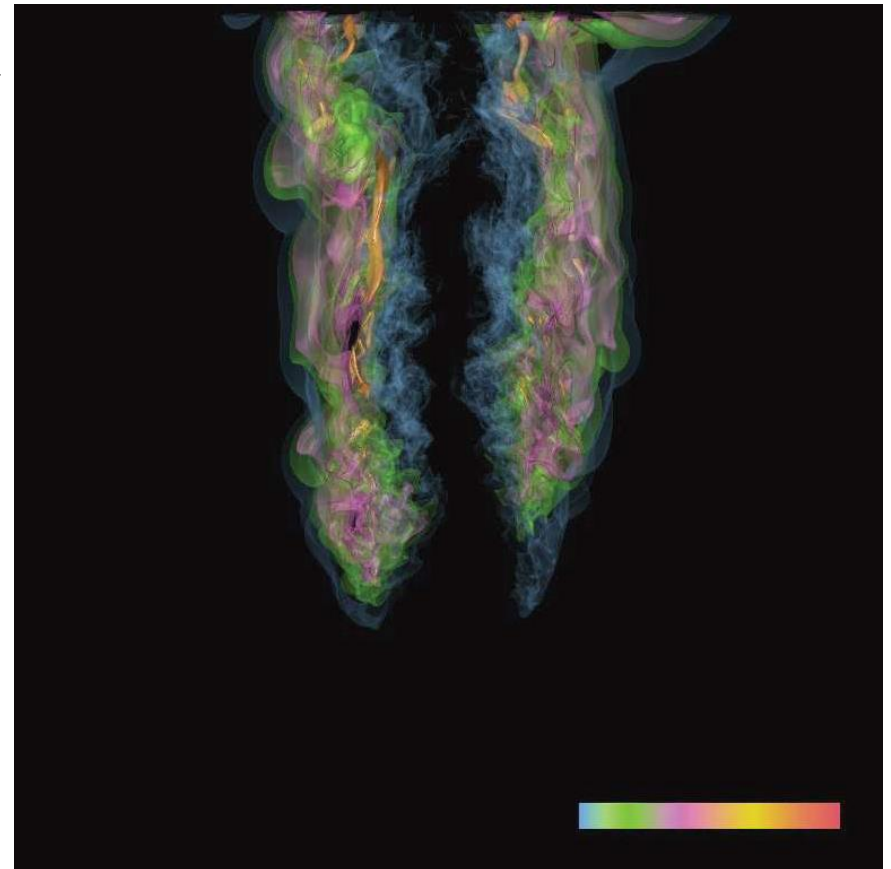
Problème des isosurfaces

Résultat de mauvaise qualité sur

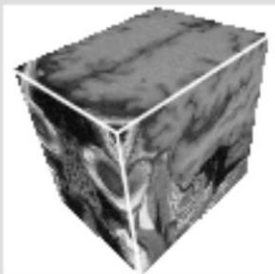
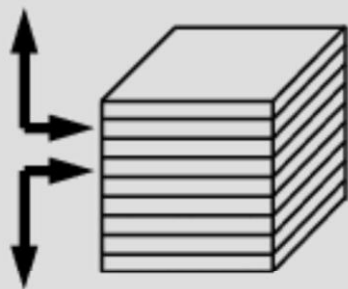
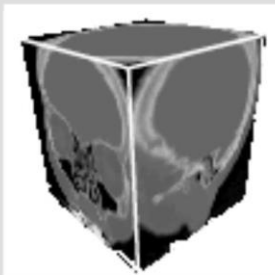
- Données « réelles » bruitées
- Objets amorphes, « mous »



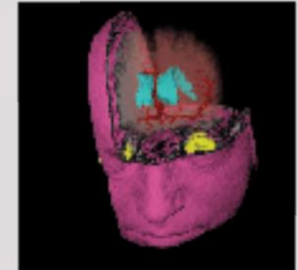
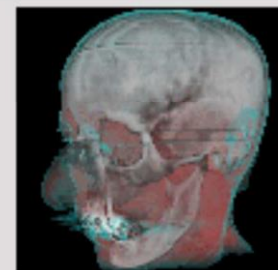
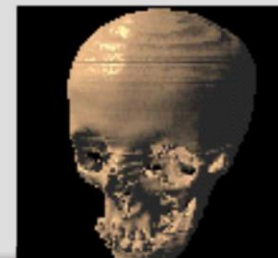
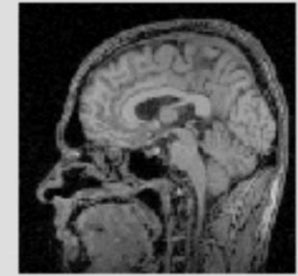
<http://www.cg.informatik.uni-siegen.de/>



<http://vis.cs.ucdavis.edu/gallery/Yu/combustion/>

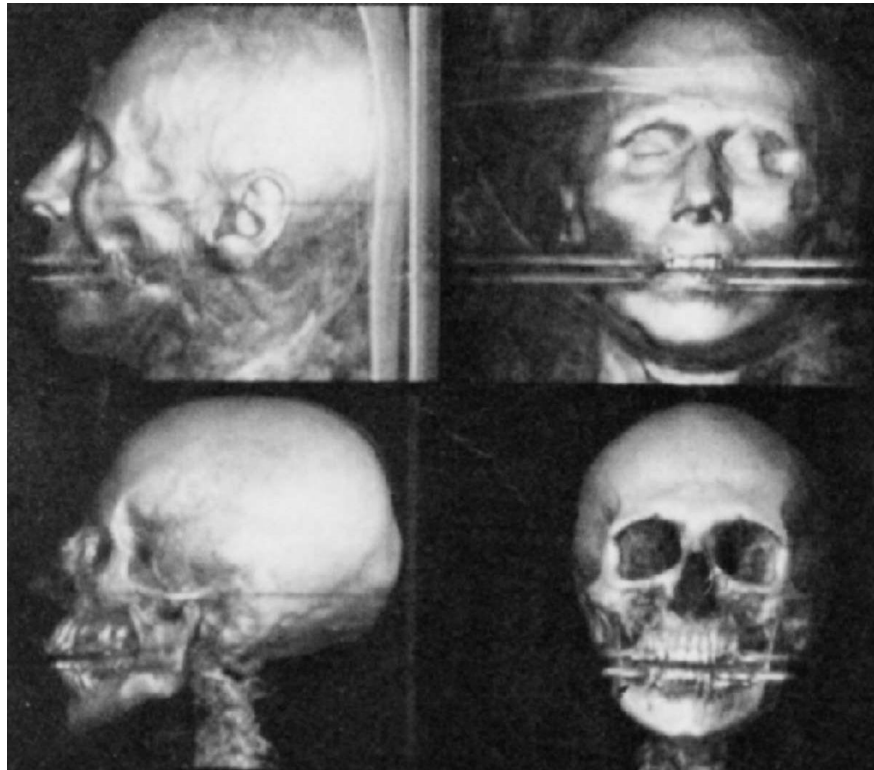


- 2D visualization slice images (or multi-planar reformatting MPR)
- *Indirect* 3D visualization isosurfaces (or surface-shaded display SSD)
- *Direct* 3D visualization (direct volume rendering DVR)



Rendu volumique ?

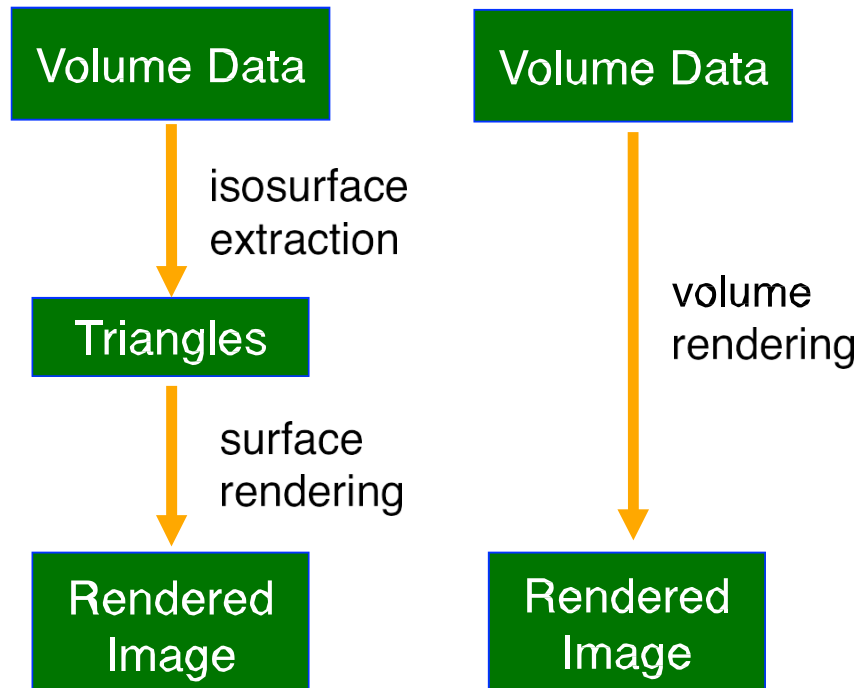
- Tous les voxels contribuent à l'image
- Plus flexible



Marc Levoy, Display of Surfaces from Volume Data, 1988

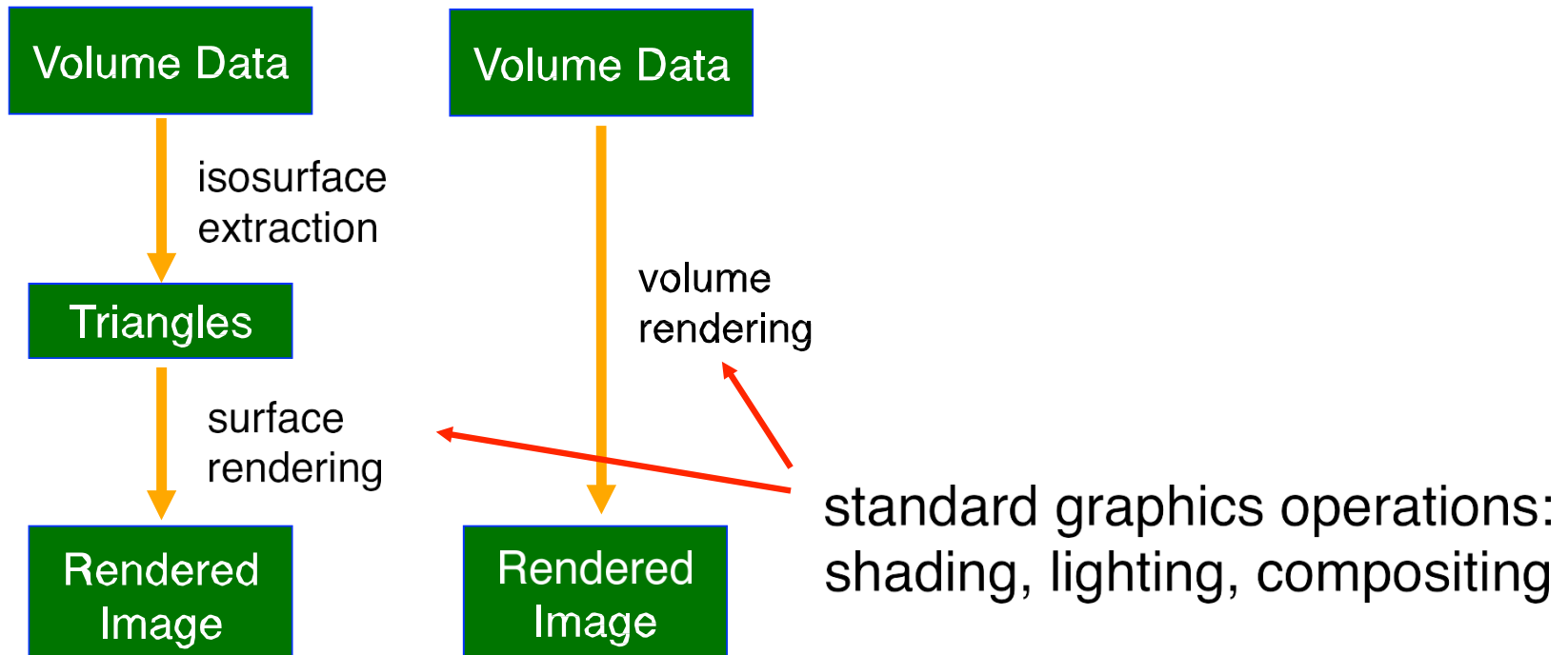
Rendu volumique

Pipelines : Isosurface vs rendu volumique



Pipelines : Isosurface vs rendu volumique

the standard pipeline:
no intermediate geometric structures



Real-Time Volume Graphics

Klaus Engel

Siemens Corporate Research, Princeton, USA

Markus Hadwiger

VR VIS Research Center, Vienna, Austria

Joe M. Kniss

SCI, University of Utah, USA

Aaron E. Lefohn

University of California, Davis, USA

Christof Rezk Salama

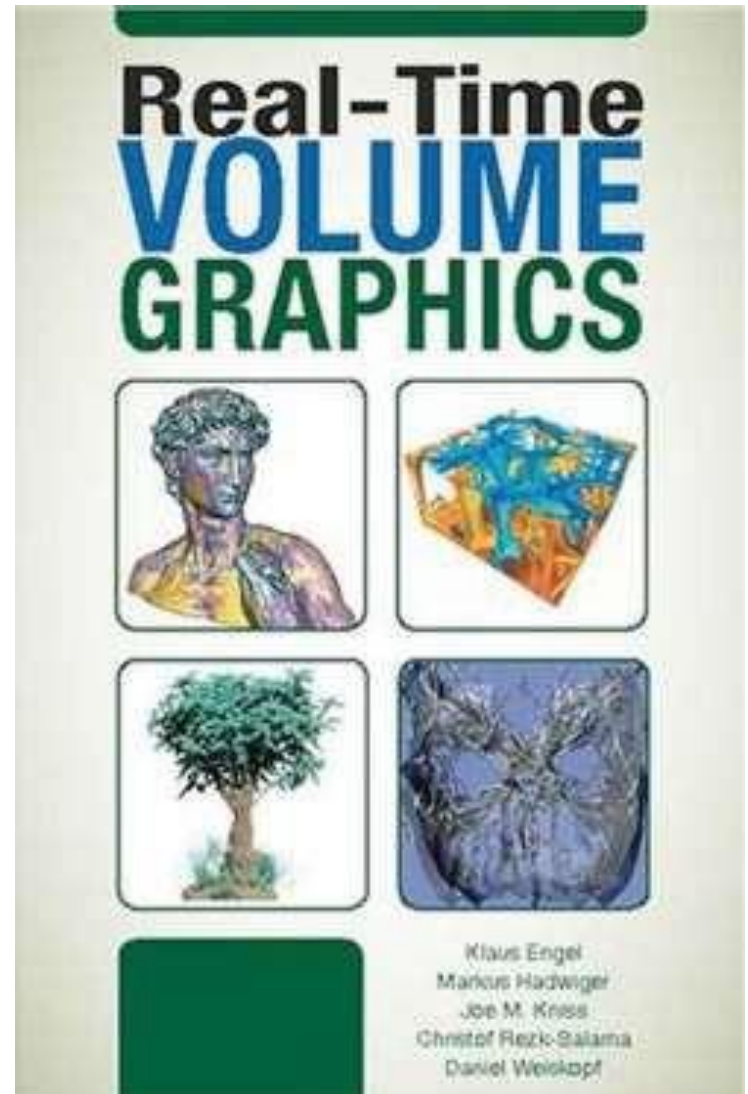
University of Siegen, Germany

Daniel Weiskopf

University of Stuttgart, Germany



SIGGRAPH2004



Real-Time Volume Graphics

GLU-based volume rendering for scientific visualization and visual arts

Tutorials

Real-Time Volume Graphics Tutorial

Here are the slides from the original course, the book is based upon. The latest revision are the powerpoint slides of the Tutorial held at the Eurographics 2006 conference in Vienna, Austria, September/5/2006. The same course was also held at SIGGRAPH

- » Part 01: Introduction and Theory ([ppt](#)) ([pdf](#))
- » Part 02: GPU Programming ([ppt](#)) ([pdf](#))
- » Part 03: Texture-Based Volume Rendering ([ppt](#)) ([pdf](#))
- » Part 04: GPU-Based Ray Casting ([ppt](#)) ([pdf](#))
- » Part 05: Transfer Functions ([ppt](#)) ([pdf](#))
- » Part 06: Local Illumination ([ppt](#)) ([pdf](#))
- » Part 07: Global Illumination ([ppt](#)) ([pdf](#))
- » Part 08: Improving Performance ([ppt](#)) ([pdf](#))
- » Part 09: Improving Quality ([ppt](#)) ([pdf](#))
- » Part 10: Transfer Functions Reloaded ([ppt](#)) ([pdf](#))
- » Part 11: Game Developers Guide to Volume Graphics ([ppt](#)) ([pdf](#))
- » Part 12: Volume Modeling, Deformation and Animation ([ppt](#)) ([pdf](#))

Topics

- »
- » [About the Book](#)
- » [Code Samples](#)
- » [Datasets](#)
- » [Errata](#)
- » [Table of Contents](#)
- » [Tutorials](#)
- » [Volume Graphics Tutorials](#)
- » [What is Volume Graphics?](#)

Topics

- » [News and Events](#)
- » [Software](#)

People

- » [Christof Rezk-Salama](#)
- » [Daniel Weiskopf](#)
- » [Joe M. Kniss](#)
- » [Klaus Engel](#)
- » [Markus Hadwiger](#)

Recent Comments

- » [Priscilla](#) on [Voreen Framework](#)
- » [is quibids legit](#) on [Voreen Framework](#)
- » [Prweb.Com](#) on [Voreen Framework](#)
- » [musculo](#) on [Voreen Framework](#)
- » [Антон Павлович](#) on [Voreen](#)

Qu'est ce que le rendu volumique ?

- Rendu volume → image sans distinction binaire ou géométrie intermédiaire

Question principale :

- Comment rend-on la donnée visible ?
- Réponse : couleur et opacité

Représentation 3D directe des données volumiques

Donnée :

- Représente un médium émetteur de lumière, semi-transparent
- (simulation de phénomènes gazeux)
- Approches basées sur les lois de la physique (emission, absorption, scattering)
- Donnée volume utilisée entièrement (intérieur : exploré, structuré)

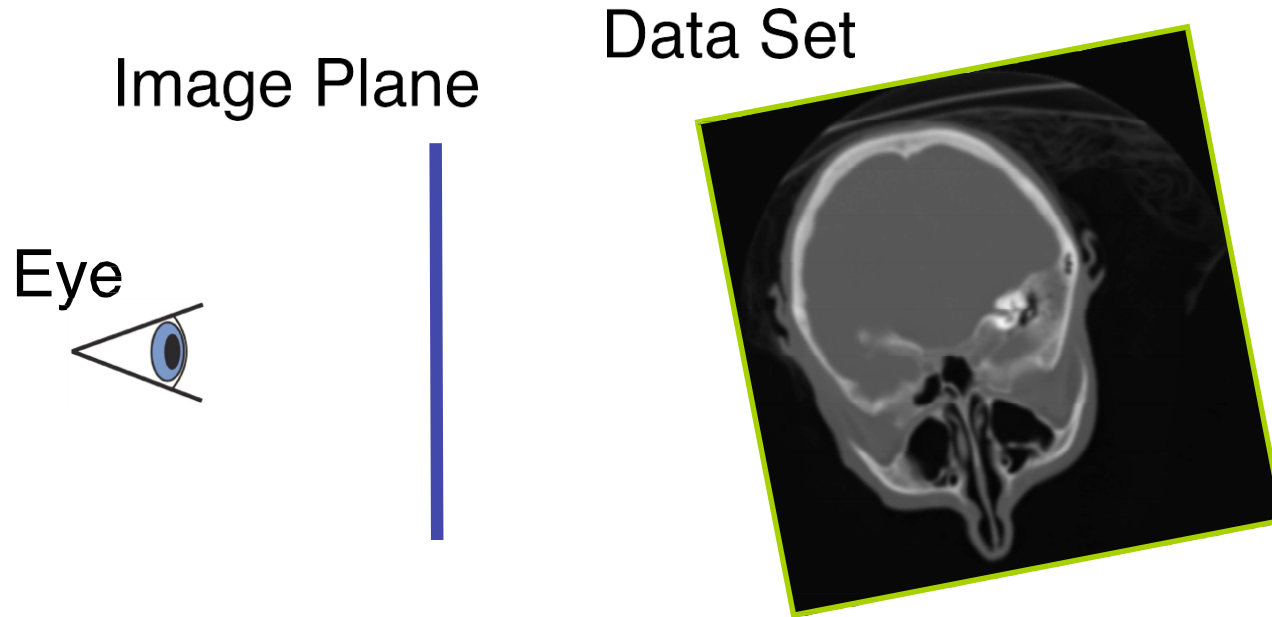
Représentation 3D directe des données volumiques

- DVR : Direct Volume rendering **sans extraire de surface**
- Mapping de valeurs scalaire vers de **propriétés optiques** (couleur, opacité)
- Demande un **modèle optique**
- Résout **l'intégrale du rendu volumique** pour visualiser des rayons dans le volume

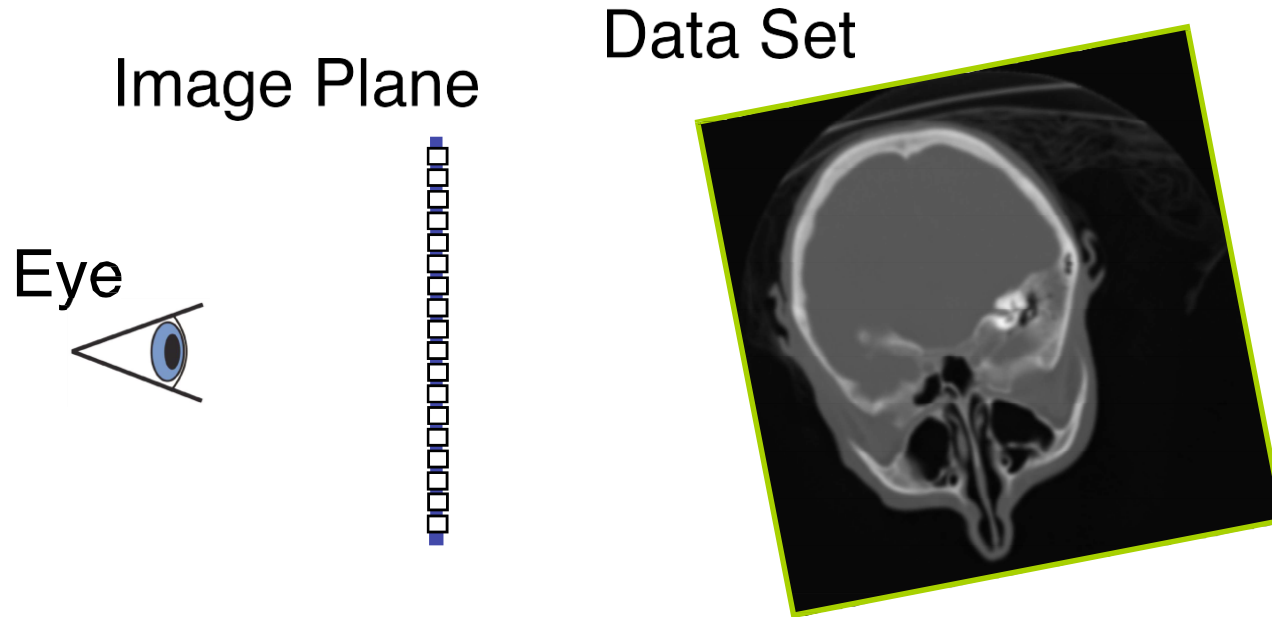


Ray casting volumique

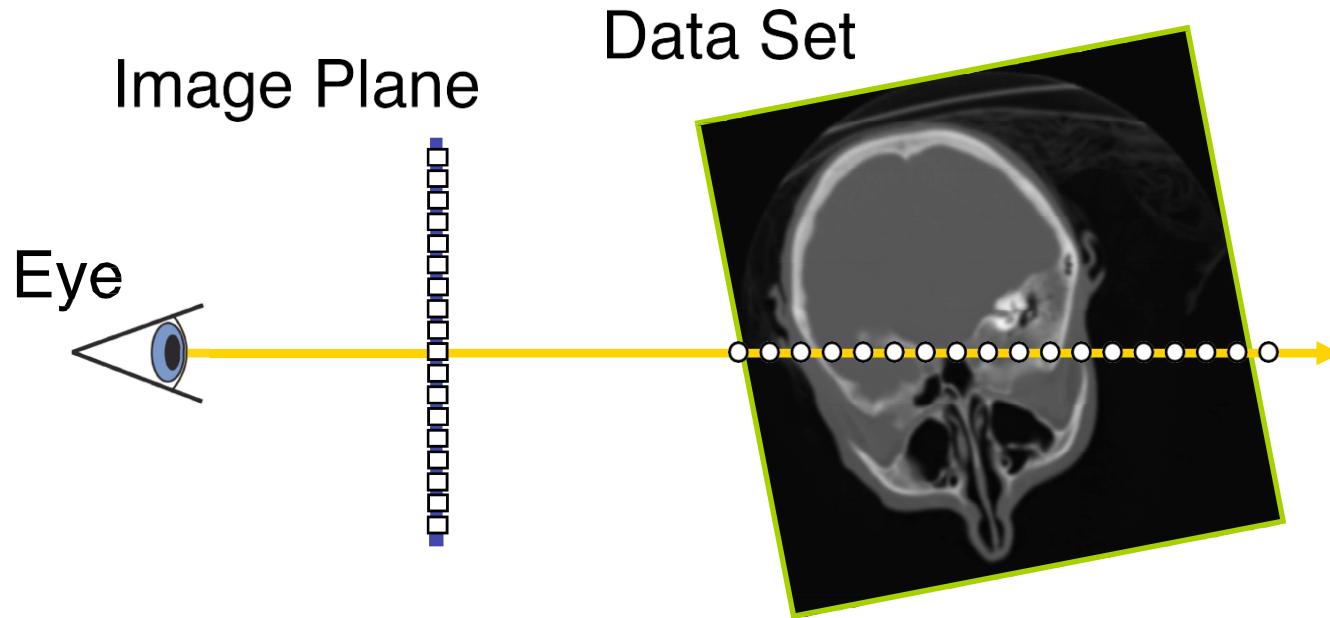
Ray casting à partir de l'image



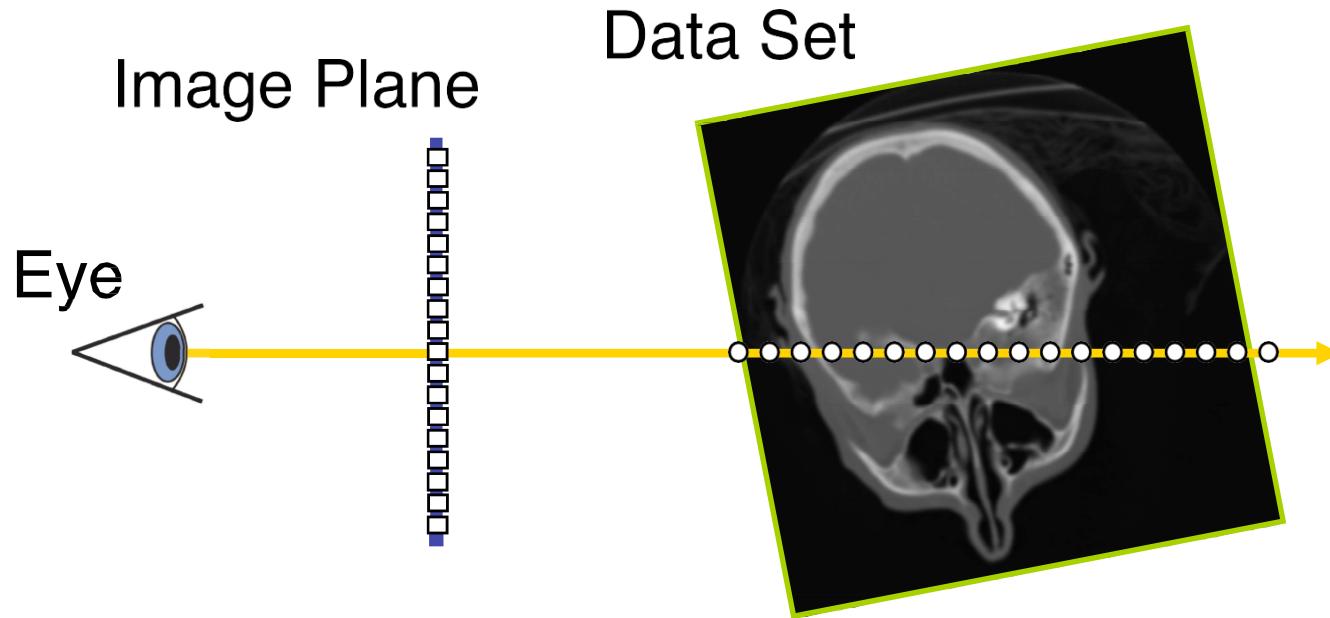
Ray casting à partir de l'image



Ray casting à partir de l'image

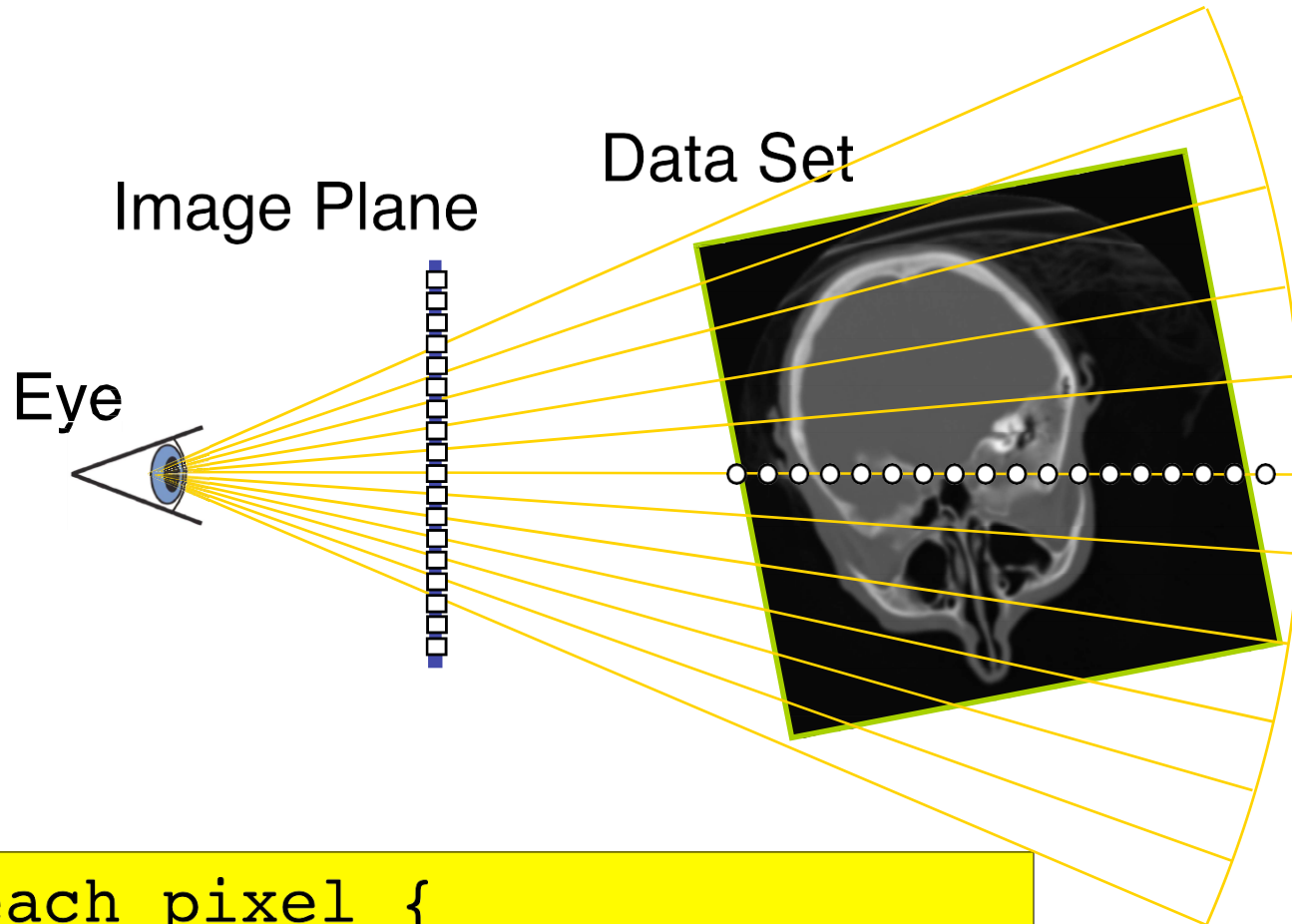


Ray casting à partir de l'image



```
For each pixel {  
    calculate color of the pixel  
}
```

Ray casting à partir de l'image



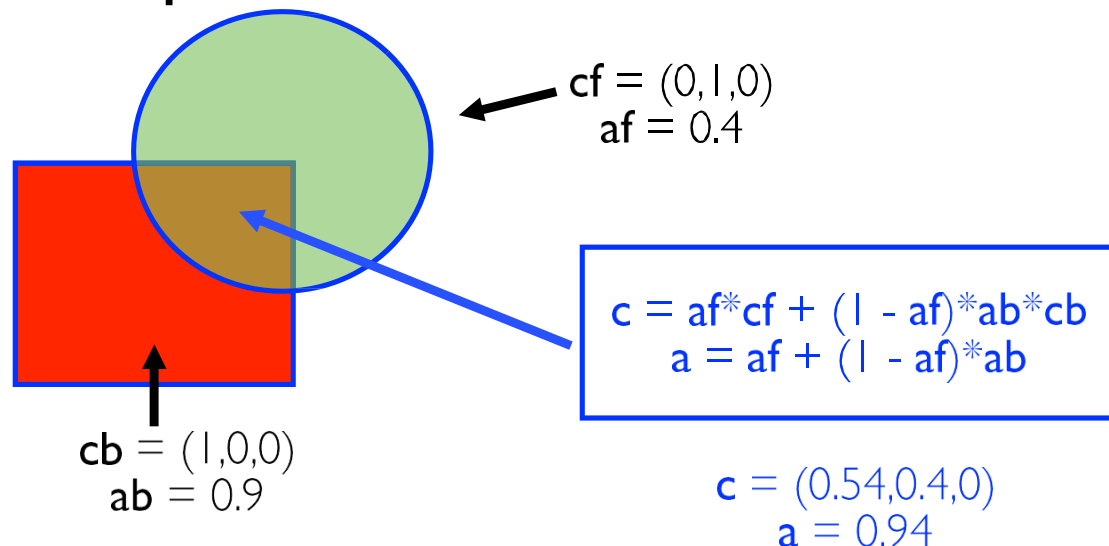
```
For each pixel {  
    calculate color of the pixel  
}
```

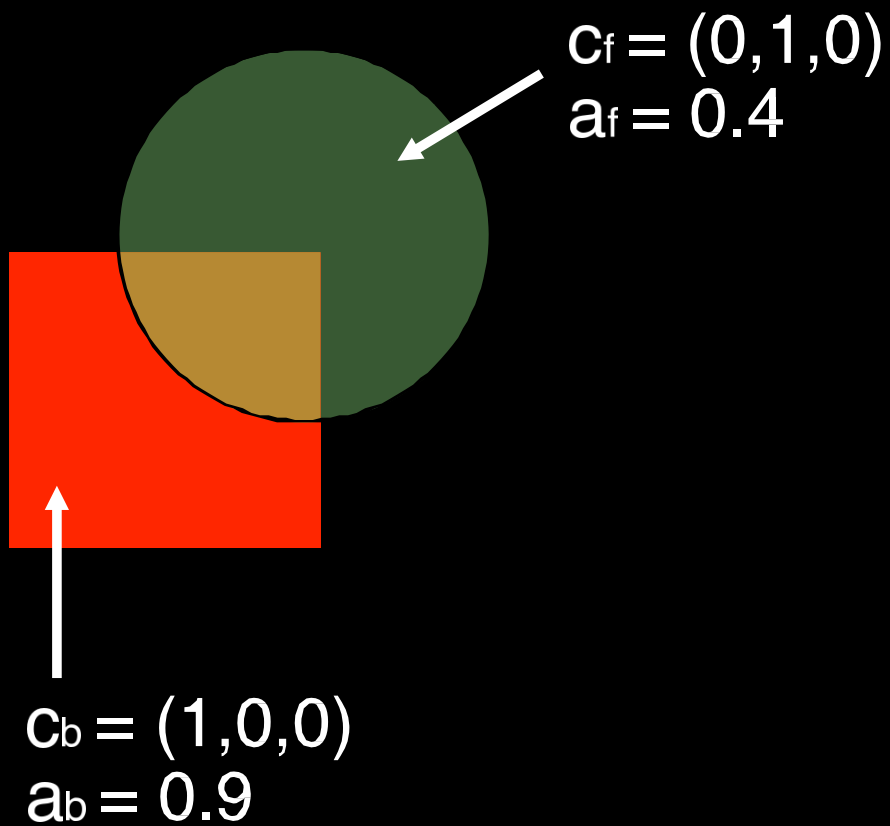
Composition

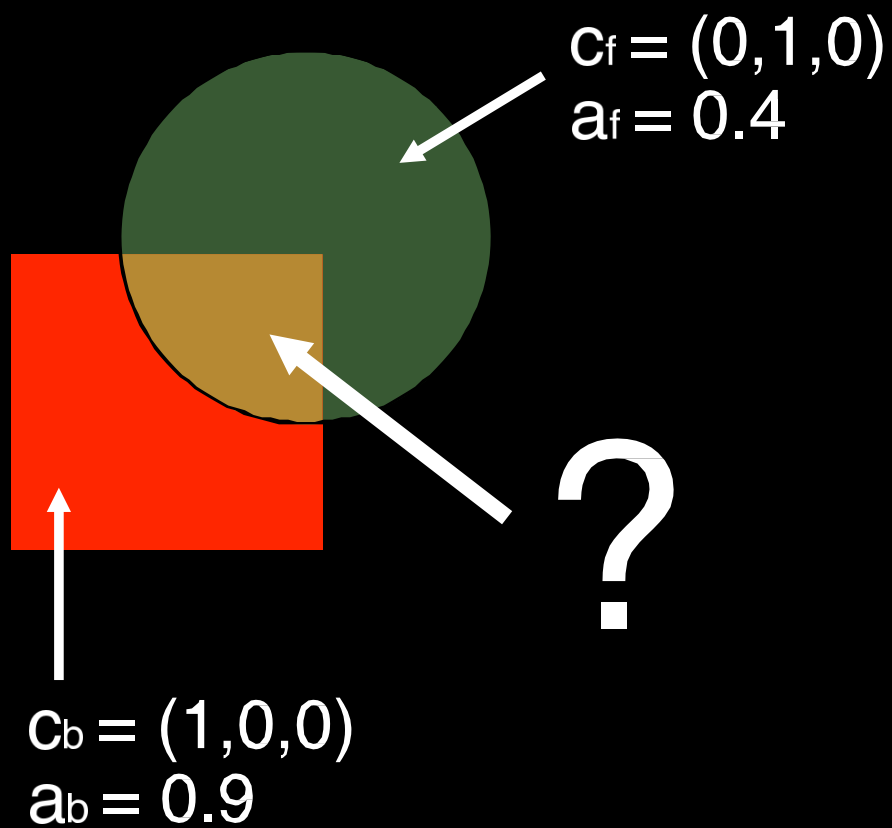
Algorithms fondamentaux

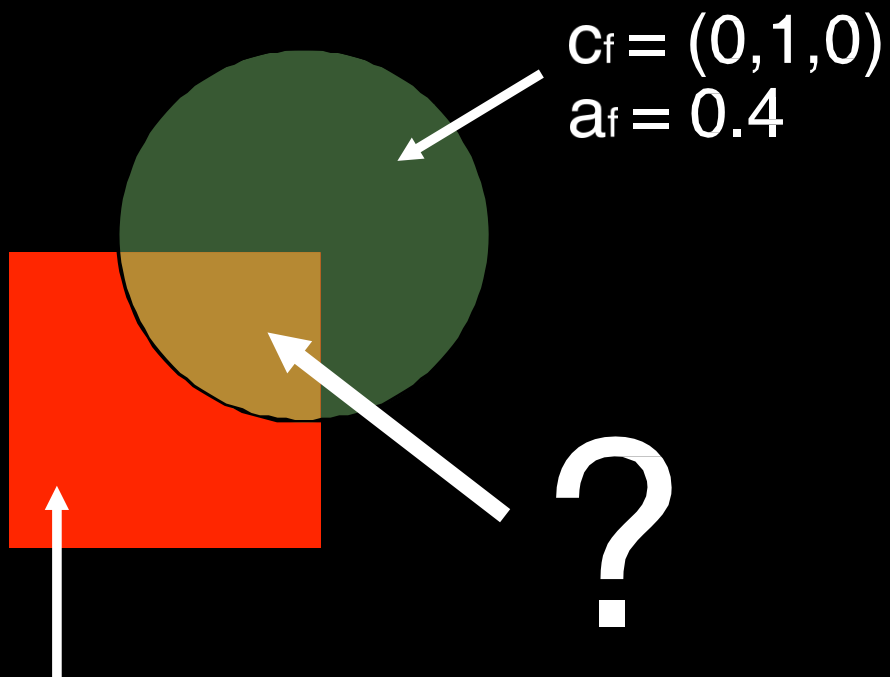
Alpha blending / composition

- Apparence approchée des objets semi-transparent devant un autre objet
- Implémenté avec l'opérateur OVER

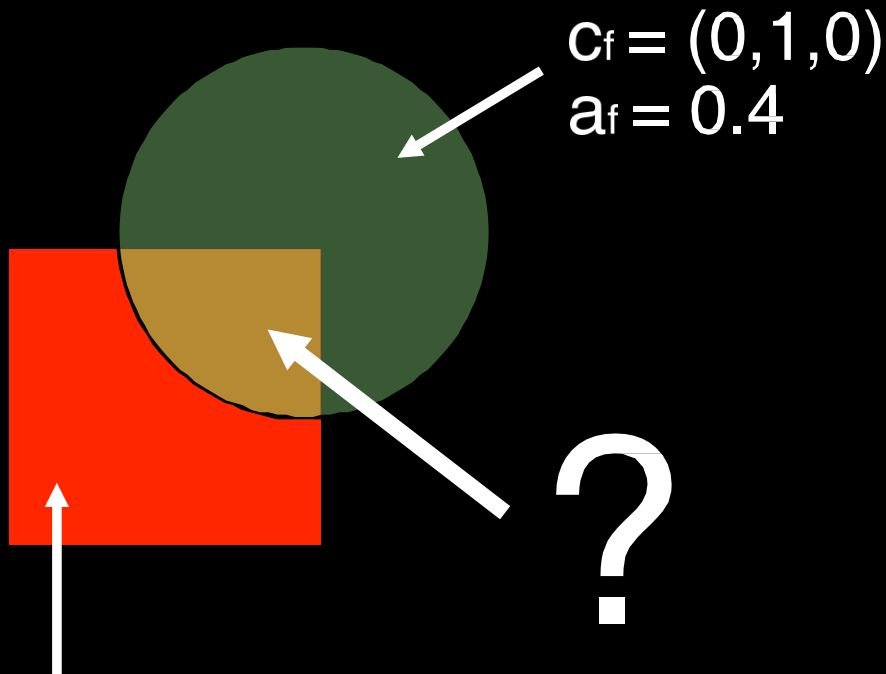








over operator

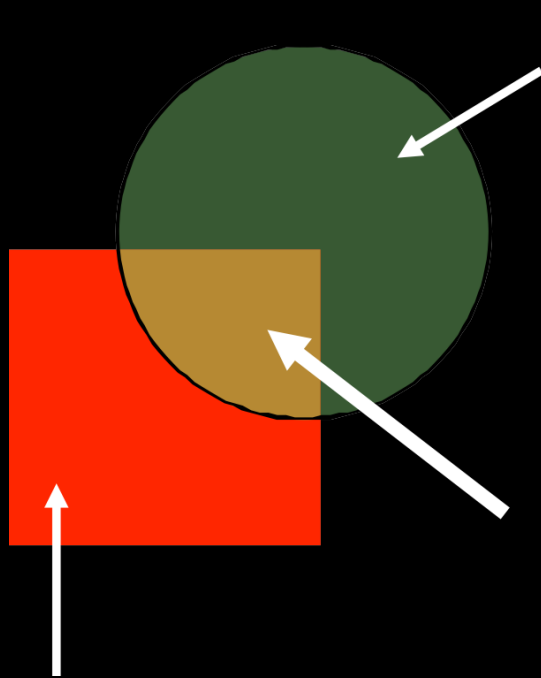


$c_f = (0, 1, 0)$
 $a_f = 0.4$

$c_b = (1, 0, 0)$
 $a_b = 0.9$

over operator

$$c = a_f * c_f + (1 - a_f) * a_b * c_b$$
$$a = a_f + (1 - a_f) * a_b$$



$$c_f = (0, 1, 0)$$
$$a_f = 0.4$$

$$c_{red} = 0.4 * 0 + (1 - 0.4) * 0.9 * 1 = 0.6 * 0.9 = 0.54$$

$$c_{green} = 0.4 * 1 + (1 - 0.4) * 0.9 * 0 = 0.4$$

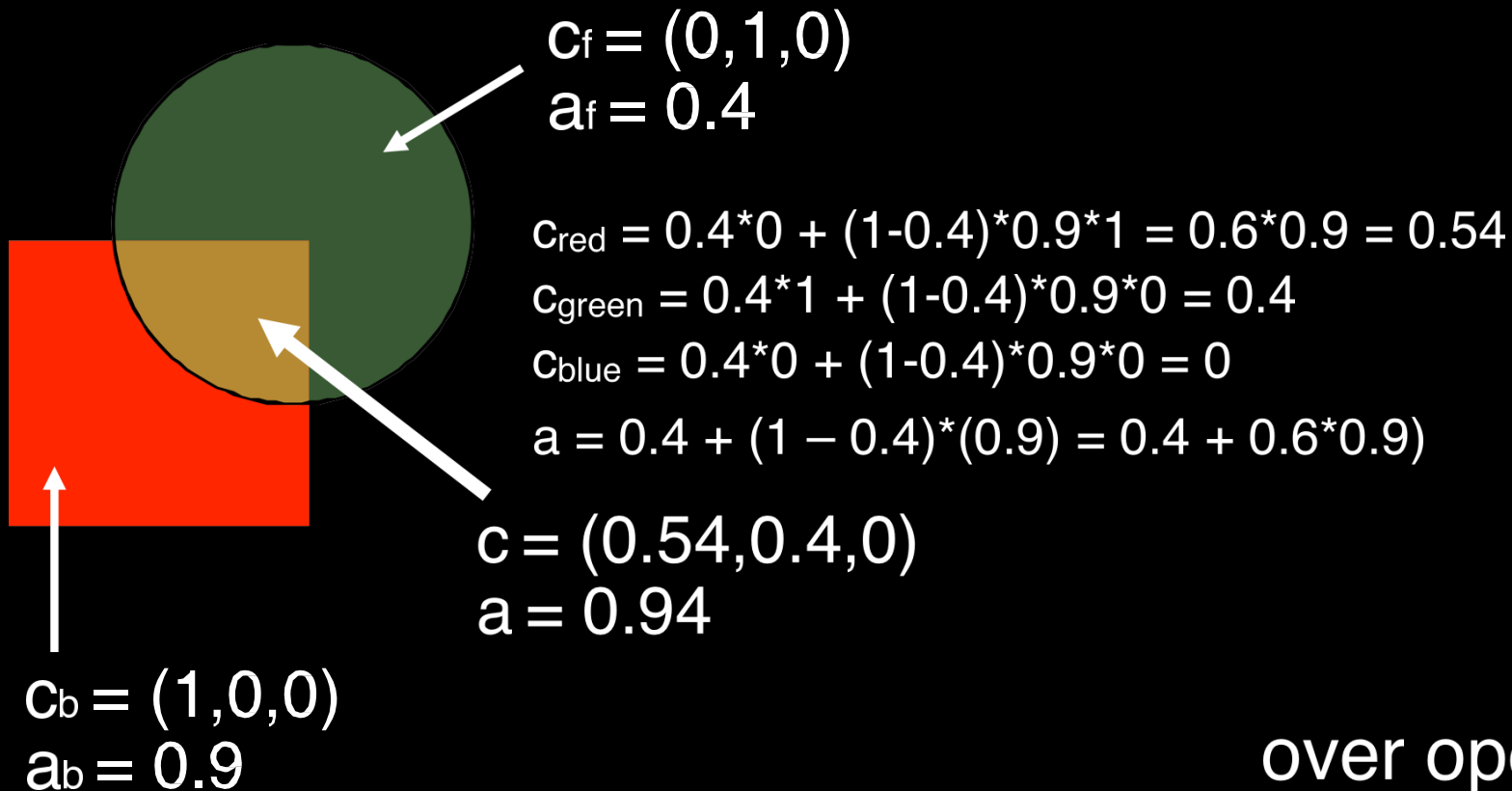
$$c_{blue} = 0.4 * 0 + (1 - 0.4) * 0.9 * 0 = 0$$

$$a = 0.4 + (1 - 0.4) * (0.9) = 0.4 + 0.6 * 0.9$$

$$c_b = (1, 0, 0)$$
$$a_b = 0.9$$

over operator

$$c = a_f * c_f + (1 - a_f) * a_b * c_b$$
$$a = a_f + (1 - a_f) * a_b$$



over operator

$$c = a_f * c_f + (1 - a_f) * a_b * c_b$$
$$a = a_f + (1 - a_f) * a_b$$

$c_f = (0, 1, 1)$
 $a_f = 0.4$

?

$c_1 = (0, 1, 0)$
 $a_1 = 0.4$

$c_b = (0.54, 0.4, 0)$
 $a_b = 0.94$

$c_2 = (1, 0, 0)$
 $a_2 = 0.9$

over operator

$$c = a_f * c_f + (1 - a_f) * a_b * c_b$$
$$a = a_f + (1 - a_f) * a_b$$

$c_f = (0, 1, 1)$
 $a_f = 0.4$

$$c_{red} = 0.4 * 0 + (1 - 0.4) * 0.94 * 0.54 = 0.6 * 0.94 * .54 = 0.30$$

$$c_{green} = 0.4 * 1 + (1 - 0.4) * 0.94 * 0.4 = 0.6 * 0.94 * .4 = 0.23$$

$$c_{blue} = 0.4 * 1 + (1 - 0.4) * 0.94 * 0 = .4$$

$$a = 0.4 + (1 - 0.4) * (0.94) = 0.4 + 0.6 * 0.94 = .964$$

$c_1 = (0, 1, 0)$
 $a_1 = 0.4$

$c_b = (0.54, 0.4, 0)$
 $a_b = 0.94$

$c_2 = (1, 0, 0)$
 $a_2 = 0.9$

over operator

$$c = a_f * c_f + (1 - a_f) * a_b * c_b$$
$$a = a_f + (1 - a_f) * a_b$$

$c_f = (0, 1, 1)$
 $a_f = 0.4$

$c = (0.3, 0.23, 0.4)$
 $a = 0.964$

$c_1 = (0, 1, 0)$
 $a_1 = 0.4$

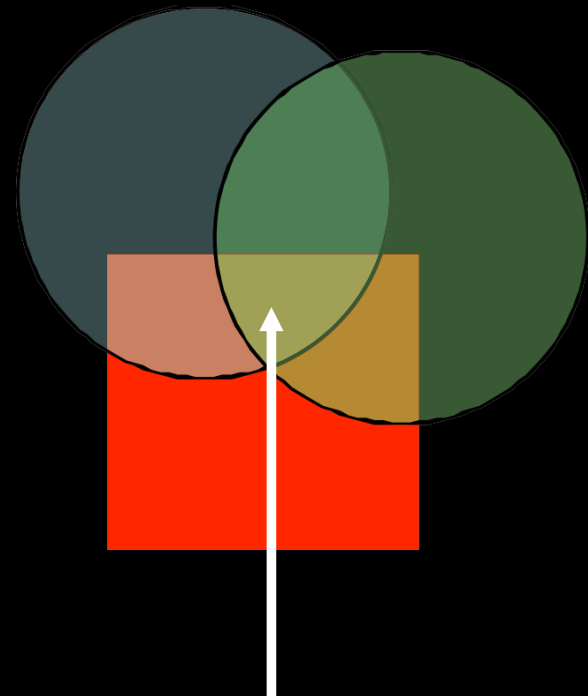
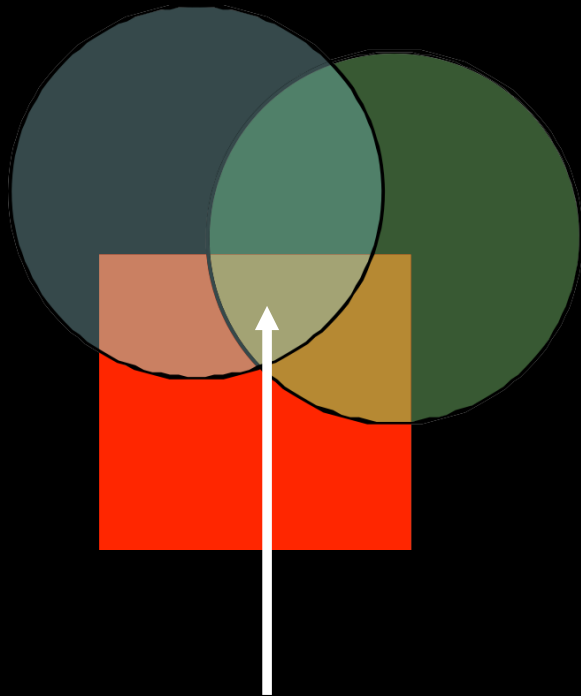
$c_b = (0.54, 0.4, 0)$
 $a_b = 0.94$

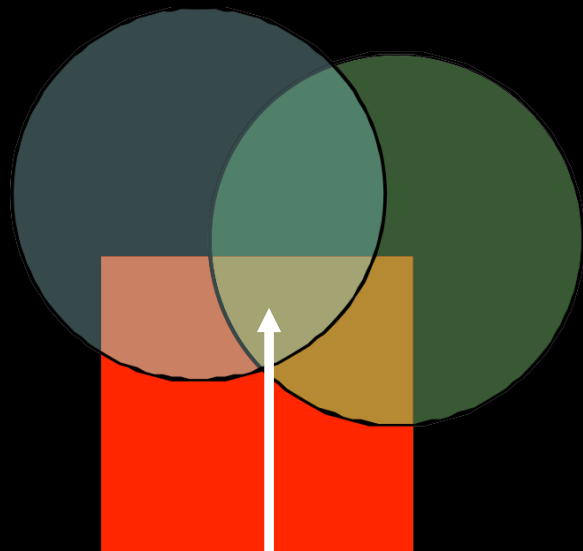
$c_2 = (1, 0, 0)$
 $a_2 = 0.9$

over operator

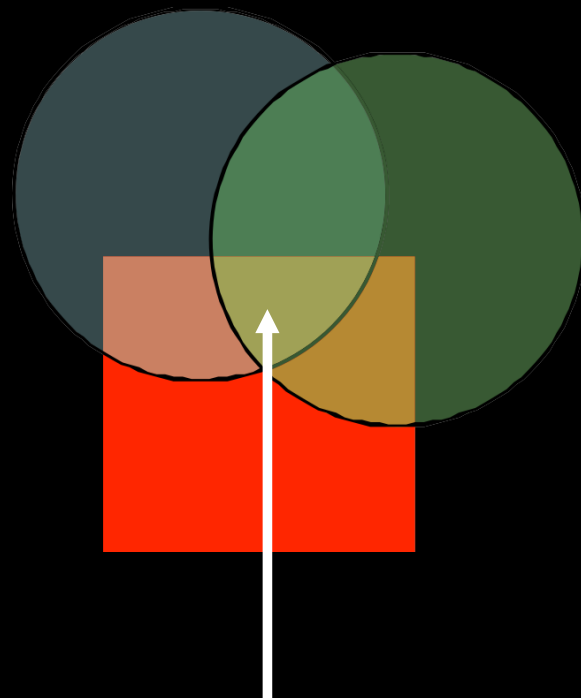
$$c = a_f * c_f + (1 - a_f) * a_b * c_b$$
$$a = a_f + (1 - a_f) * a_b$$

Order Matters!

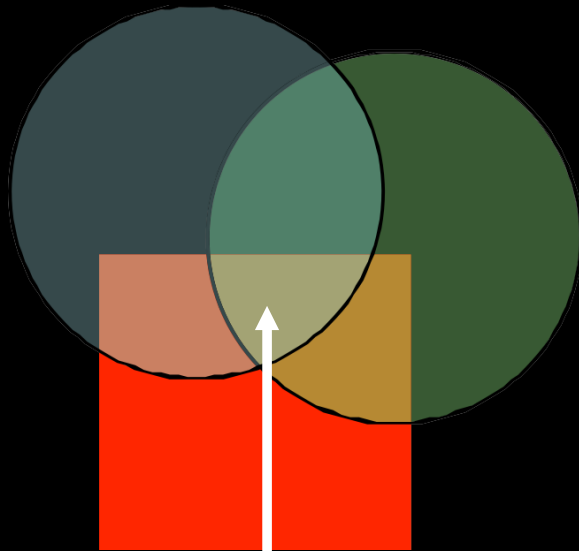




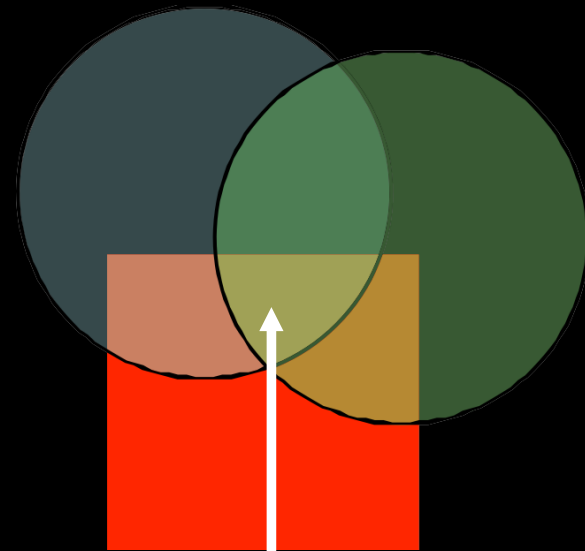
$c = (0.3, 0.23, \underline{0.4})$
 $a = 0.964$



L'ordre est important

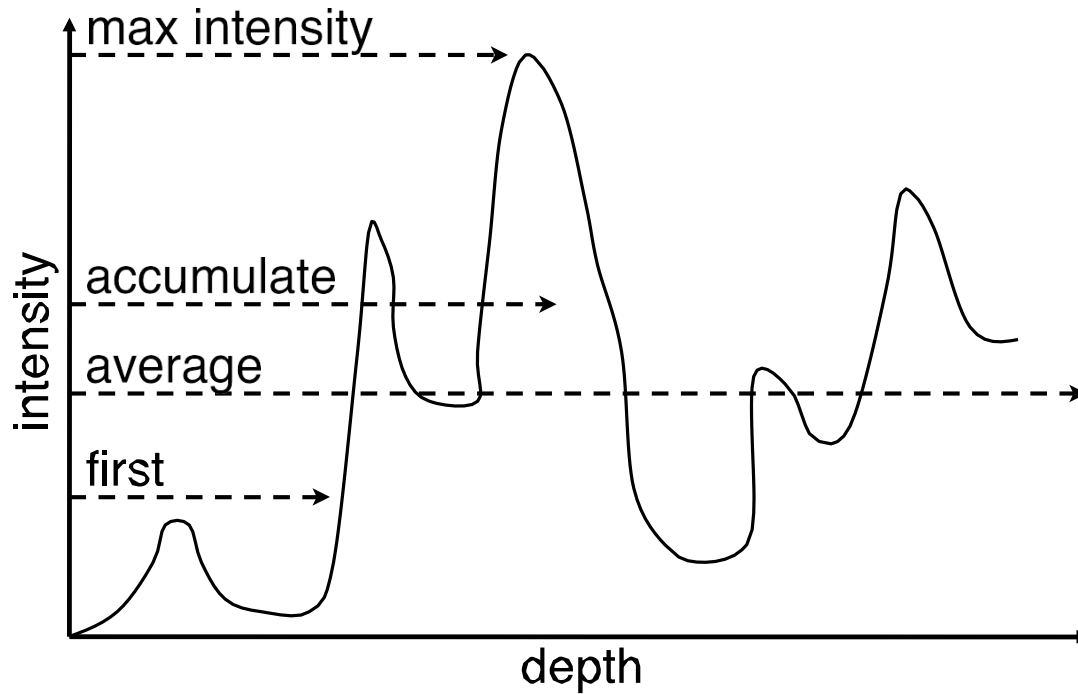


$c = (0.3, 0.23, \underline{0.4})$
 $a = 0.964$

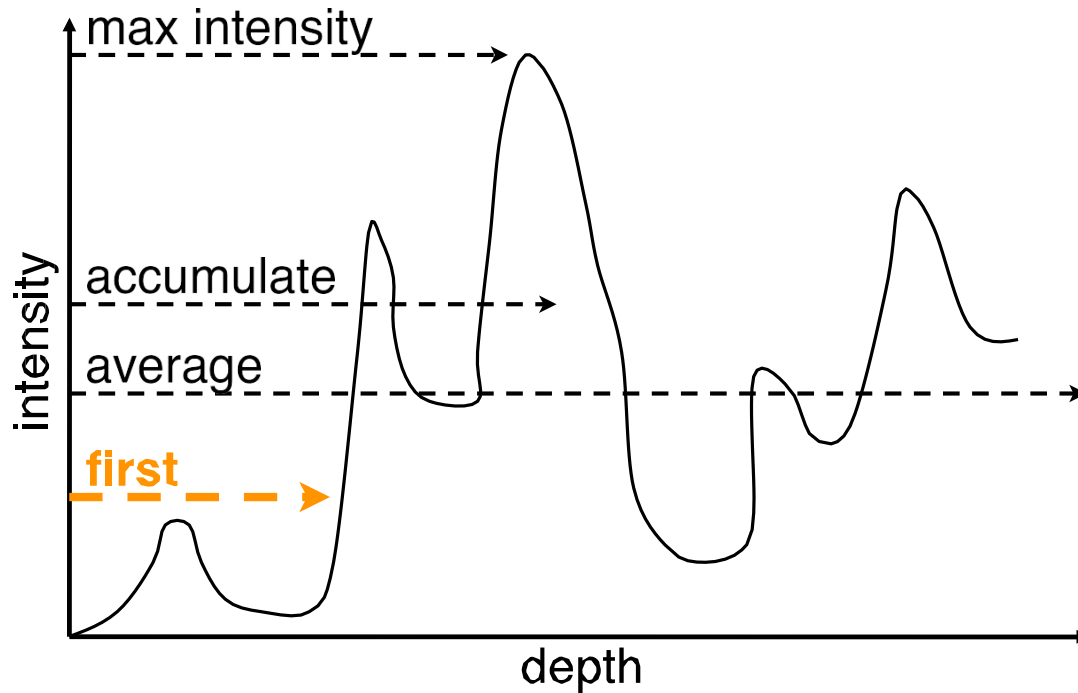


$c = (0.3, 0.23, \underline{0.23})$
 $a = 0.964$

Composition pour les pixels



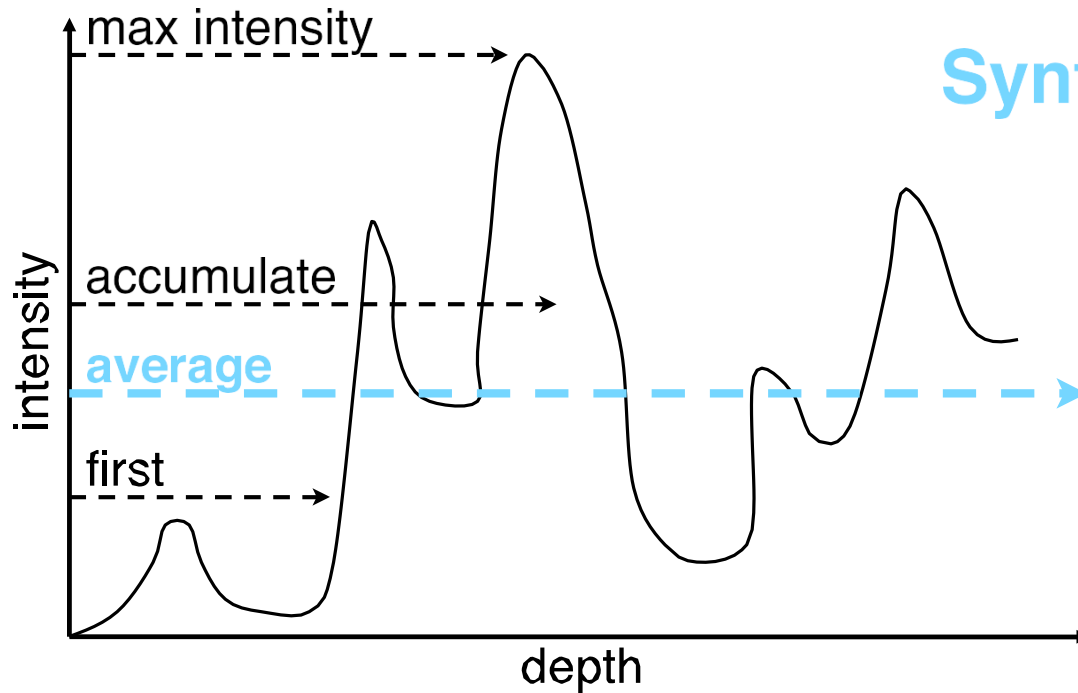
Composition pour les pixels



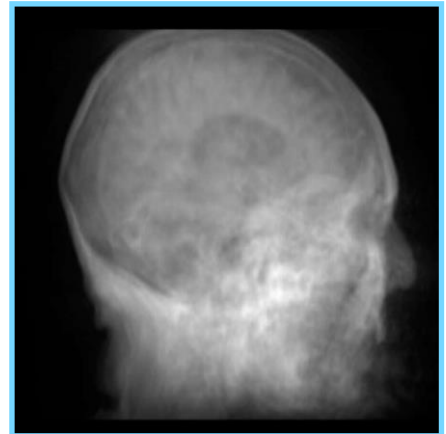
Exact Isosurface



Composition pour les pixels

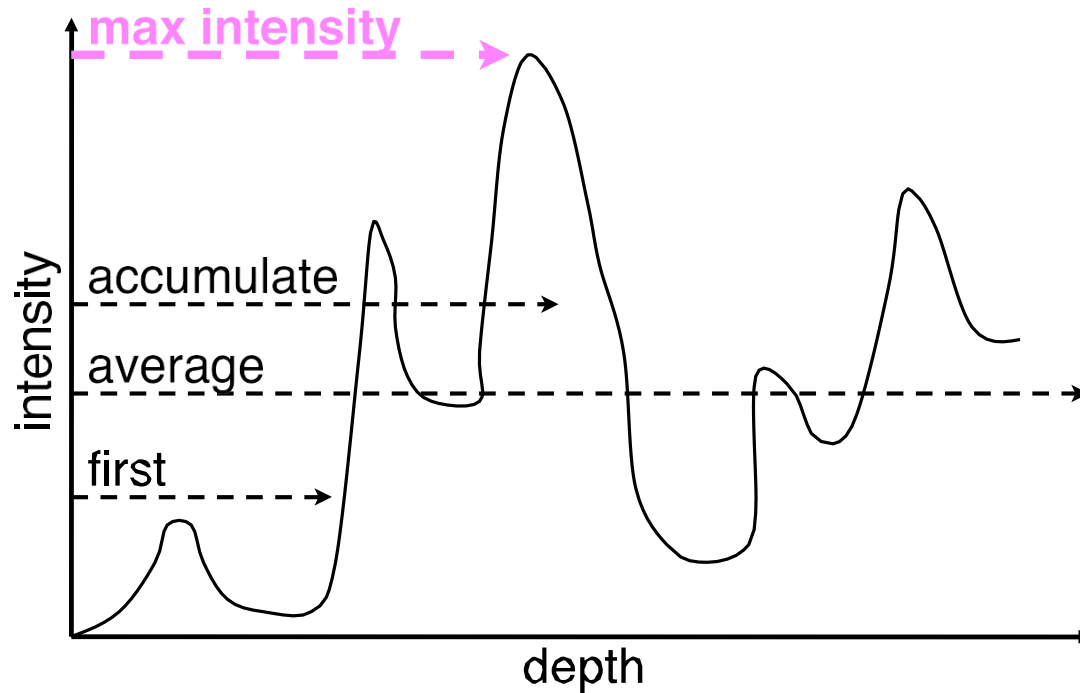


Synthetic Reprojection

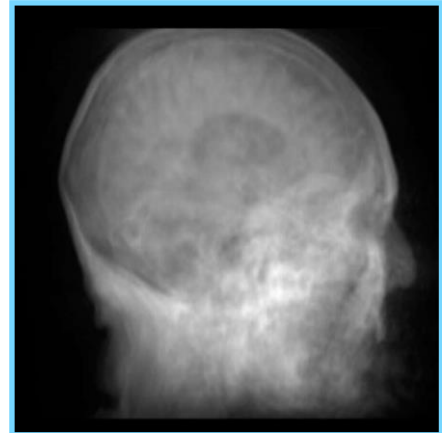


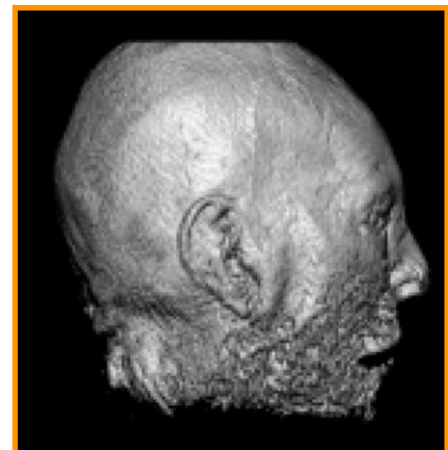
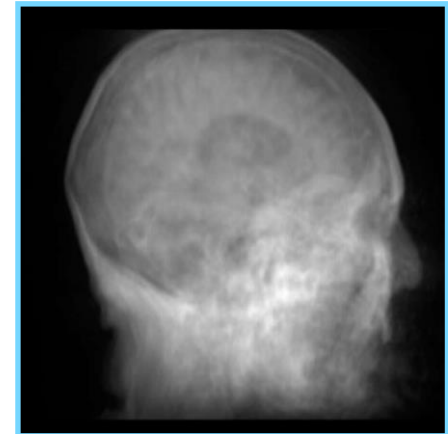
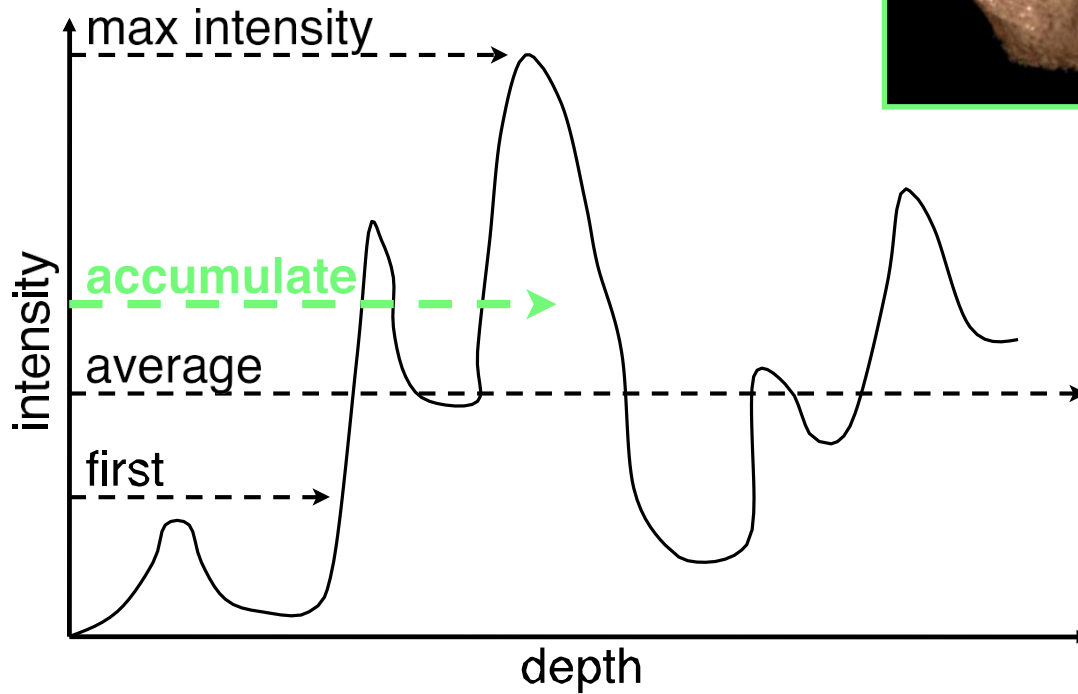
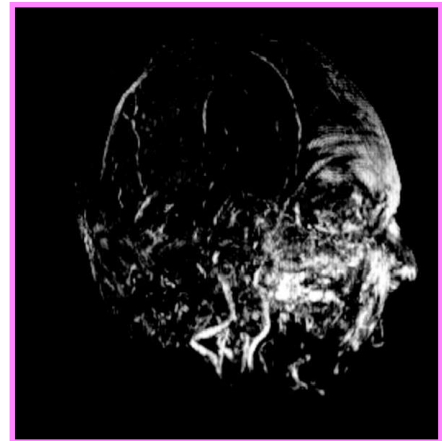
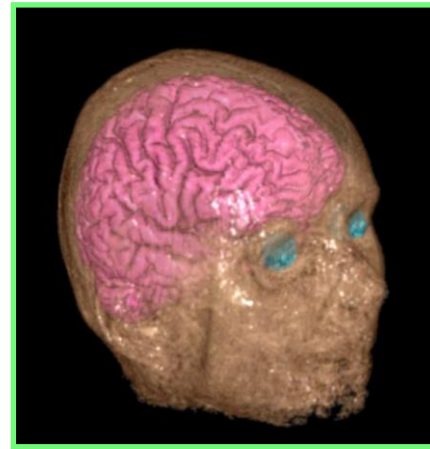
Similar to X-rays

maximum intensity projection (MIP)



Used in PET and Magnetic
Resonance Angiograms



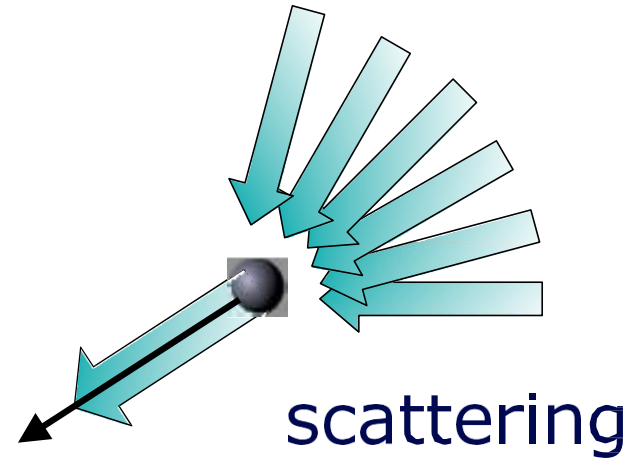
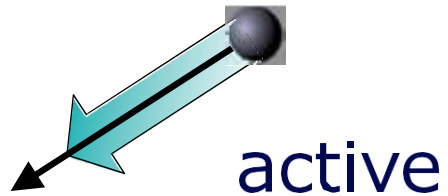


color to distinguish structures
opacity to show inside

Modéliser la lumière

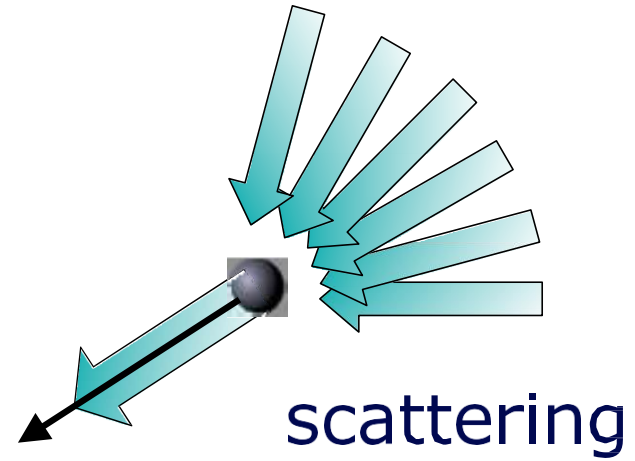
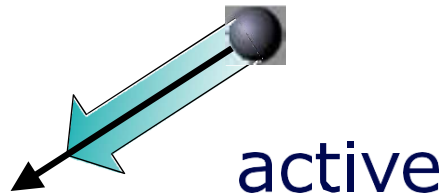
Modèles optique

Emission

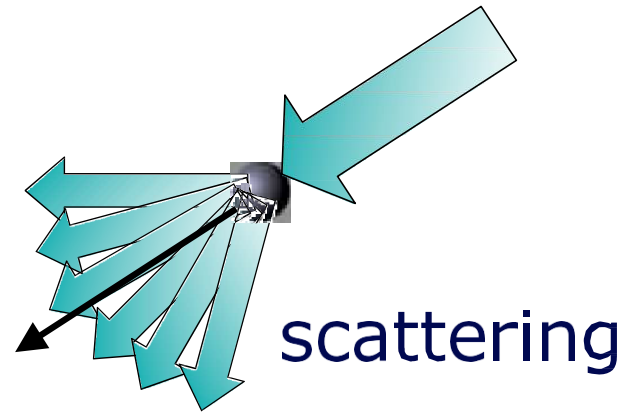
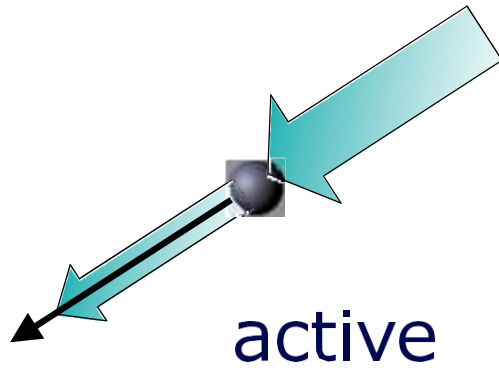


Modèles optique

Emission

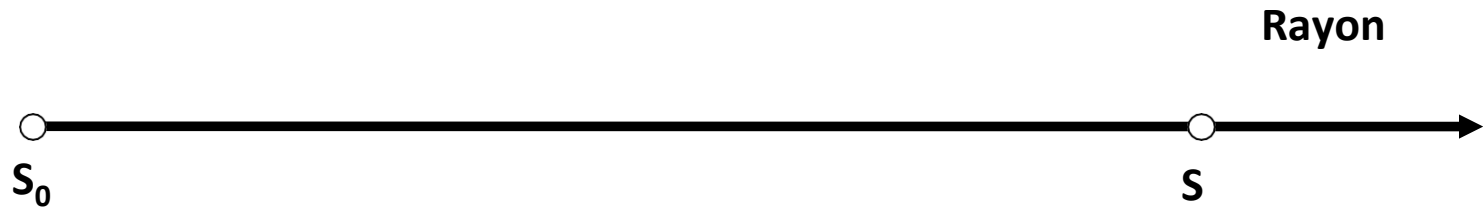


Absorption



Que doit-on intégrer ?

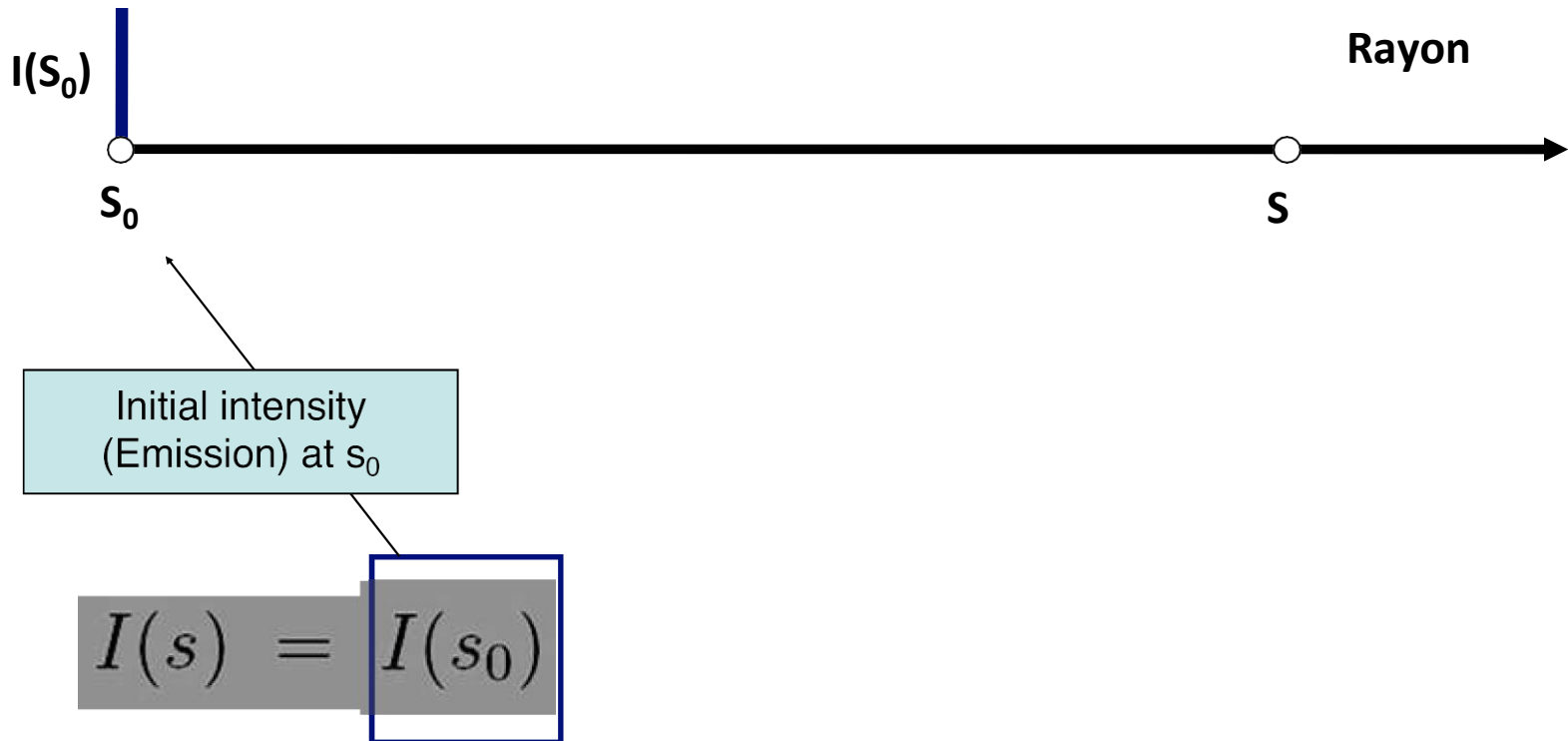
- Physiquement correct :
 - Emission et absorption de la lumière



$$I(s) =$$

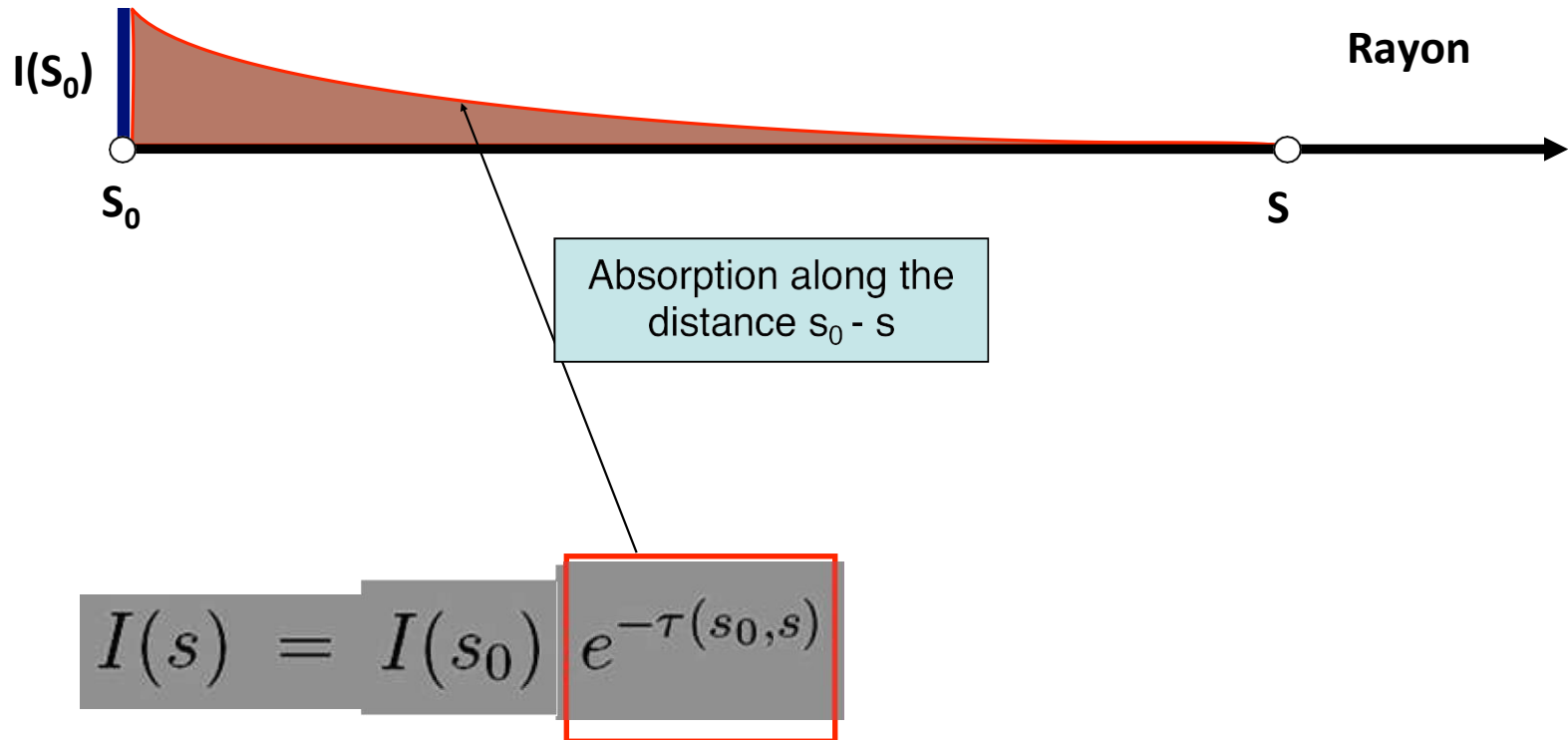
Que doit-on intégrer ?

- Physiquement correct :
 - Emission et absorption de la lumière



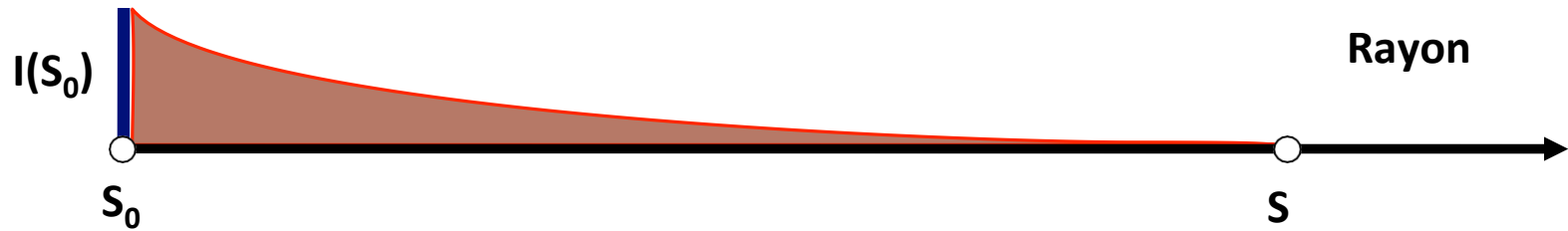
Que doit-on intégrer ?

- Physiquement correct :
 - Emission et absorption de la lumière



Que doit-on intégrer ?

- Physiquement correct :
 - Emission et absorption de la lumière



$$I(s) = I(s_0) e^{-\tau(s_0, s)}$$

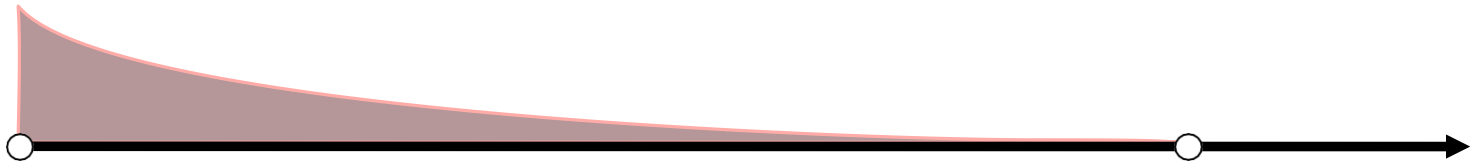
Optical Depth τ

Absorption κ

$$\tau(s_1, s_2) = \int_{s_1}^{s_2} \kappa(s) ds.$$

Que doit-on intégrer ?

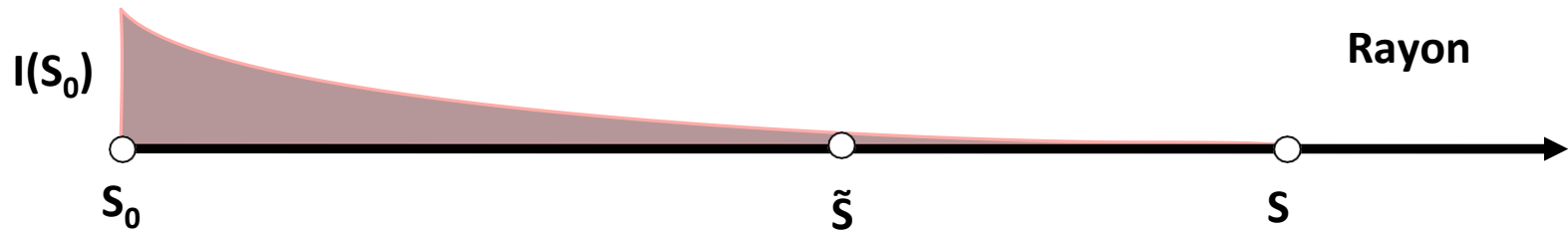
- Physiquement correct :
 - Emission et absorption de la lumière



$$I(s) = I(s_0) e^{-\tau(s_0, s)}$$

Que doit-on intégrer ?

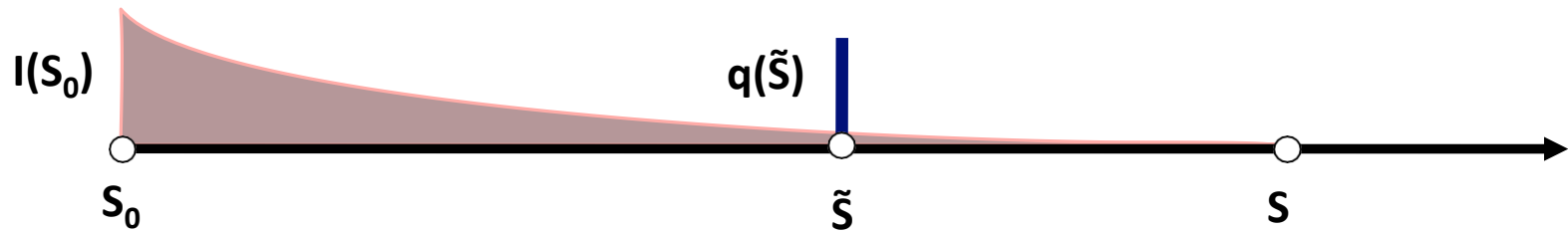
- Physiquement correct :
 - Emission et absorption de la lumière



$$I(s) = I(s_0) e^{-\tau(s_0, s)}$$

Que doit-on intégrer ?

- Physiquement correct :
 - Emission et absorption de la lumière

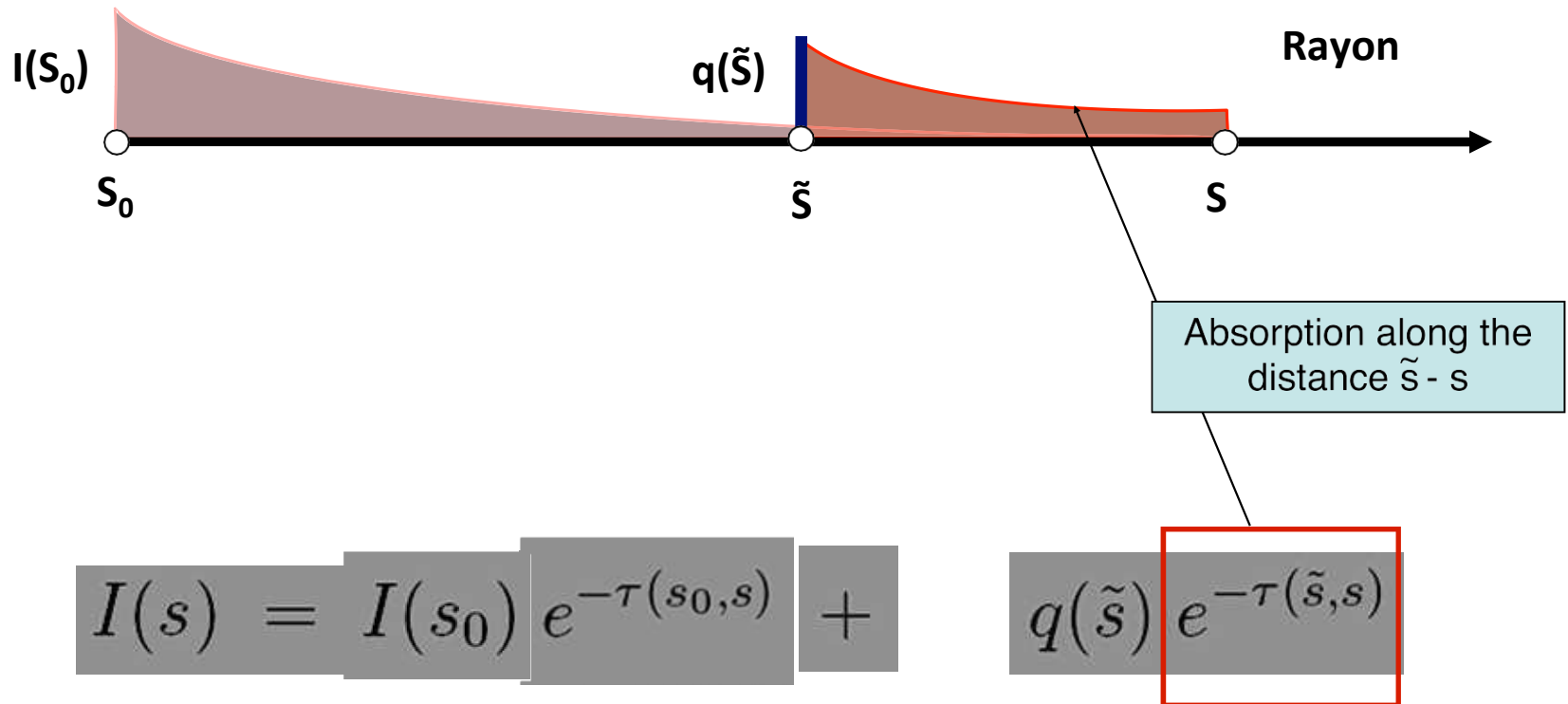


Active Emission
at $\tilde{s}$

$$I(s) = I(s_0) e^{-\tau(s_0, s)} + q(\tilde{s}) e^{-\tau(\tilde{s}, s)}$$

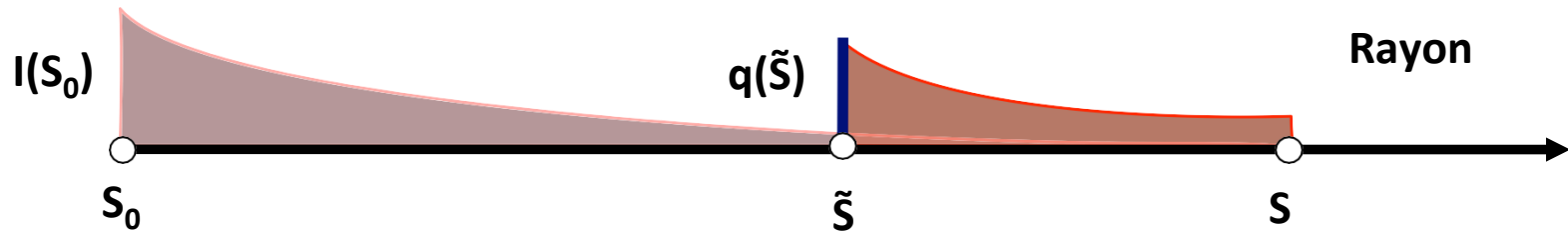
Que doit-on intégrer ?

- Physiquement correct :
 - Emission et absorption de la lumière



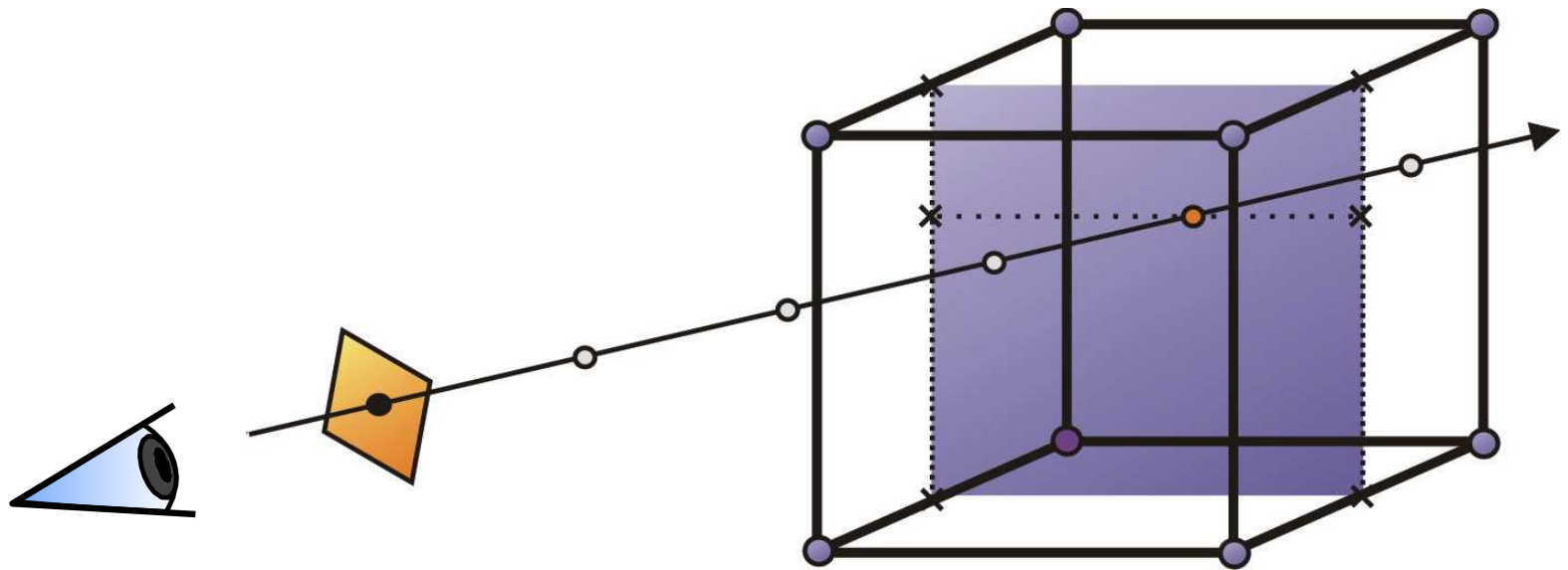
Que doit-on intégrer ?

- Physiquement correct :
 - Emission et absorption de la lumière



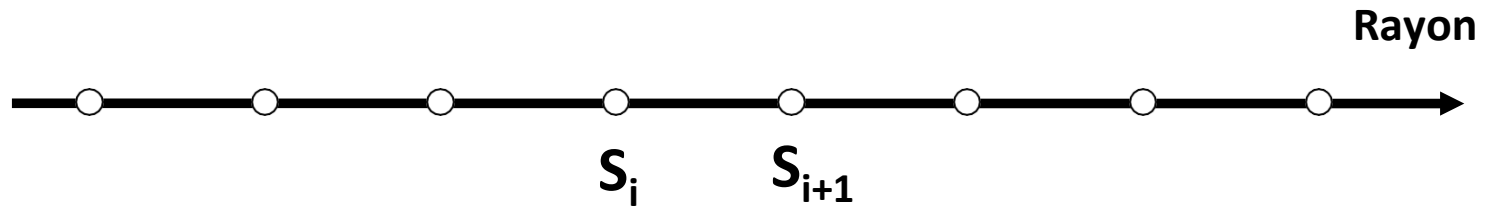
$$I(s) = I(s_0) e^{-\tau(s_0, s)} + \int_{s_0}^s q(\tilde{s}) e^{-\tau(\tilde{s}, s)} d\tilde{s}$$

- Approximation numérique de l'intégrale
- Idée : resampler le volume à intervalles équidistants
- Faire une interpolation trilinéaire



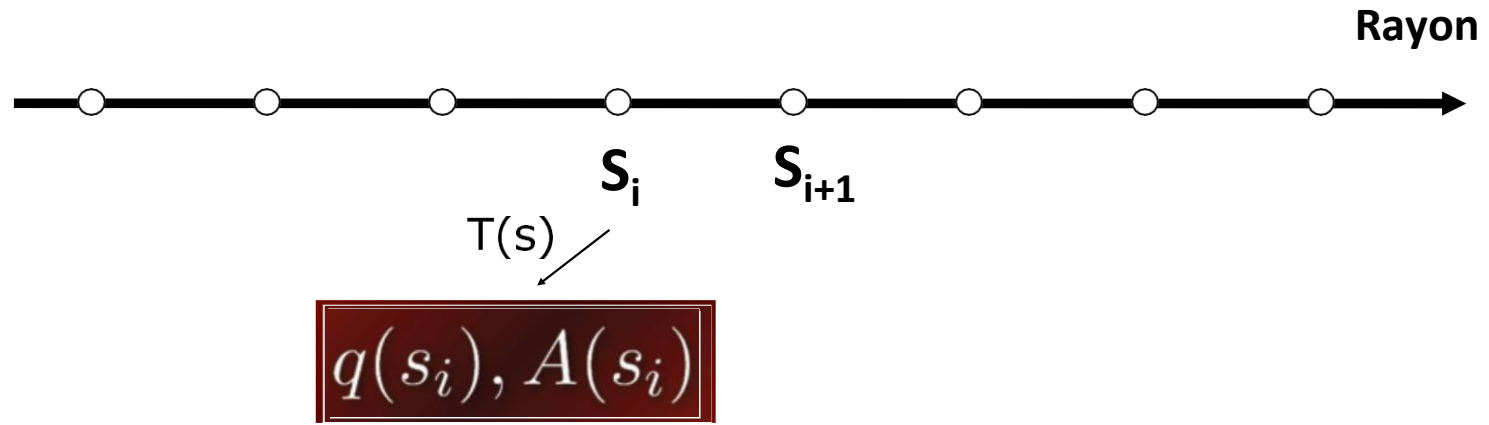
Solution discrète

- Resampler le champ scalaire à des positions discrètes sur le rayon de vue



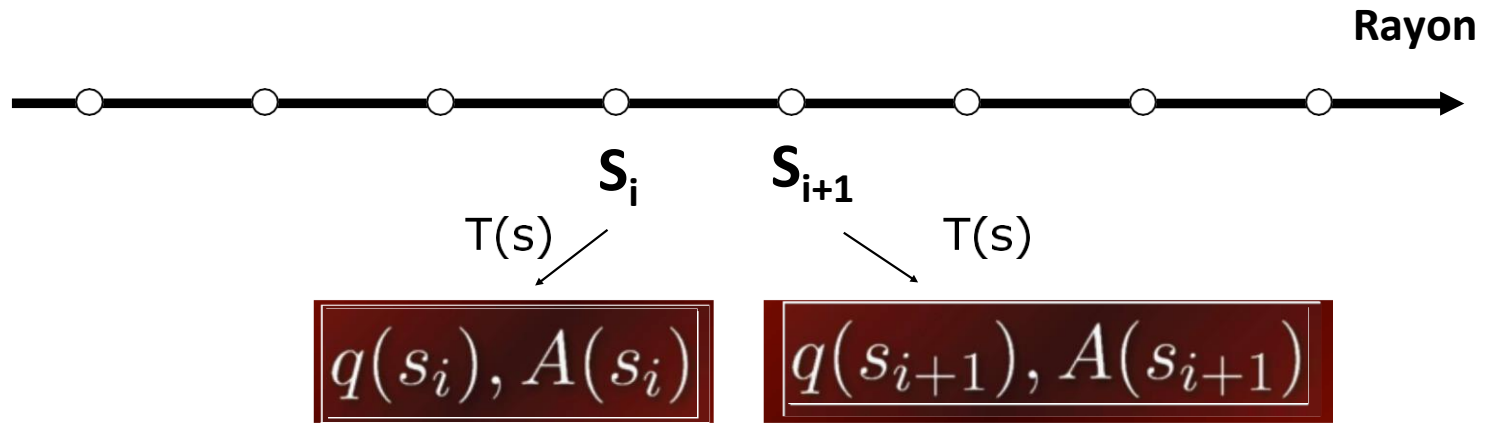
Solution discrète

- Resampler le champ scalaire à des positions discrètes sur le rayon de vue



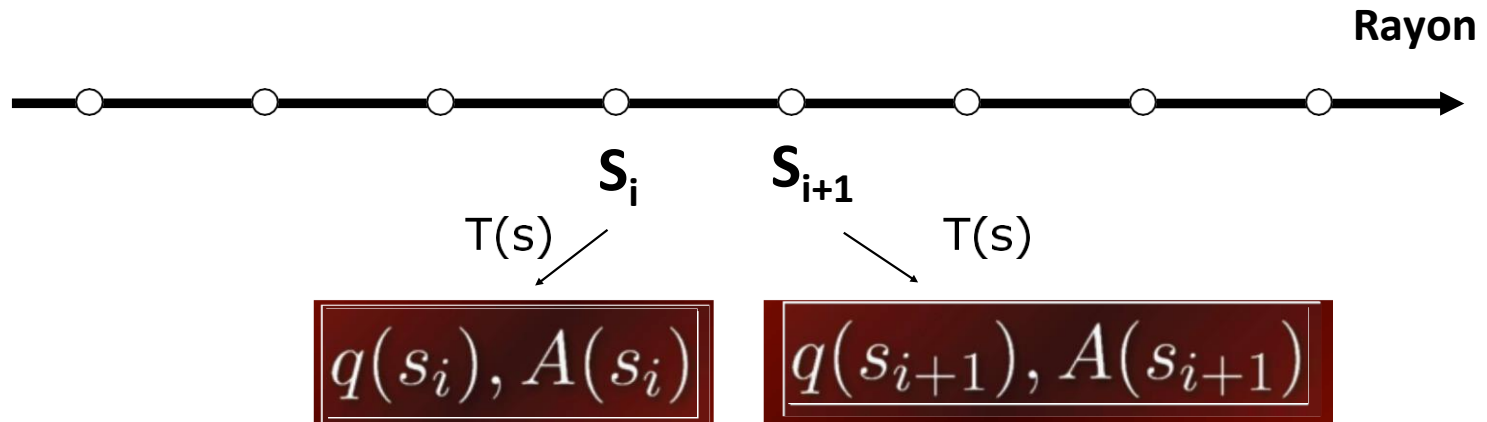
Solution discrète

- Resampler le champ scalaire à des positions discrètes sur le rayon de vue



Solution discrète

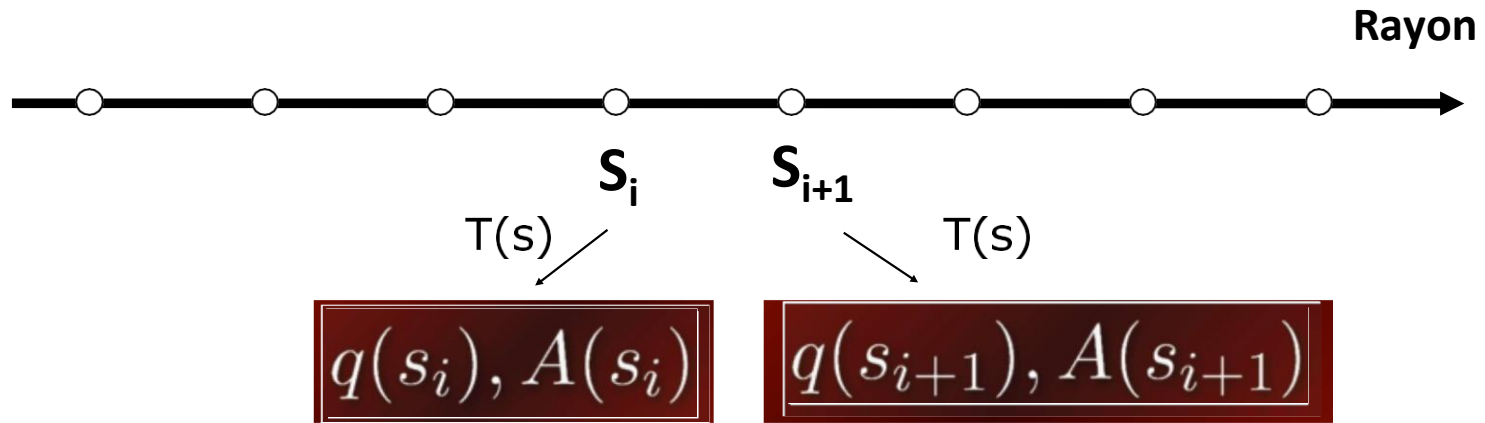
- Resampler le champ scalaire à des positions discrètes sur le rayon de vue



$$I(s_{i+1}) =$$

Solution discrète

- Resampler le champ scalaire à des positions discrètes sur le rayon de vue



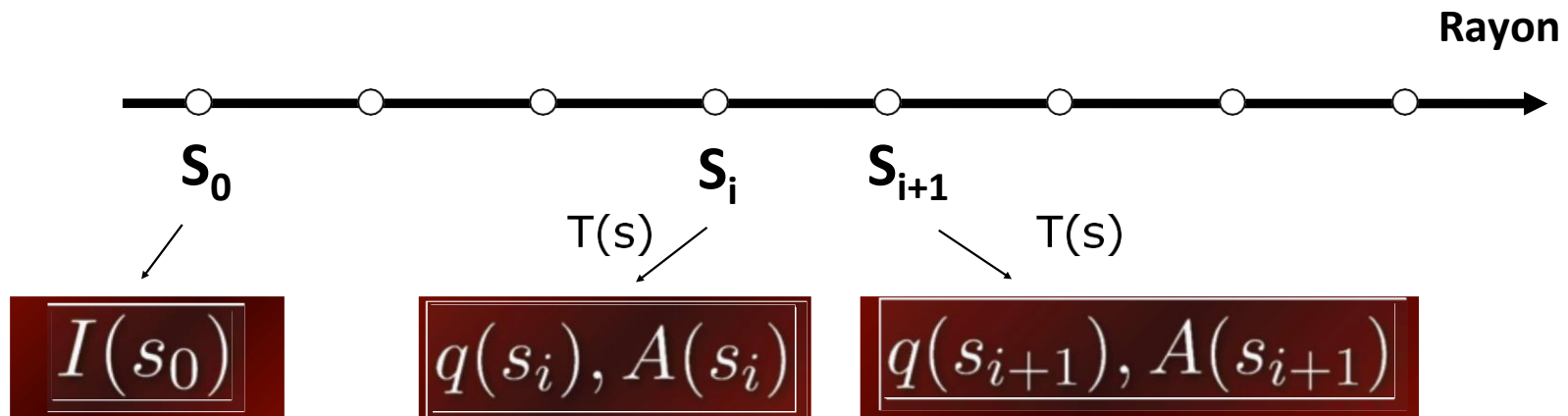
Composition « back-to-front » avec

$$\alpha = A(s_i)$$

$$I(s_{i+1}) = \alpha \cdot q(s_{i+1}) + (1 - \alpha)I(s_i)$$

Solution discrète

- Resampler le champ scalaire à des positions discrètes sur le rayon de vue



Composition « back-to-front » avec $\alpha = A(s_i)$

$$I(s_{i+1}) = \alpha \cdot q(s_{i+1}) + (1 - \alpha)I(s_i)$$

Alpha blending

- **Approximation numérique** de l'intégrale de rendu volumique
- Composition « back-to-front »

$$C'_i = C_i + (1 - A_i)C'_{i+1}$$

- Nouvelle couleur composée = couleur à la position courante + couleur composée précédente
- **Couleur pondérées par l'opacité**

Raycasting

- Avantages

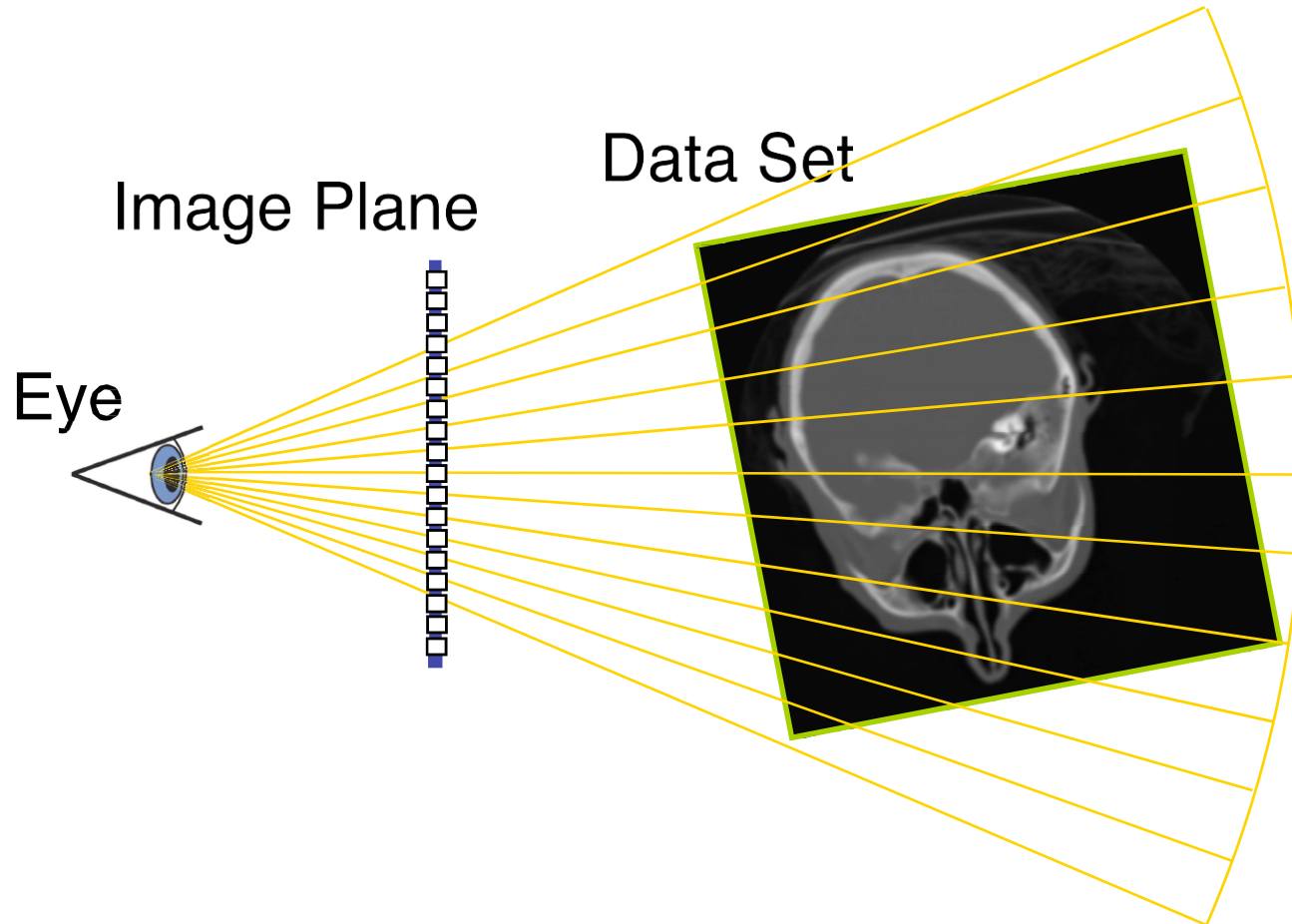
- Algorithme simple
- Intrinsèquement parallèle
- On peut ajouter des options (comme un ray tracer)

- Désavantages

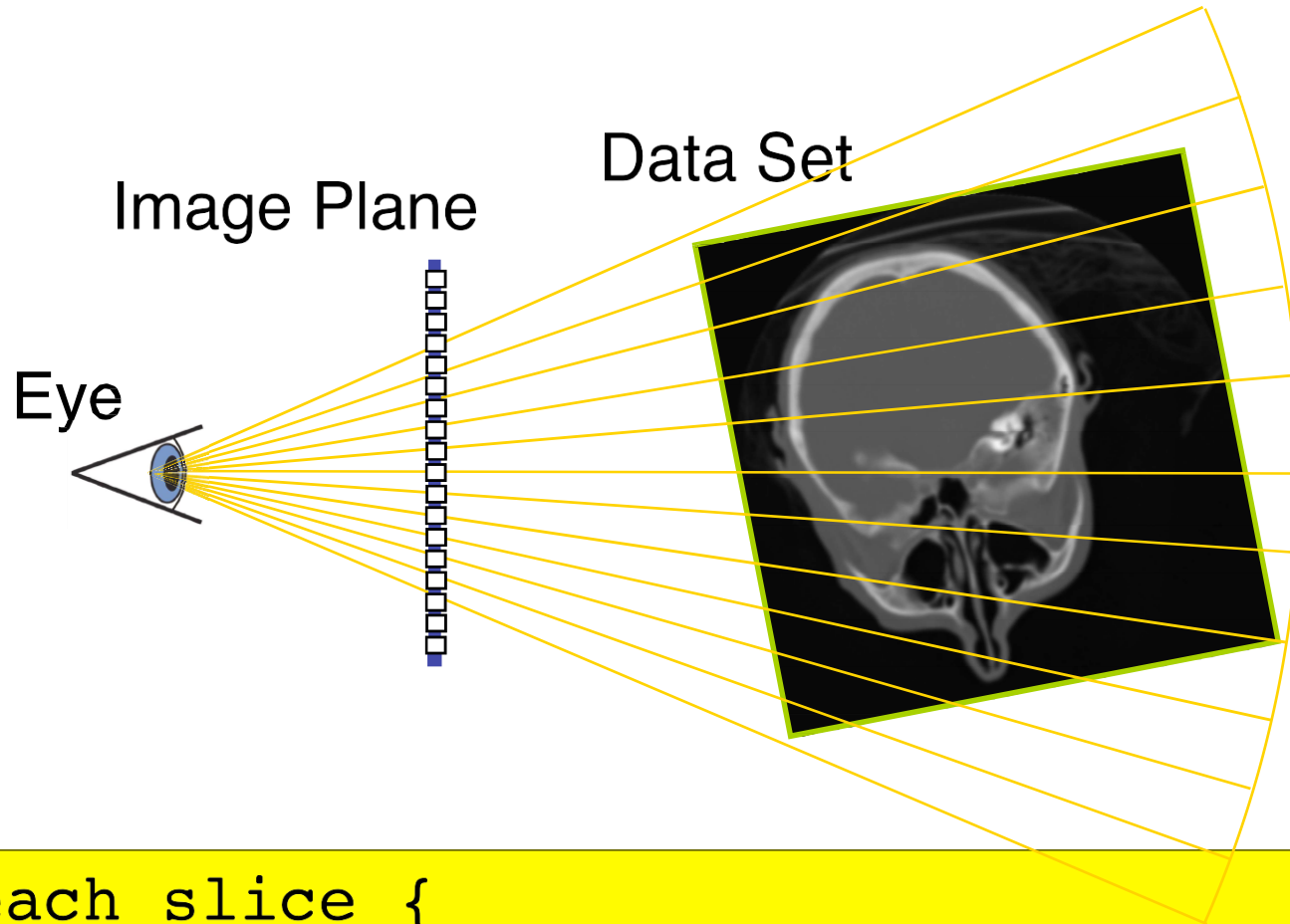
- Lent (beaucoup de rayons et d'échantillons)
- Echantillonnage doit être dense
- Demande la donnée en mémoire

Approches ordonnancement d'objets

Approche objet

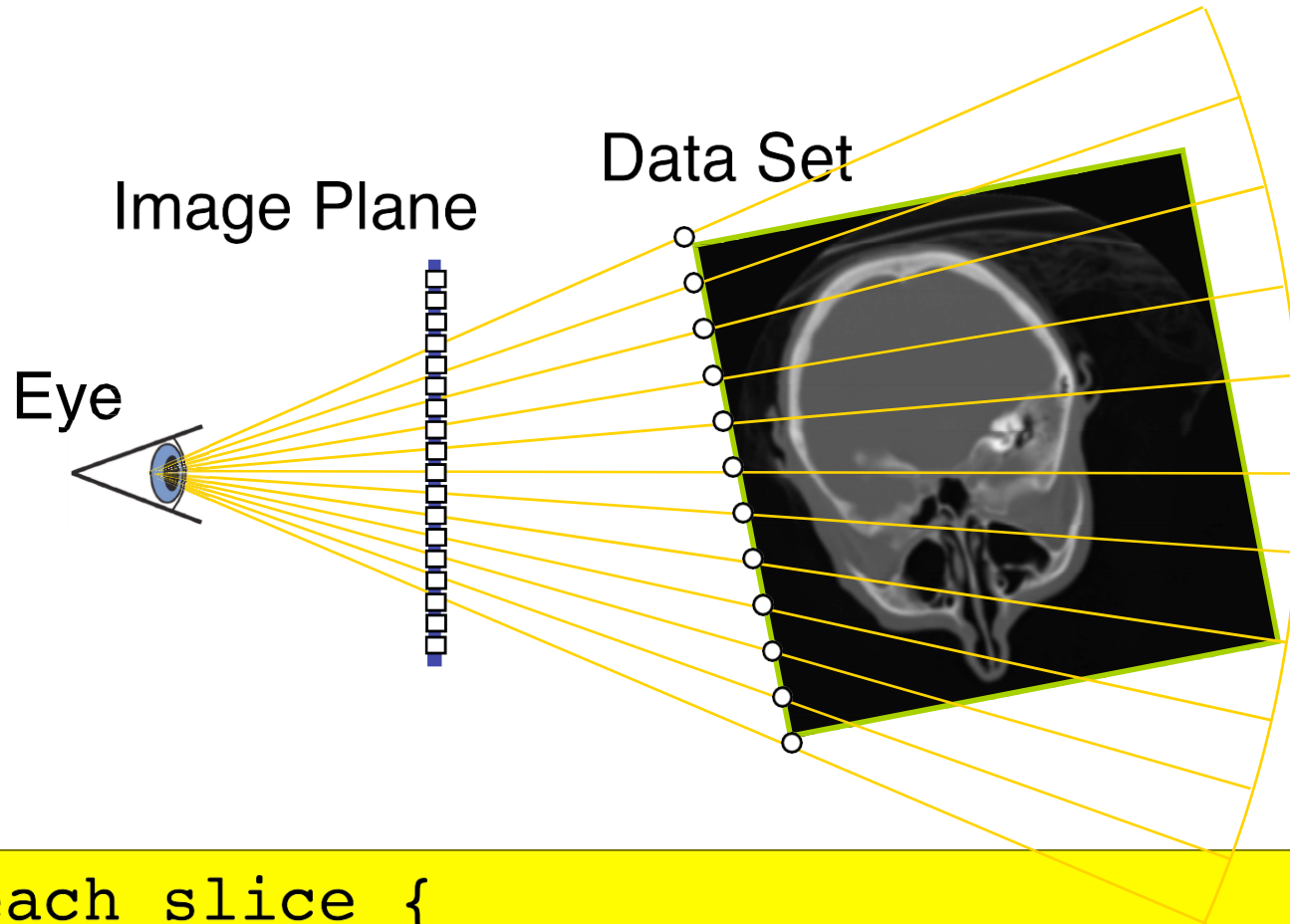


Approche objet



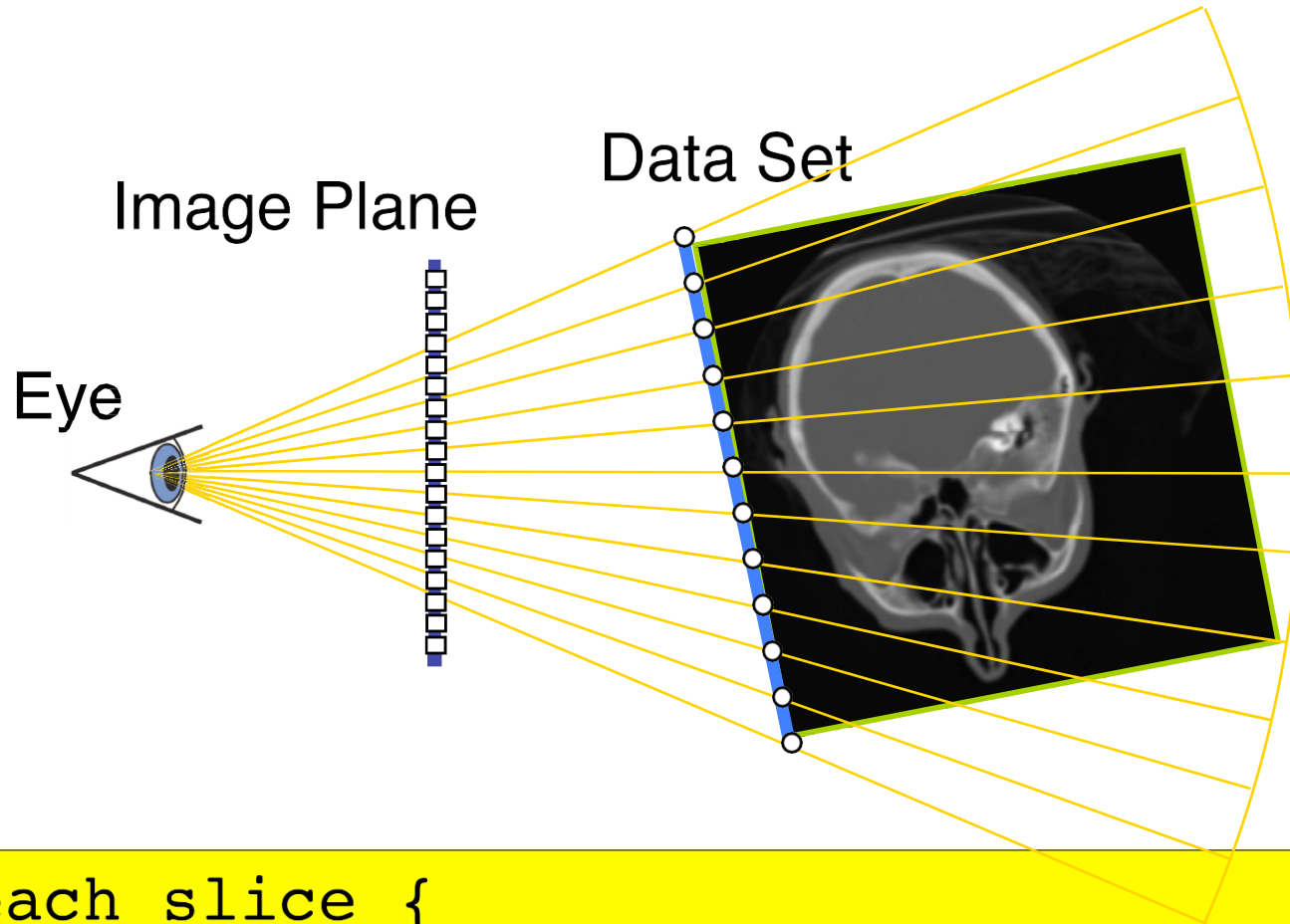
```
For each slice {  
    calculate contribution to the image  
}
```

Approche objet

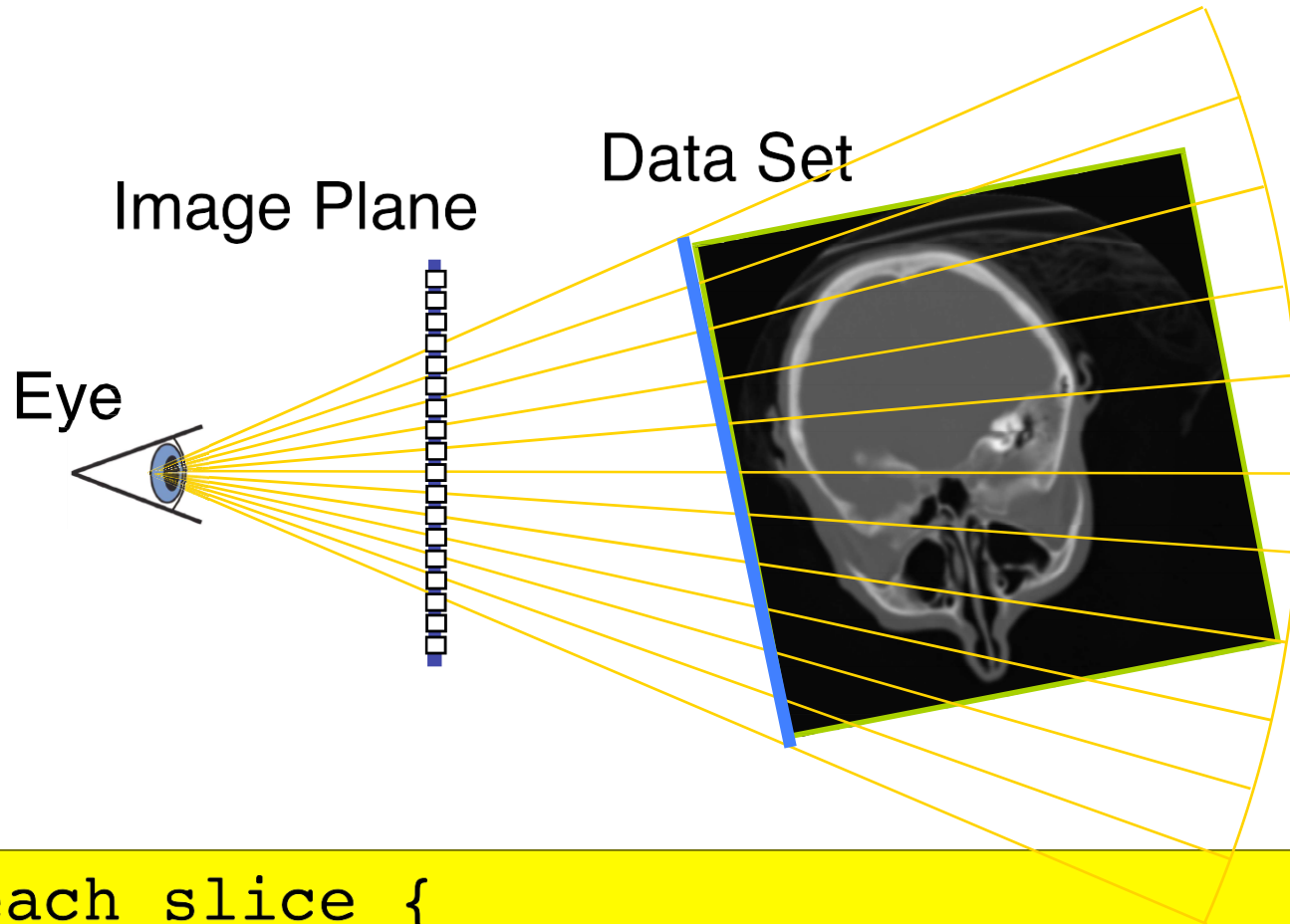


```
For each slice {  
    calculate contribution to the image  
}
```


Approche objet

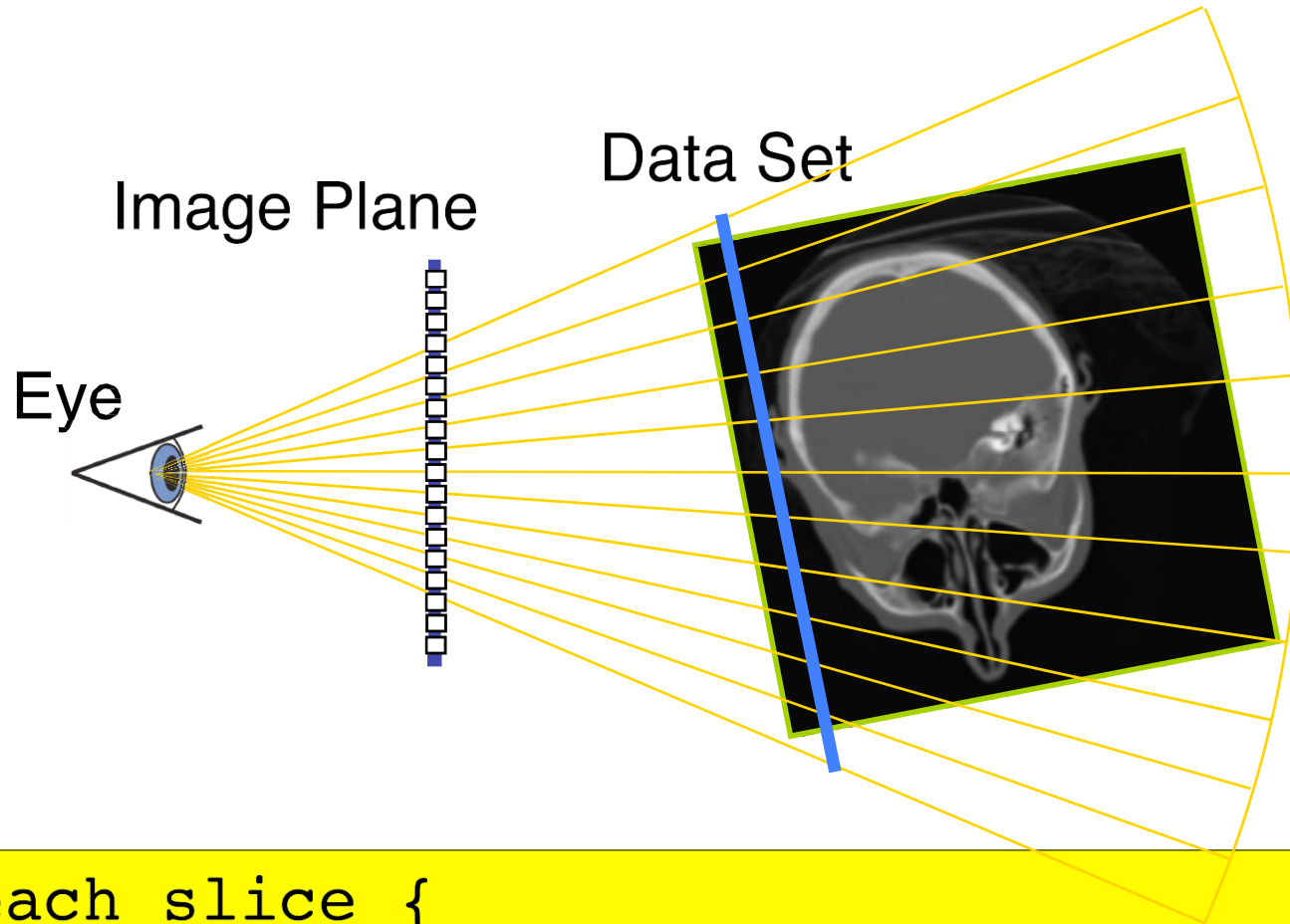


Approche objet



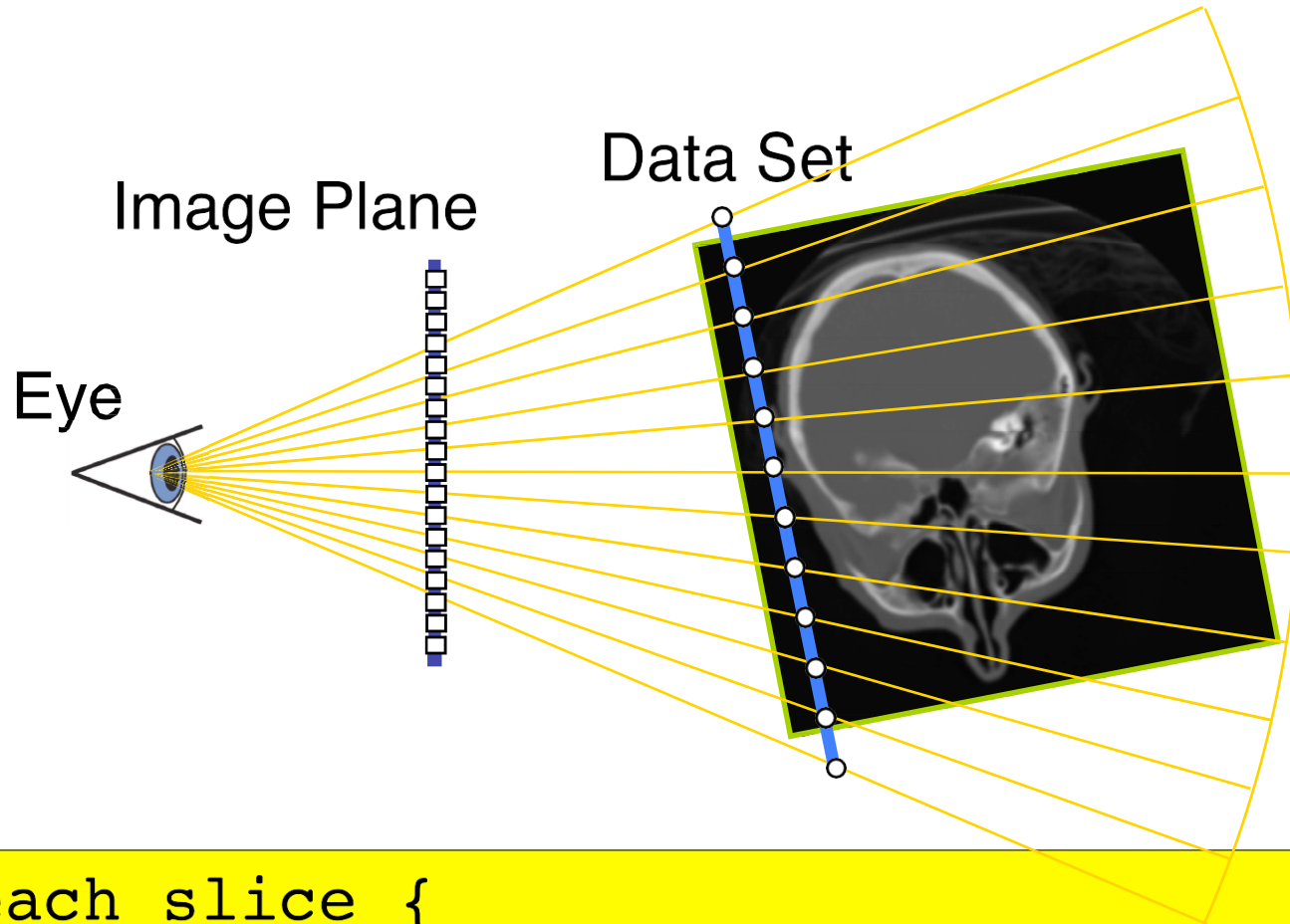
```
For each slice {  
    calculate contribution to the image  
}
```

Approche objet

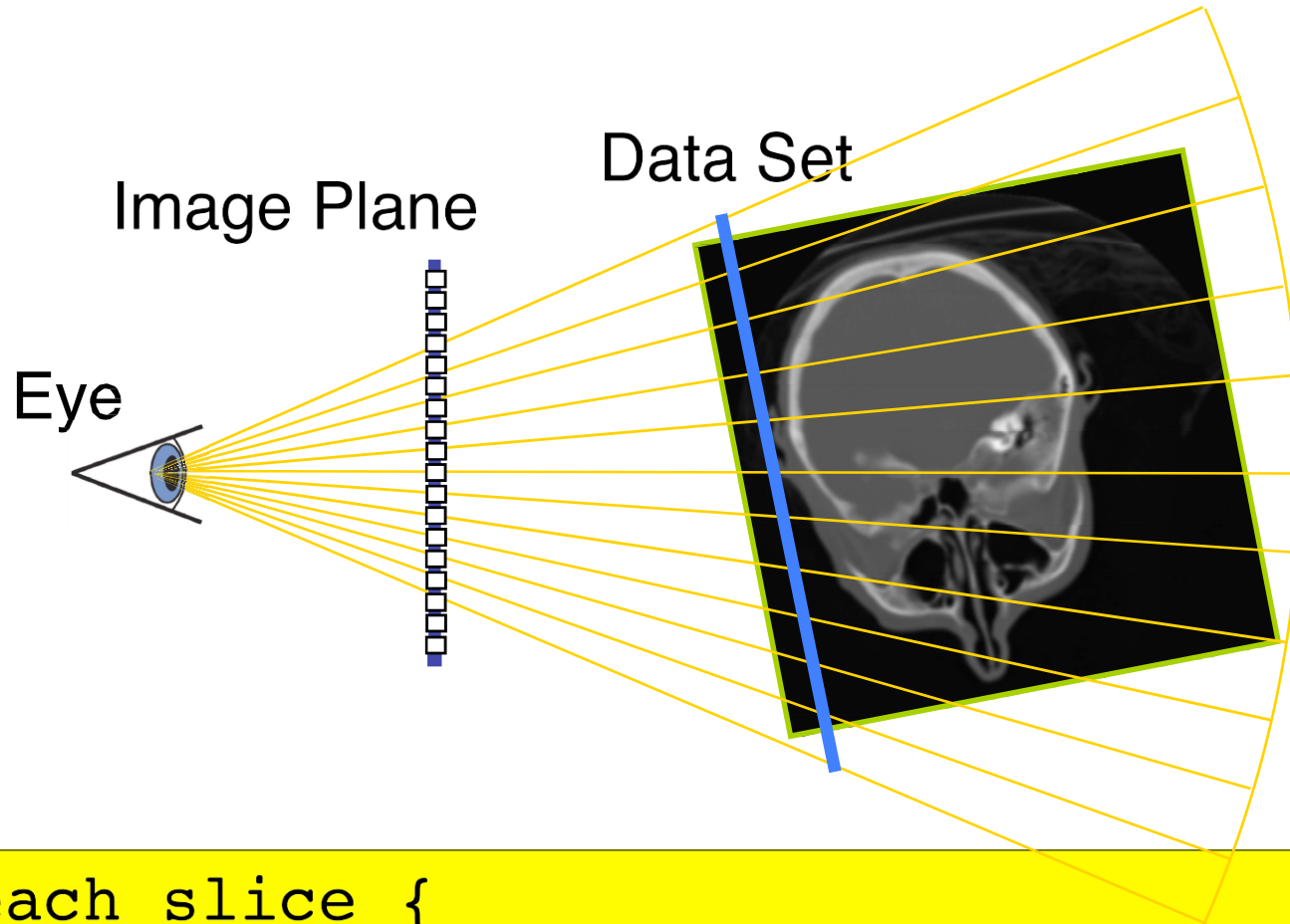


```
For each slice {  
    calculate contribution to the image  
}
```

Approche objet

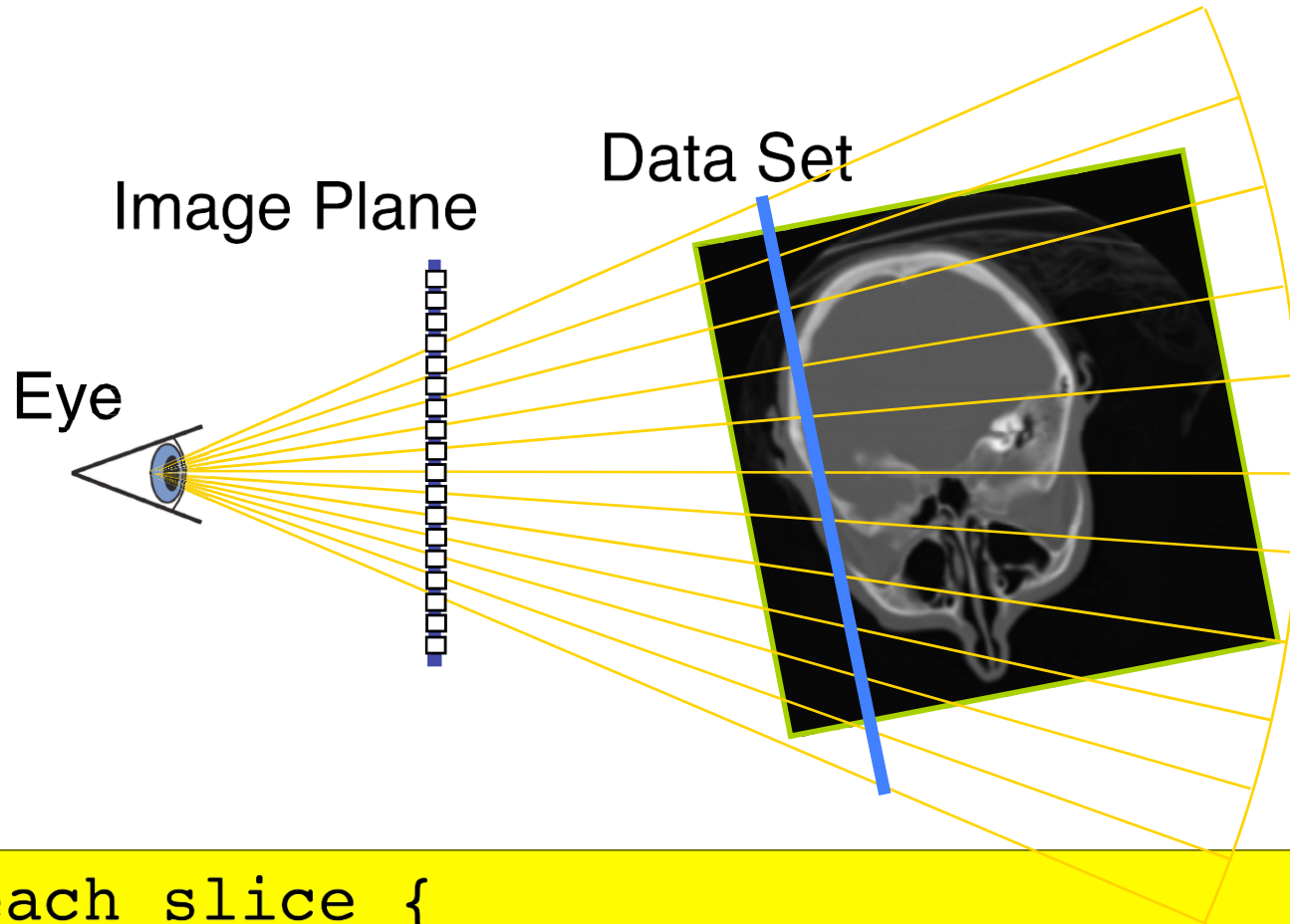


Approche objet



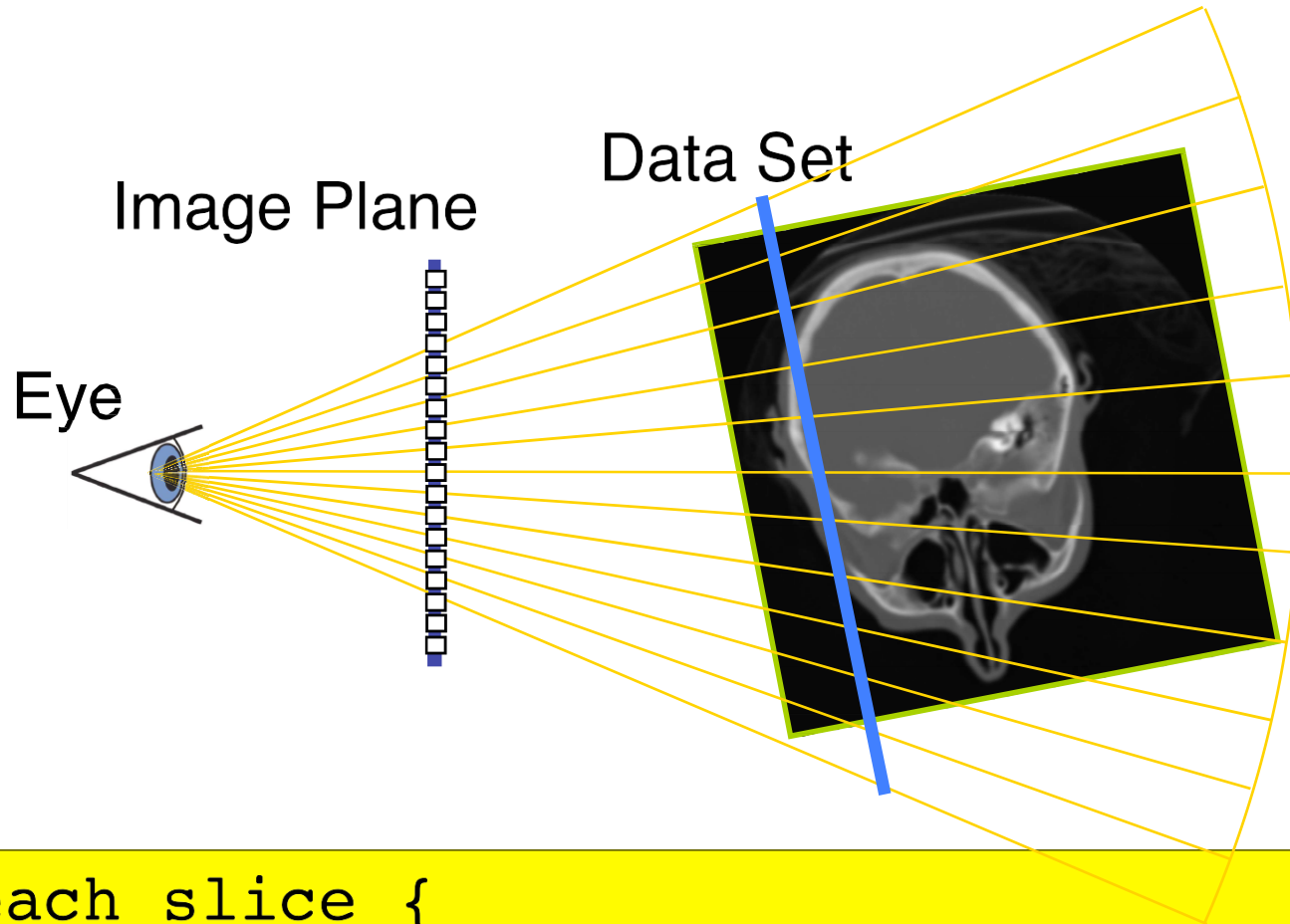
```
For each slice {  
    calculate contribution to the image  
}
```

Approche objet



```
For each slice {  
    calculate contribution to the image  
}
```

Approche objet



```
For each slice {  
    calculate contribution to the image  
}
```

Ordre des objets : splatting

Idée : « jeter » les voxels sur l'image

- Back-to-front ou front-to-back

Méthode originale de faible qualité mais améliorée depuis

- Crawfis'93: textured splats
- Swan'96, Mueller'97: anti-aliasing
- Mueller'98: image-aligned sheet-based splatting
- Mueller'99: post-classified splatting
- Huang'00: new splat primitive: FastSplats



Computer Graphics, Volume 24, Number 4, August 1990

Footprint Evaluation for Volume Rendering

Lee Westover

Numerical Design Limited

The University of North Carolina at Chapel Hill

ABSTRACT

This paper presents a forward mapping rendering algorithm to display regular volumetric grids that may not have the same spacings in the three grid directions. It takes advantage of the fact that convolution can be thought of as distributing energy from input samples into space. The renderer calculates an image plane footprint for each data sample and uses the footprint to spread the sample's energy onto the image plane. A result of the technique is that the forward mapping algorithm can support perspective without excessive cost, and support adaptive resampling of the three-dimensional data set during image generation.

KEYWORDS: 3D Image, Volume Rendering, Reconstruction, Algorithms.

INTRODUCTION

Volume rendering is the direct display of data sampled in three dimensions. There are two principle approaches to volume rendering: backward mapping algorithms that map the image plane onto the data by shooting rays from pixels into the data space, and forward mapping algorithms that map the data onto the image plane.

This distinction principally manifests itself in how and when reconstruction of the three-dimensional signal is done. Convolution can be thought of as either generating an output sample from many input samples or as spreading one input sample to many output samples. Backward mapping algorithms typically reconstruct the signal at a point in space by looking at that point's nearest data samples and performing some type of interpolation. Forward mapping algorithms differ in that they incrementally reconstruct the original signal by spreading each data sample's energy into space.

Forward mapping algorithms are important because they are easily made parallel. Since each data sample only needs to know about a small surrounding neighborhood of other samples, shading and transforming can be done in parallel for sub-sections of the data. With today's parallel machines having limited local memory, this data distribution gets around the backward mapping problem of having the entire data set at each node.

The reconstruction step is the most complicated part of the algorithm. The renderer must determine the screen space contribution of each sample point to the final image. A brute force method would perform a one-dimensional integration of the reconstruction kernel for every pixel for every input sample. If the renderer can calculate the screen space extent of the kernel, the number of integrations reduces to the number of samples times the number of pixels that fall within the extent. However, this is still an enormous number of integrations.

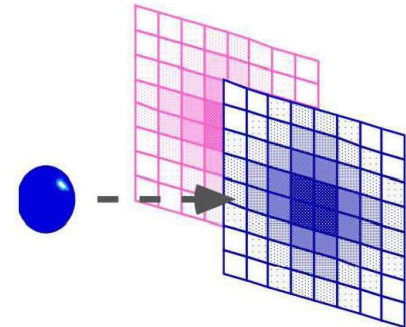
In an orthographic view, the footprint of the projected reconstruction kernel for any sample is a constant except

* Author's current address:

Sun Microsystems Inc
PO Box 13447
Research Triangle Park NC
27709

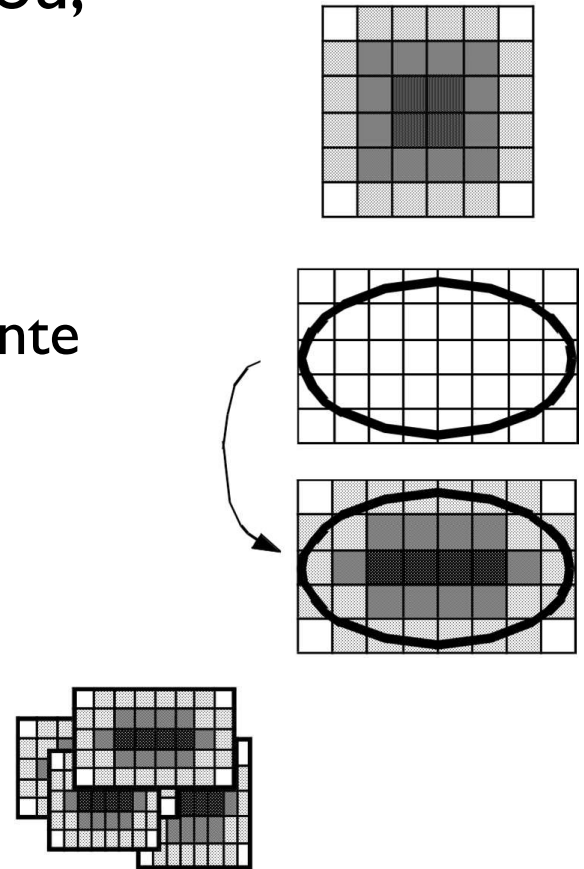
Splatting

- Au lieu de demander quel échantillon contribue à une valeur de pixel, à quel pixel une valeur contribue ?
 - Ray casting : valeur du pixel calculée avec plusieurs échantillons
 - Splatting : plusieurs valeurs de pixel (partiellement) calculées avec un seul échantillon
- Overview :
 - Bonne qualité
 - Relativement couteux et lent
- Idée : faire contribuer tous les pixels à l'image
 - Projection d'un voxel : splat
 - Composition dans l'espace image



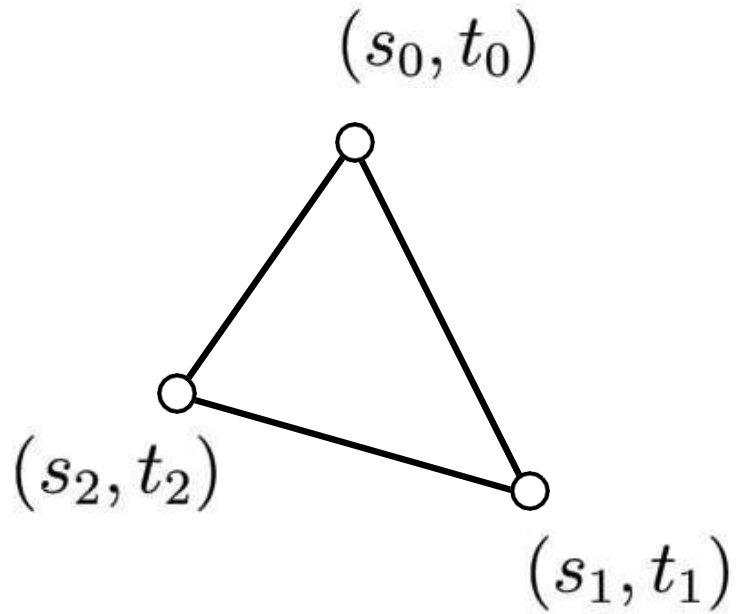
Splatting – footprint

- Une forte empreinte augmente le flou, utilisé pour un ratio élevé de pixel-sur-voxel
- Empreinte géométrique
 - Projection orthographique : indépendante du point de vue
 - Projection perspective : elliptique
- Pré-intégration de l'empreinte
- Perspective : précalcule des orientations de l'ellipse

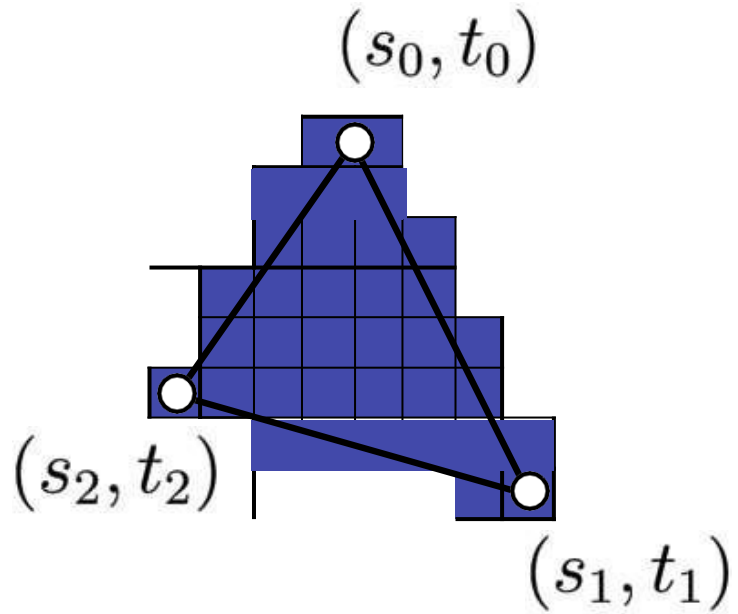


Mapping de textures

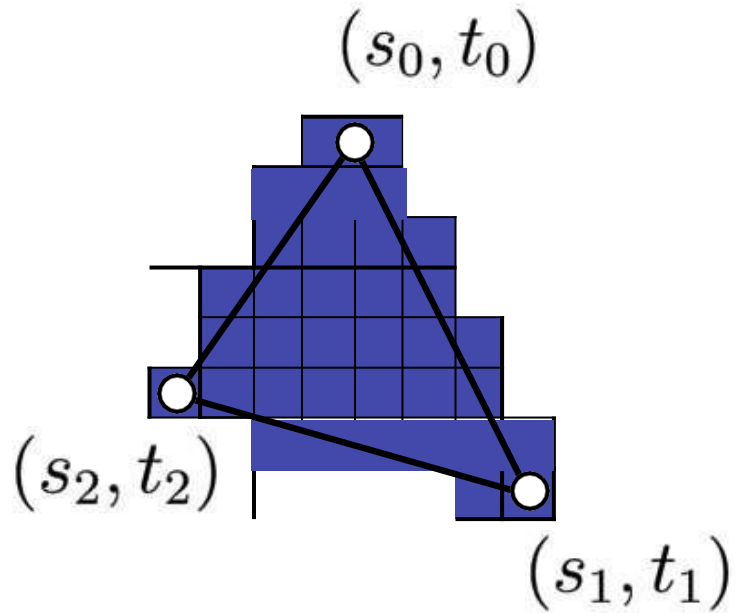
Principe



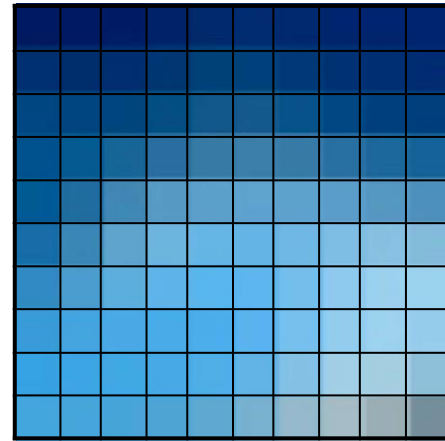
Principe



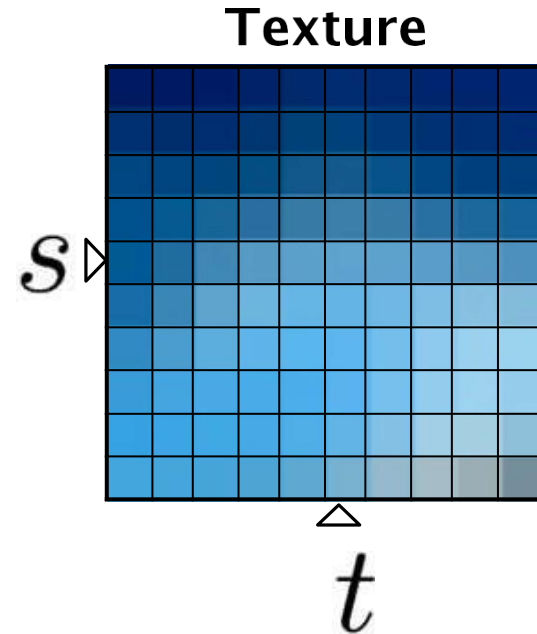
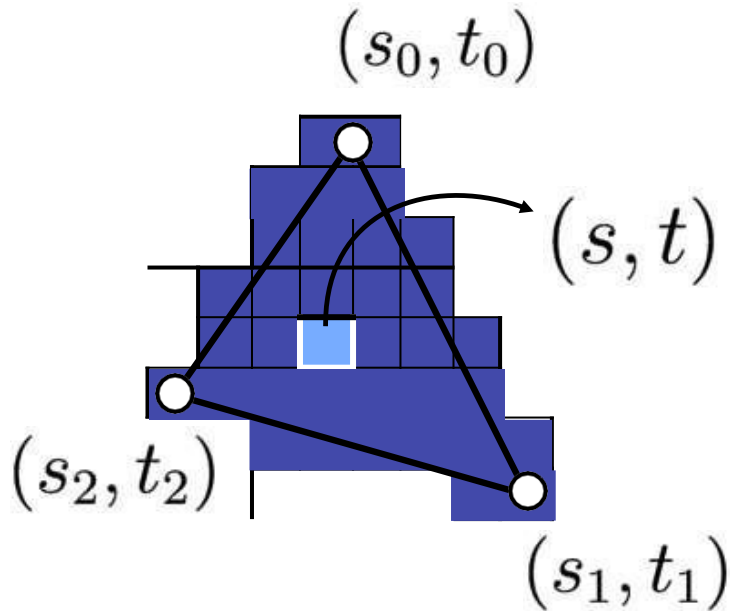
Principe



Texture

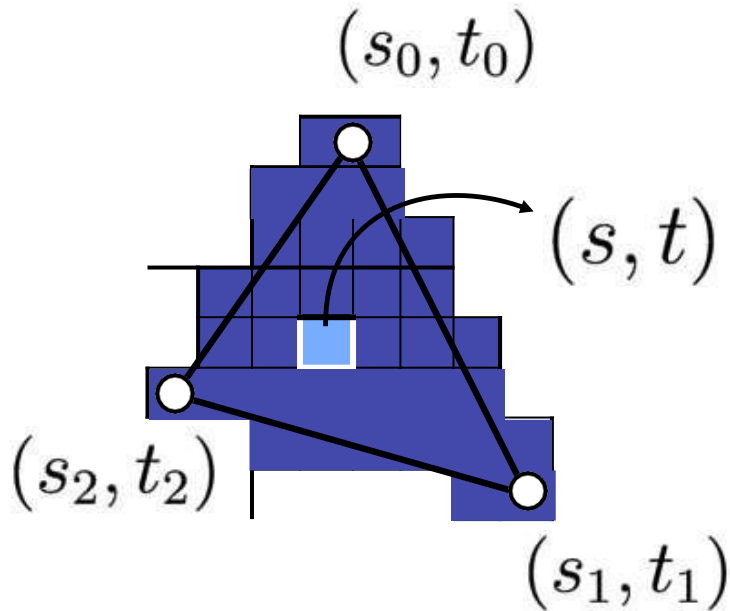


Principe

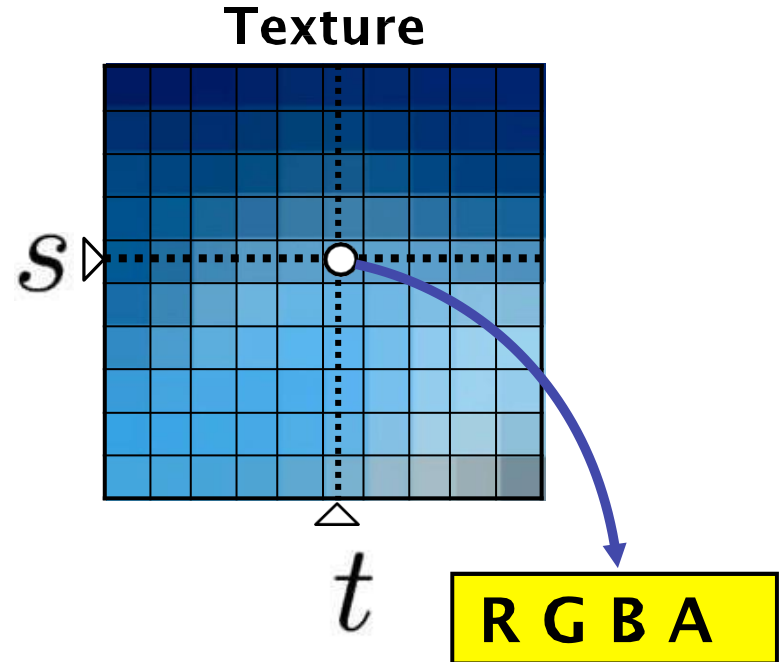


Pour chaque fragment :
interpoler les coordonnées
de texture (barycentrique)

How does a texture work?



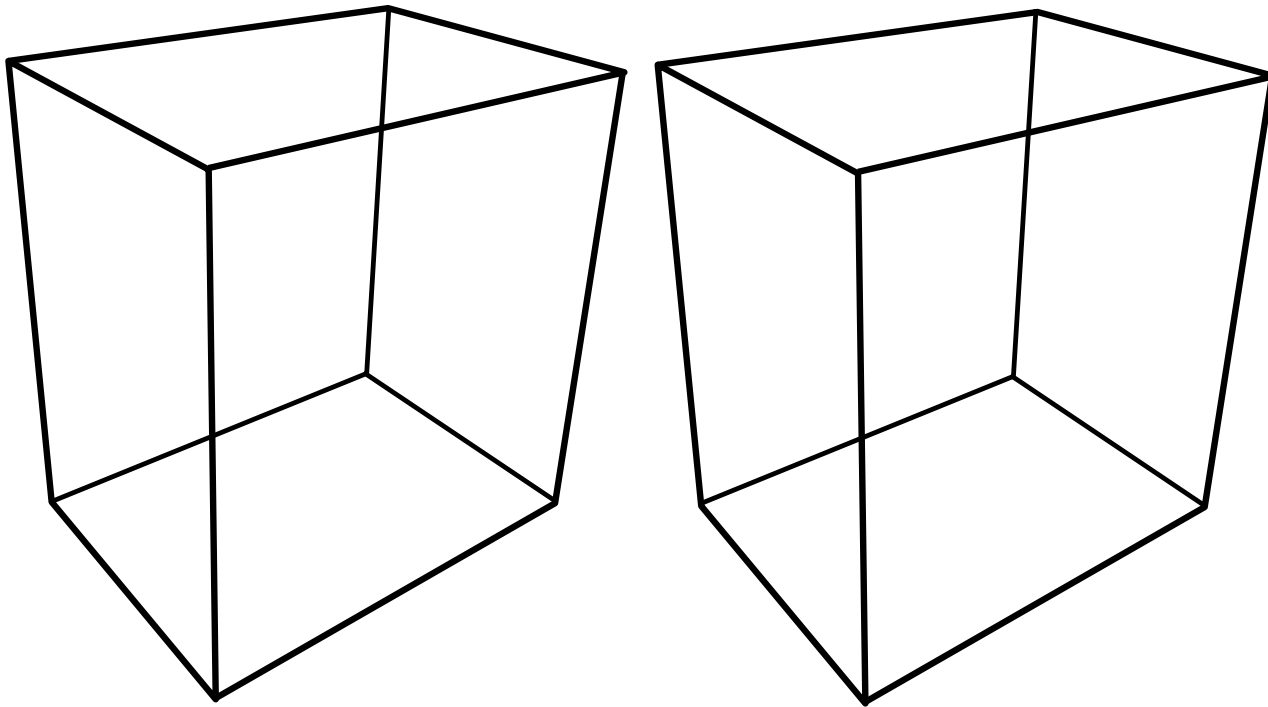
Pour chaque fragment :
interpoler les coordonnées
de texture (barycentrique)



Texture-Lookup :
Interpoler la couleur de
texture (bilinéaire)

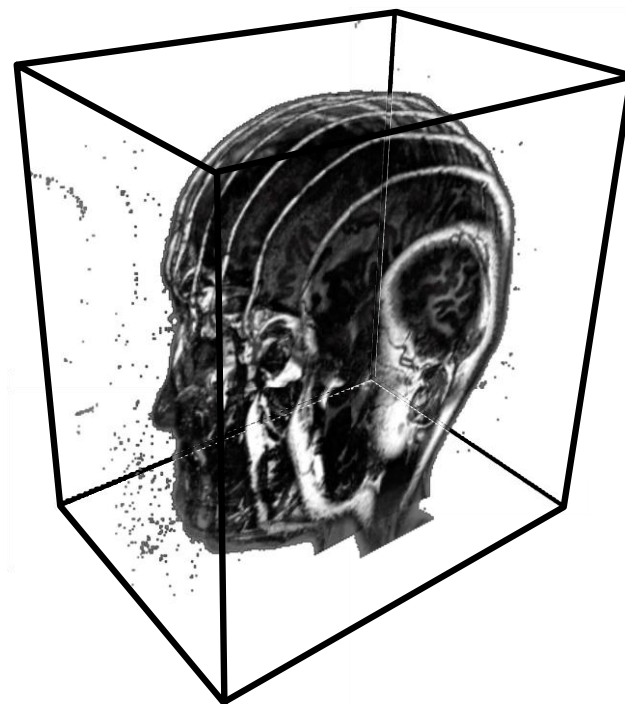
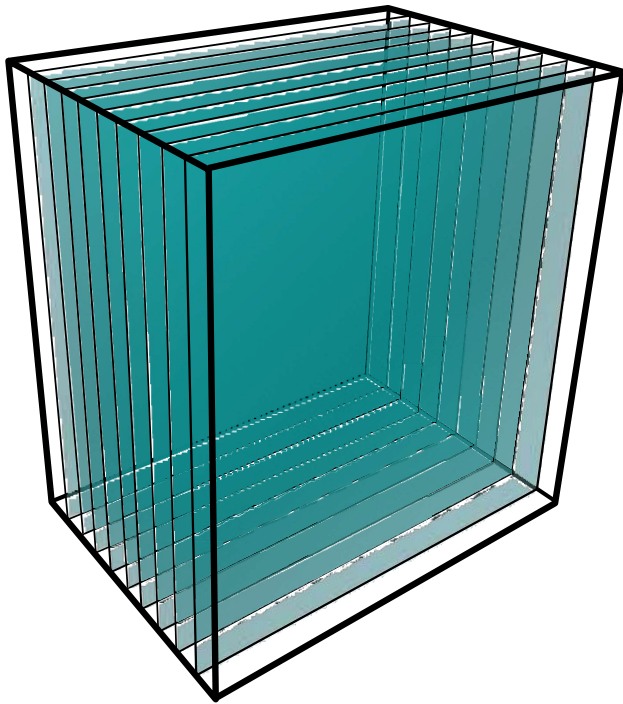
Rendu volumique avec des texture 2D

- Plans parallèles au « plan de base » (front face du volume la plus orthogonale au point de vue)
- Dessiner un rectangle texturé en utilisant une interpolation bilinéaire
- Rendre en back-to-front en utilisant de la composition



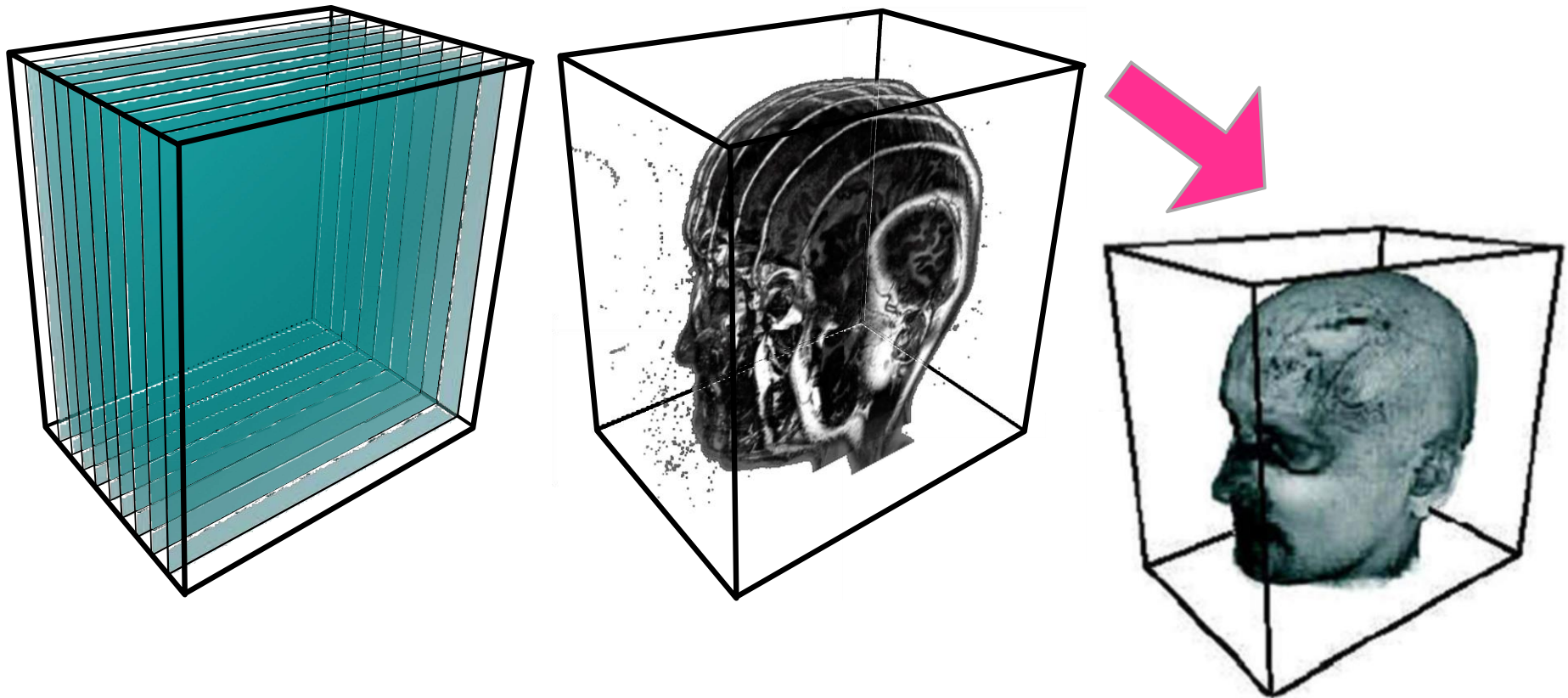
Rendu volumique avec des texture 2D

- Plans parallèles au « plan de base » (front face du volume la plus orthogonale au point de vue)
- Dessiner un rectangle texturé en utilisant une interpolation bilinéaire
- Rendre en back-to-front en utilisant de la composition



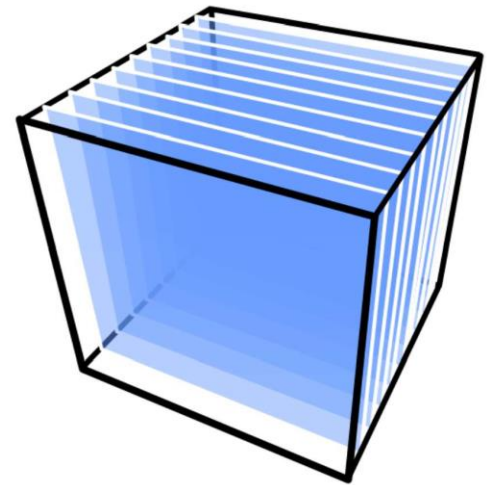
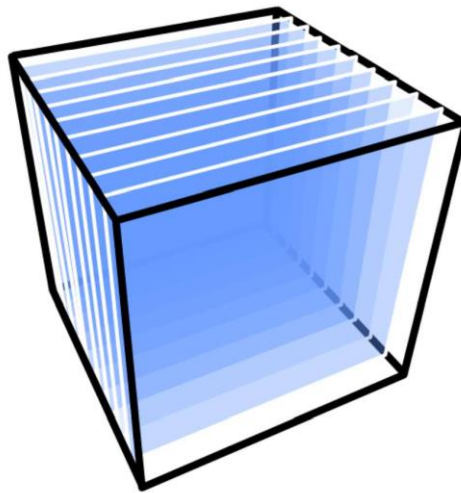
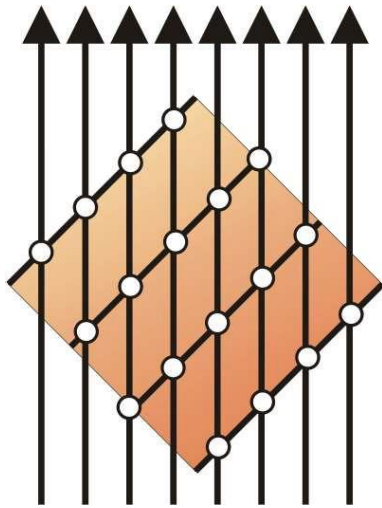
Rendu volumique avec des texture 2D

- Plans parallèles au « plan de base » (front face du volume la plus orthogonale au point de vue)
- Dessiner un rectangle texturé en utilisant une interpolation bilinéaire
- Rendre en back-to-front en utilisant de la composition



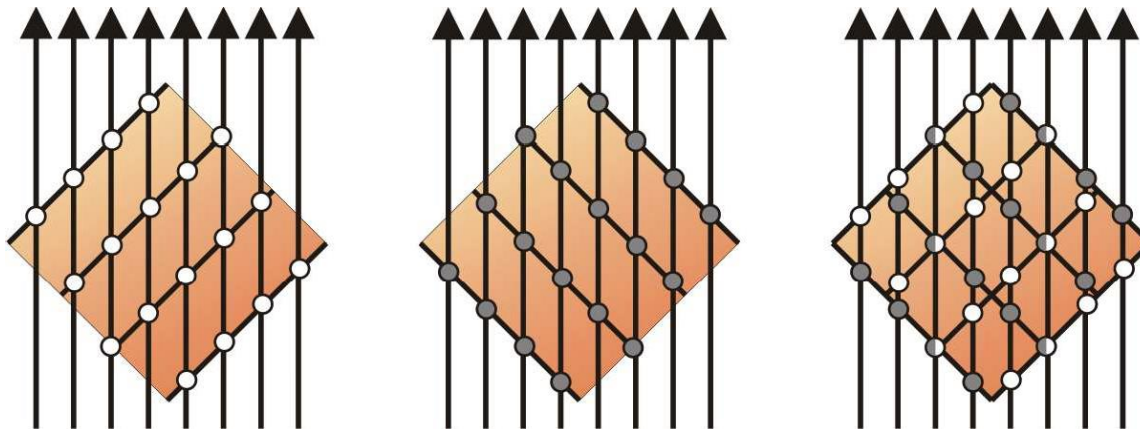
Rendu volumique avec des texture 2D

- Slices 2D texturées
 - Alignées sur l'objet
 - 3 stacks de textures 2D
 - Interpolation bilinéaire



Rendu volumique avec des texture 2D

- Stack de textures 2D
 - Artefacts quand la stack est vue à 45°
 - Positions des échantillons peut changer abruptement

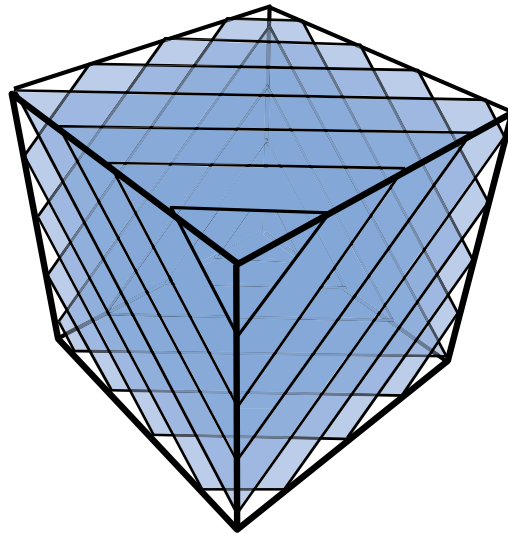


Texture 3D

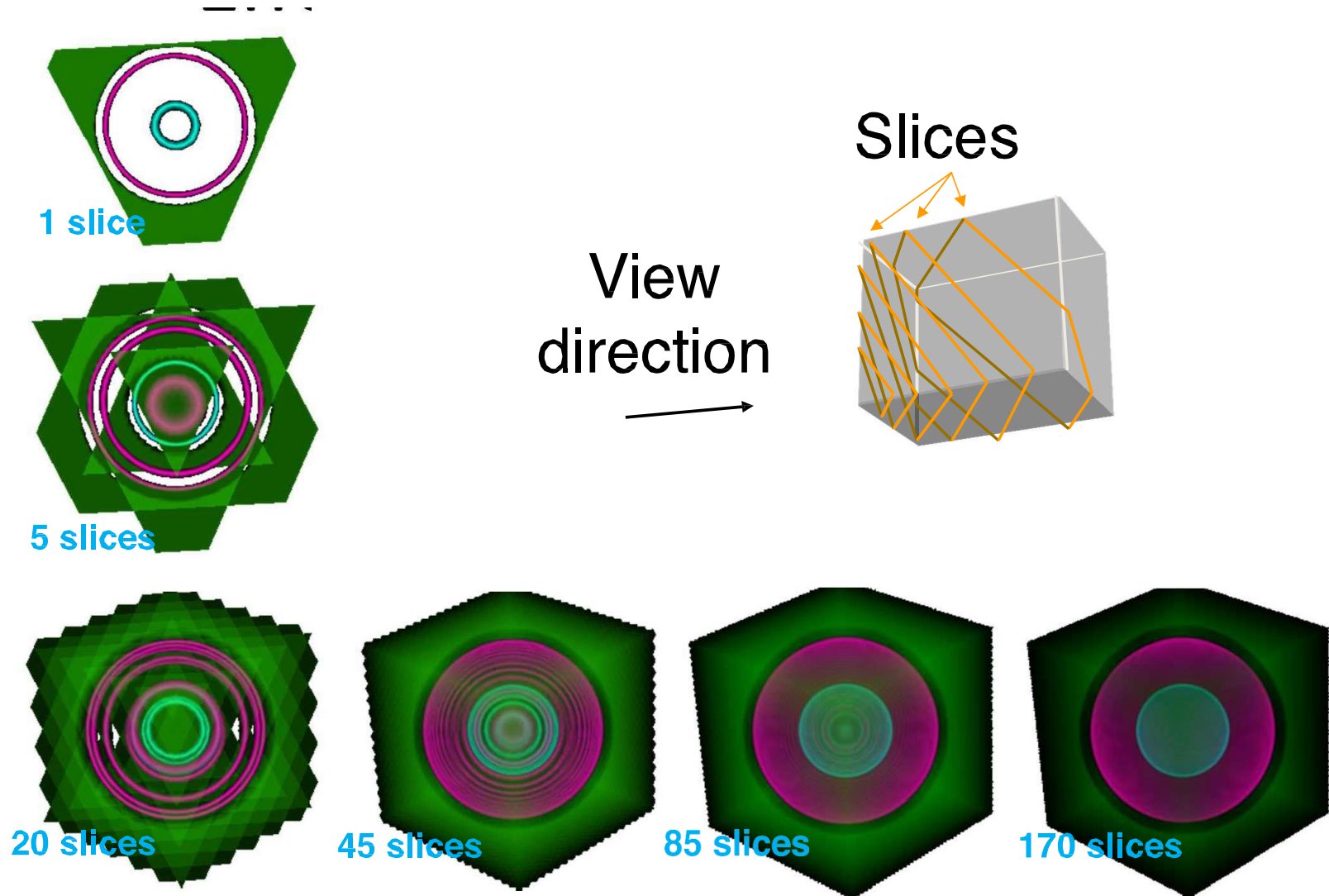
- Les données voxéliques devient une texture 3D
 - Interpolation trilineaire hardware
 - Utilisation d'un nombre arbitraire de plans parallèles au plan image
 - Composition back-to-front comme pour les textures 2D
 - Peut être limité par la taille mémoire

Texture 3D

- Les données voxéliques devient une texture 3D
 - Interpolation trilinéaire hardware
 - Utilisation d'un nombre arbitraire de plans parallèles au plan image
 - Composition back-to-front comme pour les textures 2D
 - Peut être limité par la taille mémoire

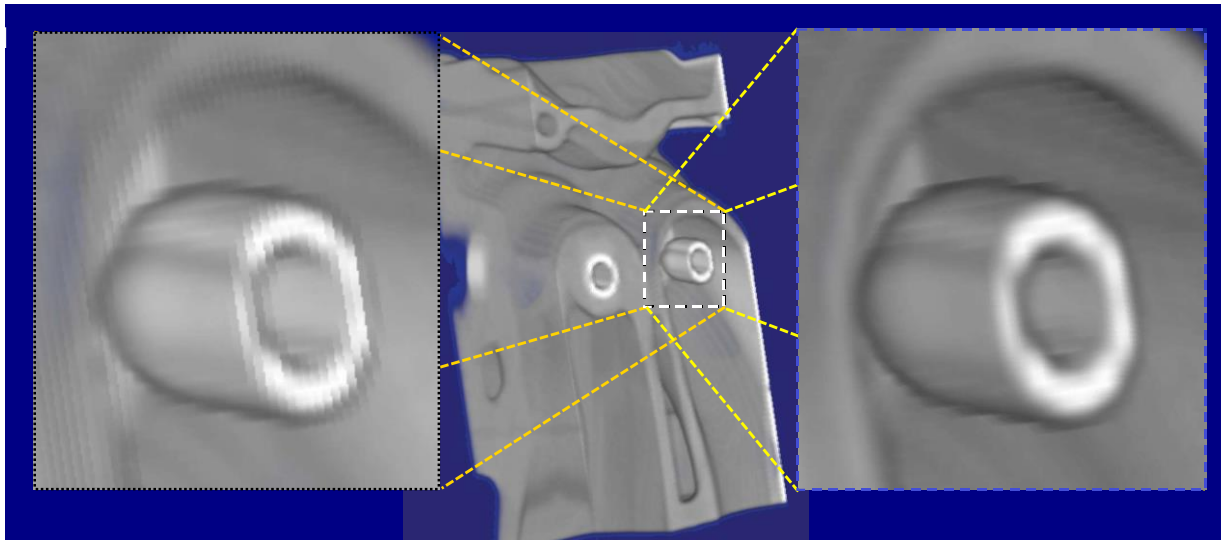


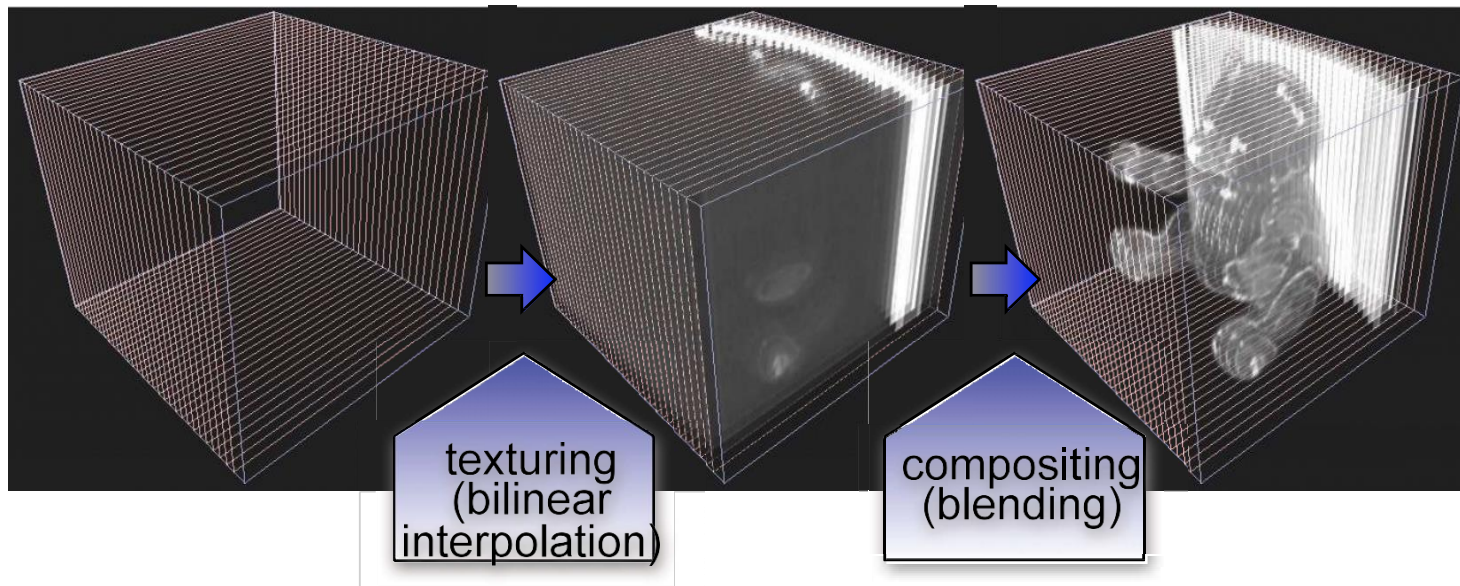
Echantillonnage



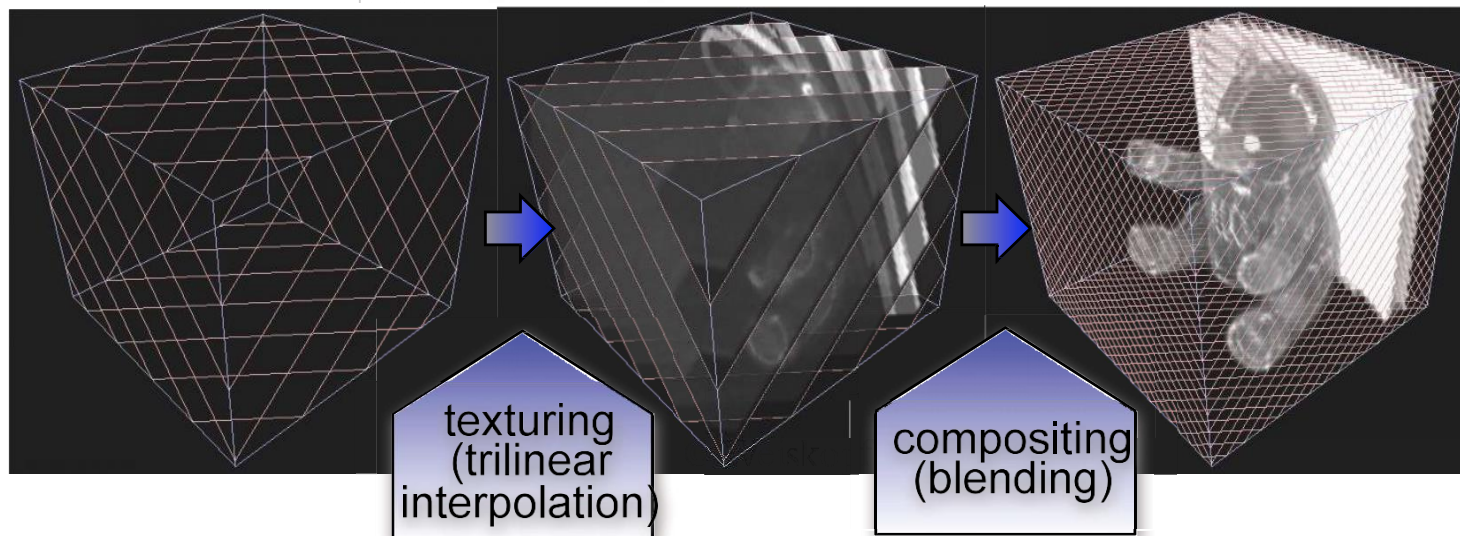
Rendu volumique à base de texture

- Texture 3D :
 - Pas d'artefacts dû à l'angle de vue
 - Augmentation de l'échantillonnage → plus de slices
 - Facile avec des textures 3D





Textures 2D
alignées sur
les axes

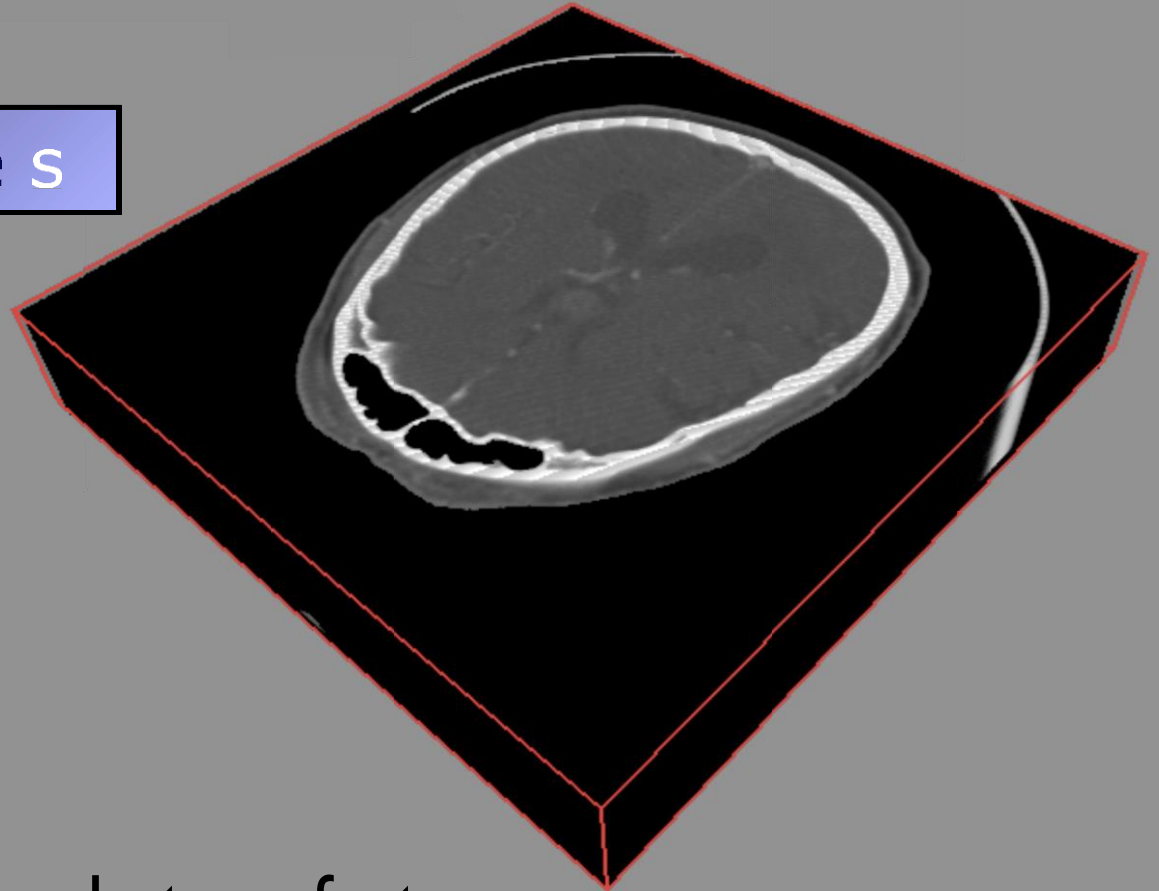


Textures 3D
alignées sur la
vue

Classification

- Comment j'obtiens la valeur d'émission $q(S)$ et l'absorption $A(S)$?

scalar value s

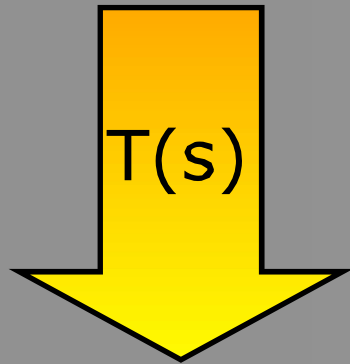


Réponse : la fonction de transfert

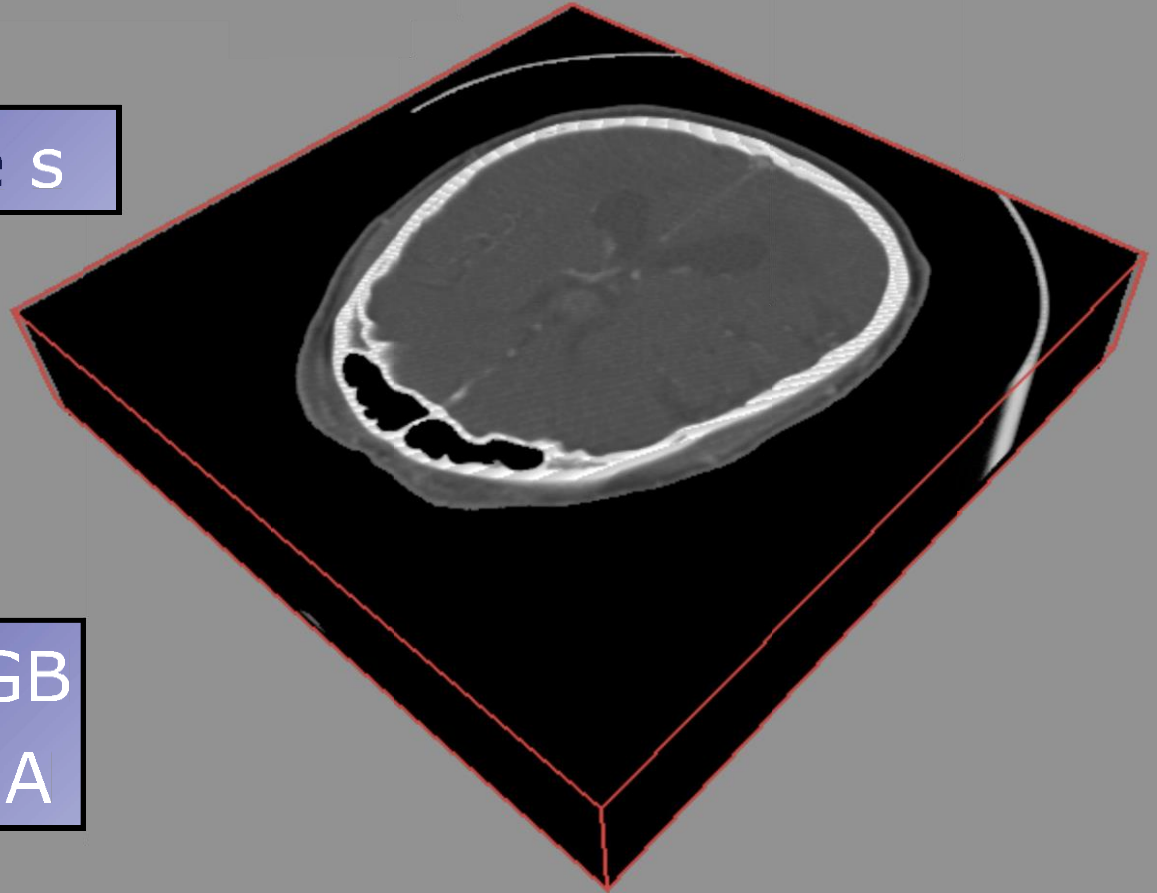
Classification

- Comment j'obtiens la valeur d'émission $q(S)$ et l'absorption $A(S)$?

scalar value s



emission RGB
absorption A



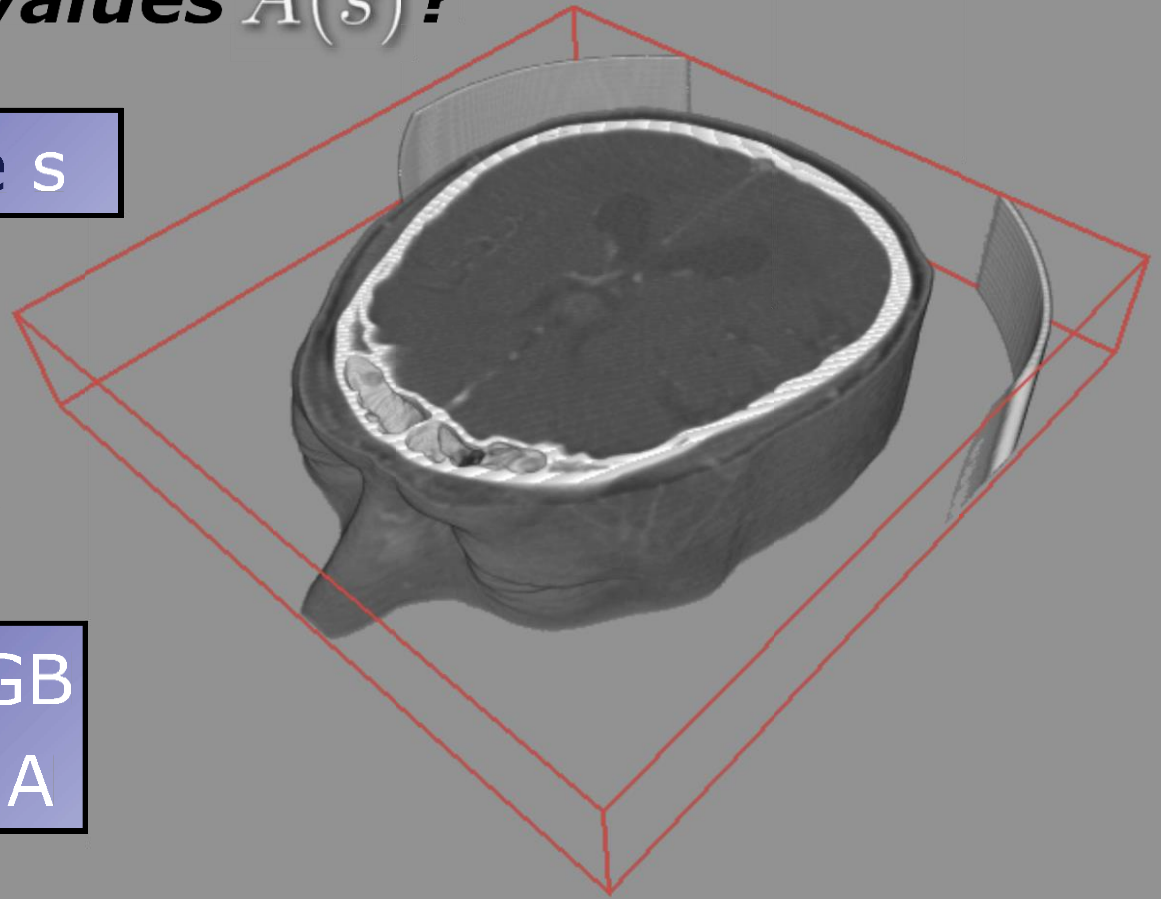
Classification

How do I obtain the emission values $q(s)$ and Absorption values $A(s)$?

scalar value s

$T(s)$

emission RGB
absorption A



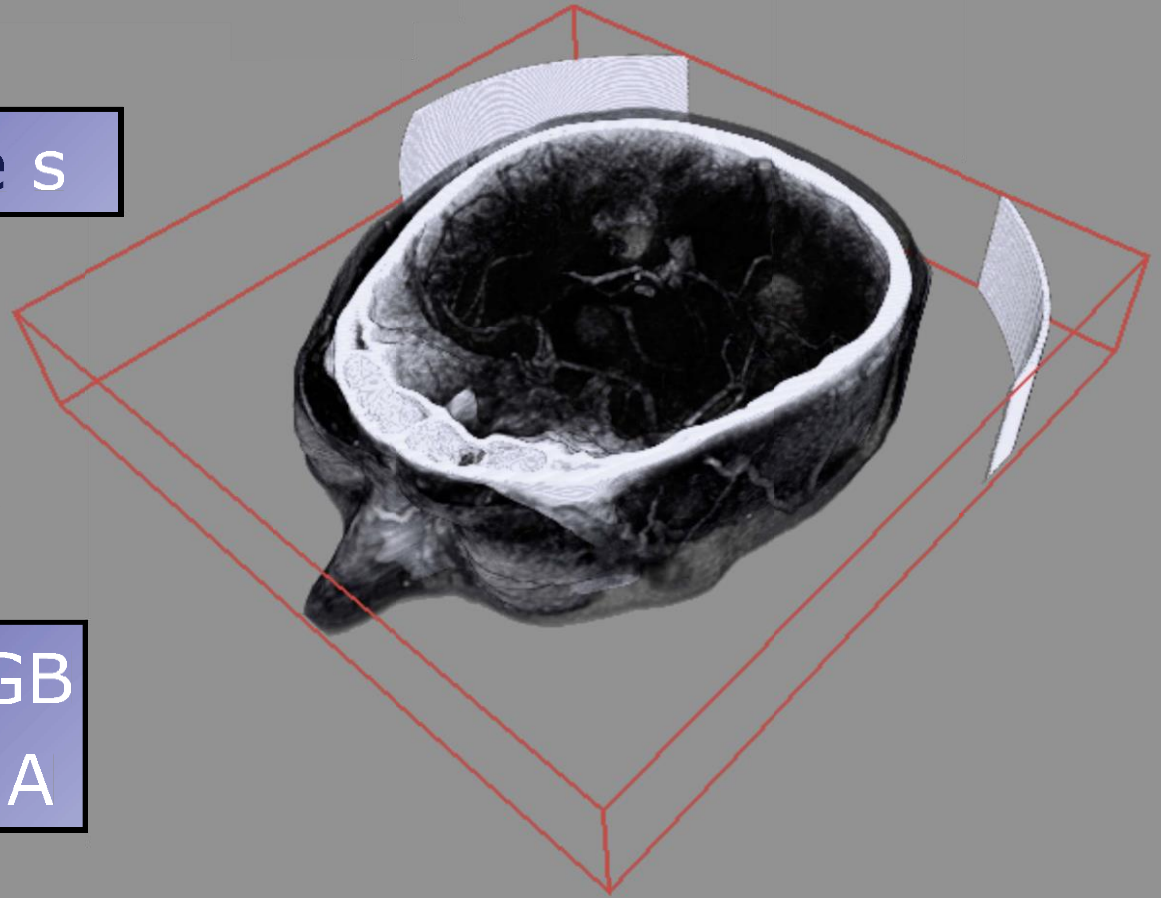
Classification

- Comment j'obtiens la valeur d'émission $q(S)$ et l'absorption $A(S)$?

scalar value s

$T(s)$

emission RGB
absorption A



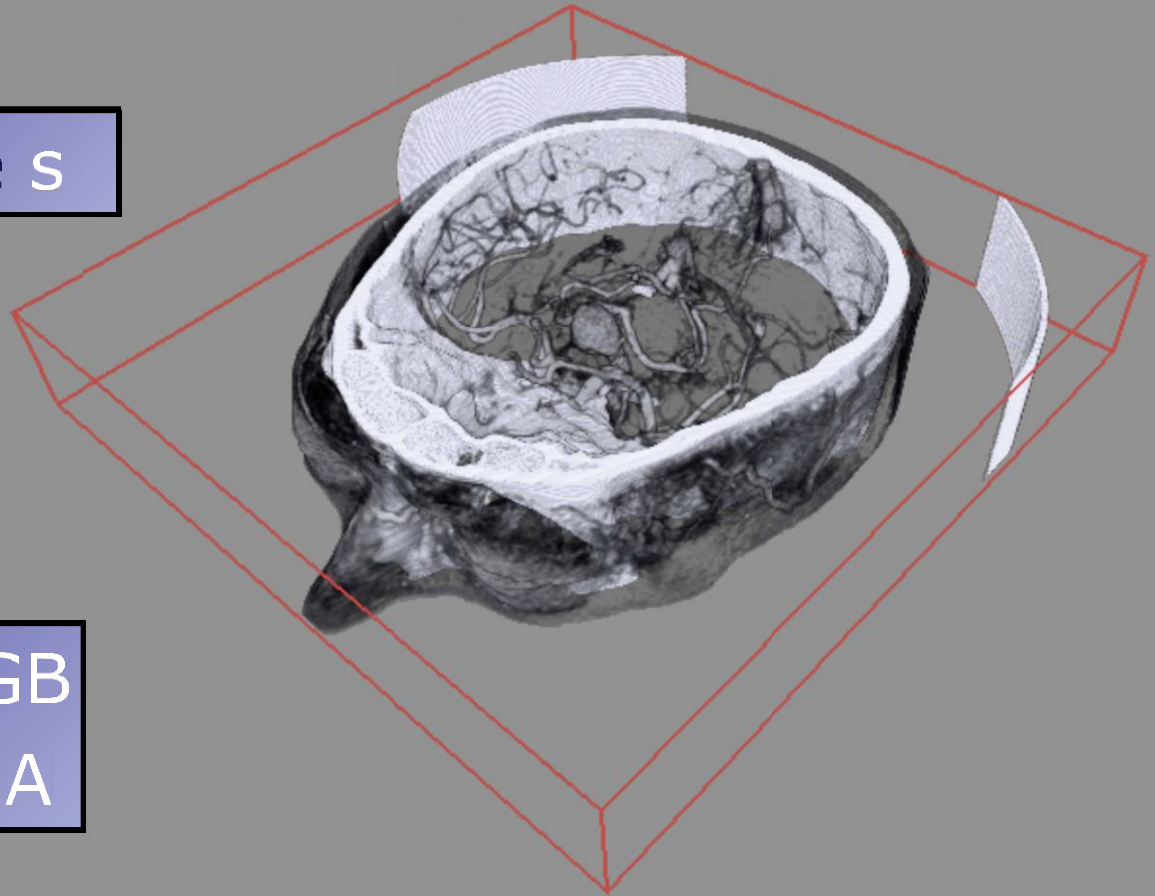
Classification

- Comment j'obtiens la valeur d'émission $q(S)$ et l'absorption $A(S)$?

scalar value s

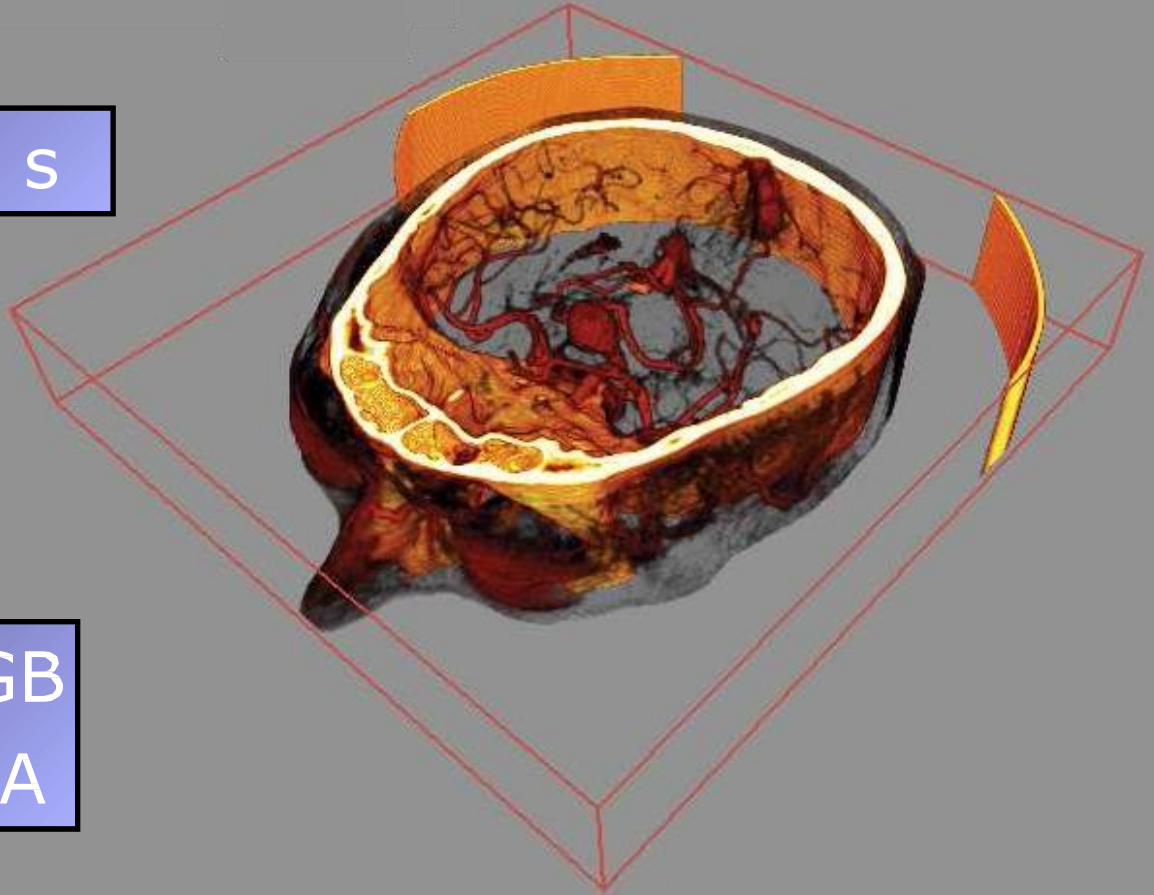
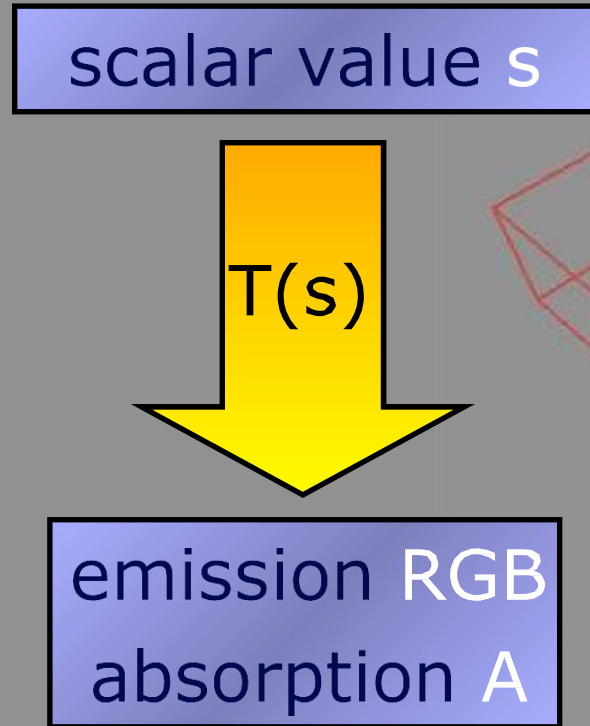
$T(s)$

emission RGB
absorption A

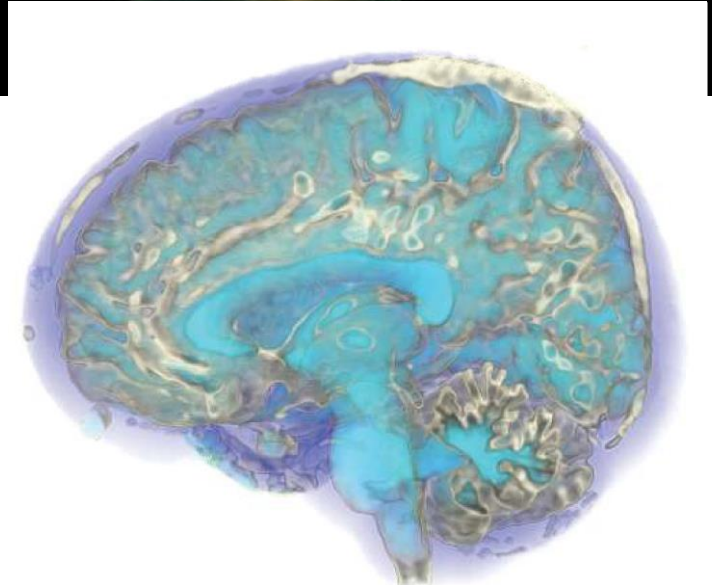
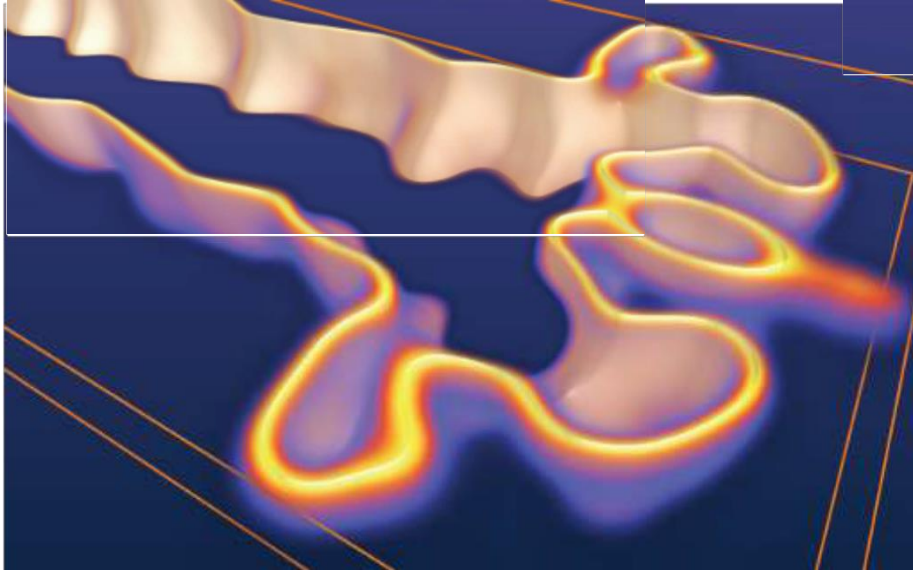
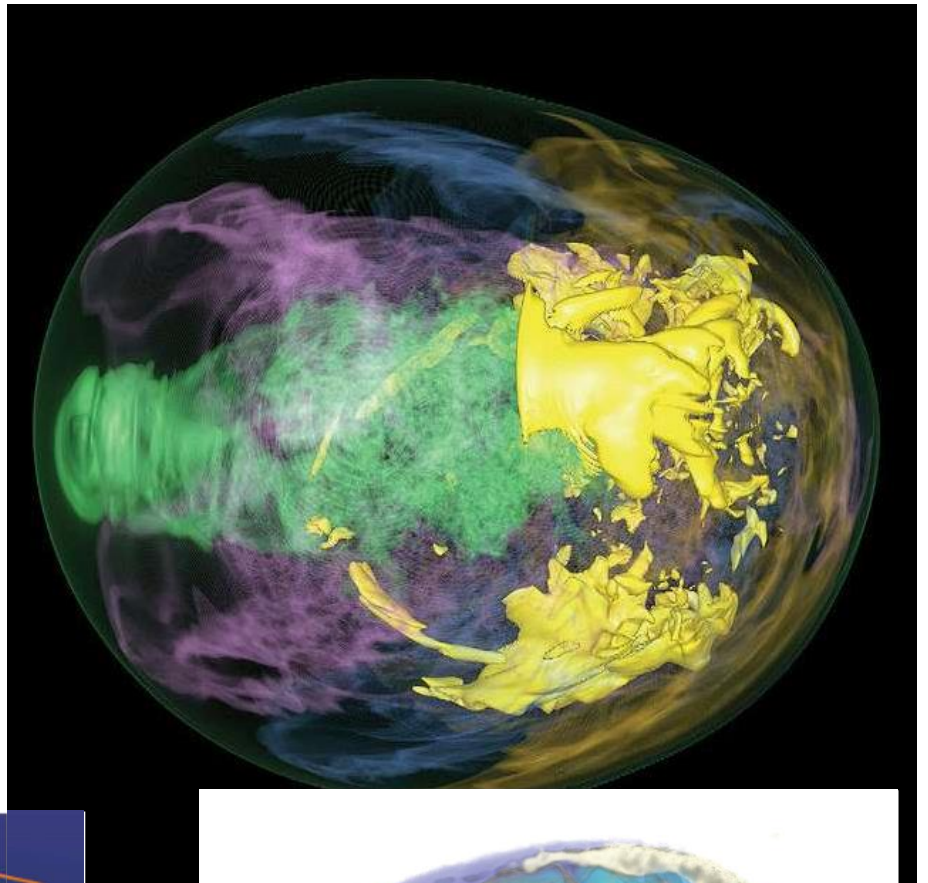
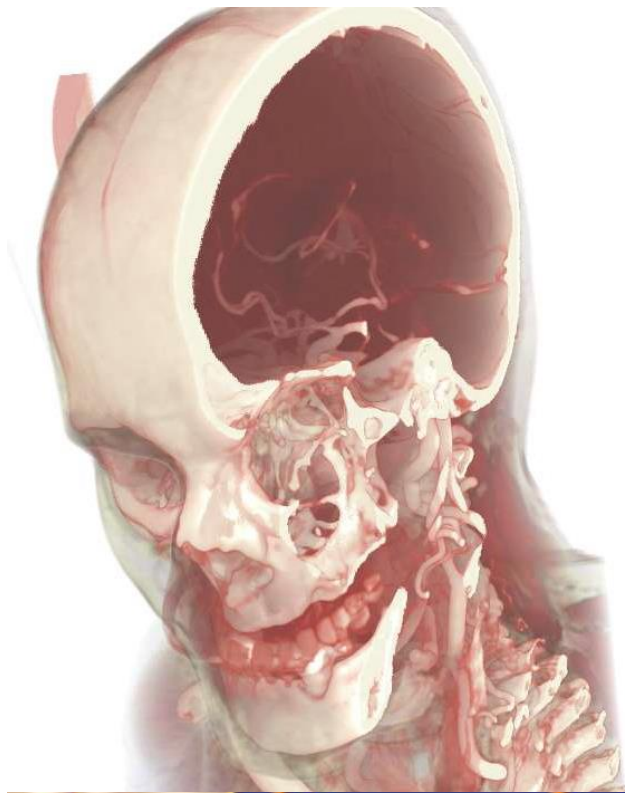


Classification

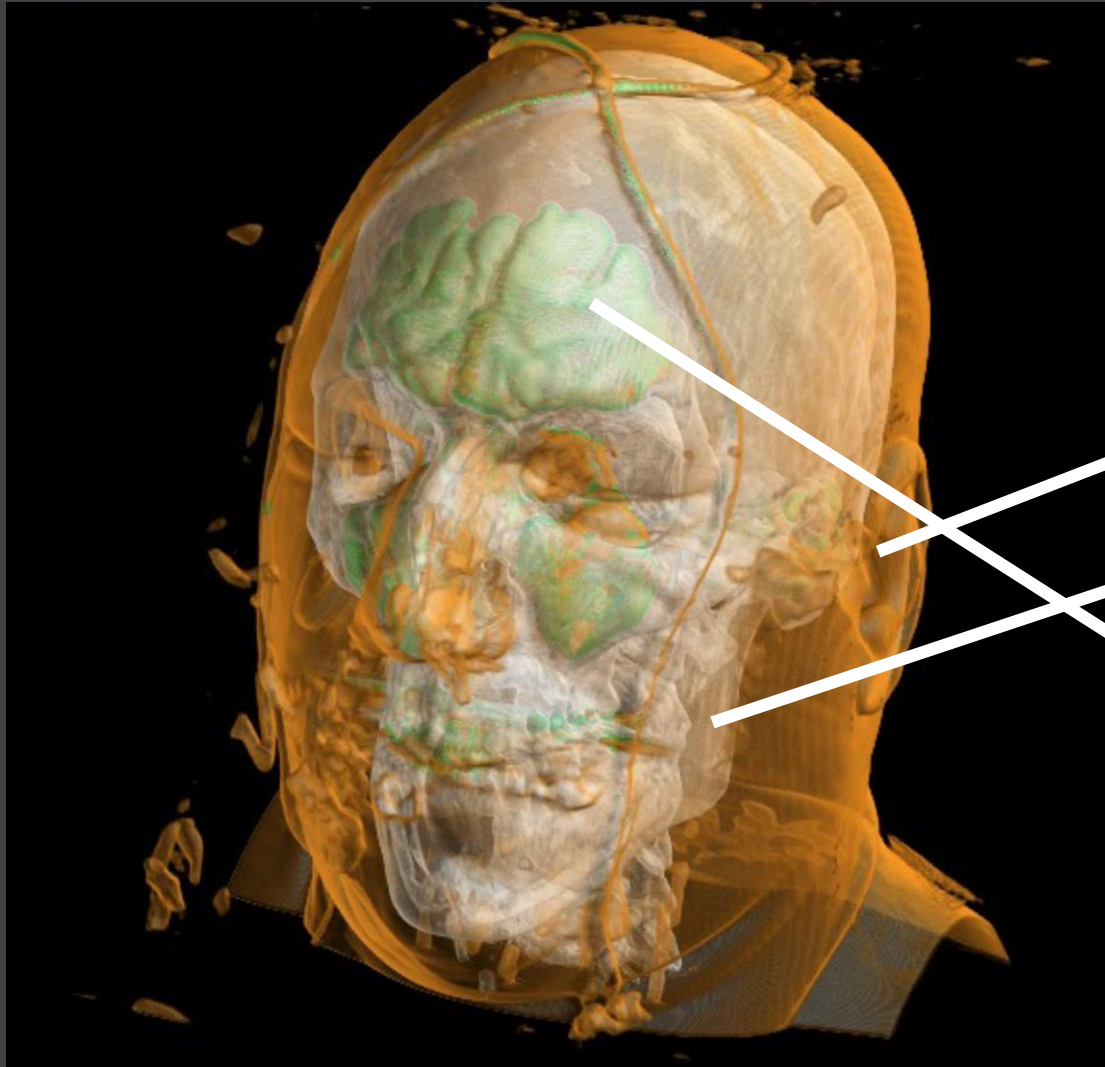
- Comment j'obtiens la valeur d'émission $q(S)$ et l'absorption $A(S)$?



Résumé



Volume Rendering: Interfaces



**Transfer function,
With shading**

Skin/Air

Bone/Soft tissue

Bone/Air



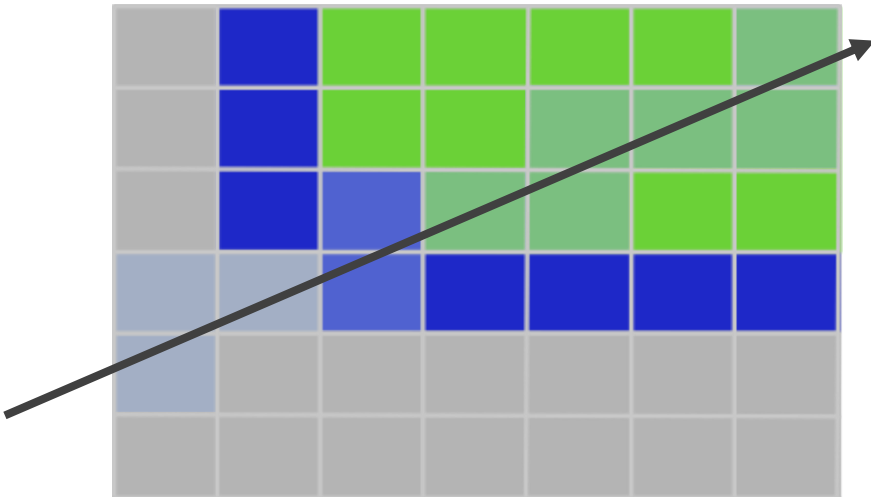
Voxel grid visualization

Volume rendering

- Ray casting
 - [Kruger 2003],[Grimm 2004]...
- GPU-based
 - [Green 2004], [Kratz 2006]...



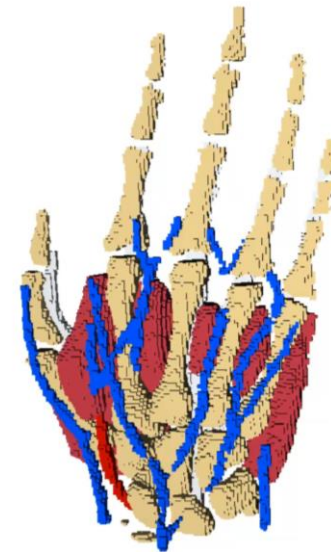
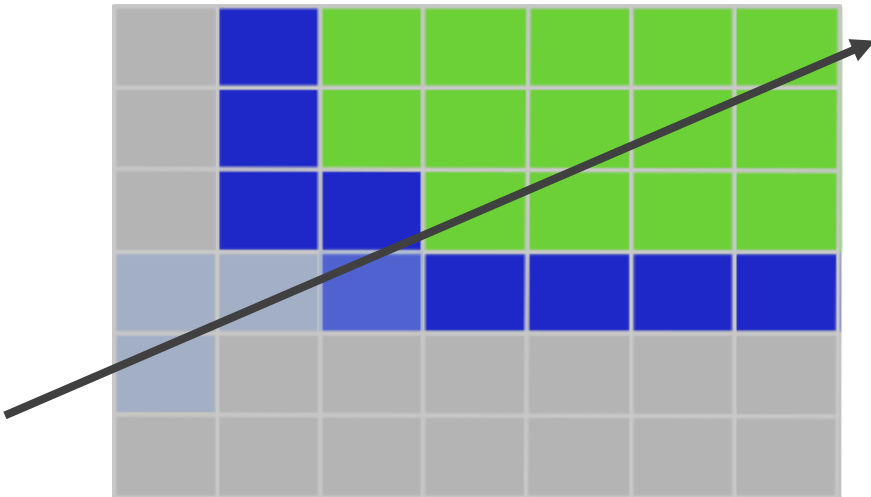
[Scharsach 2005]



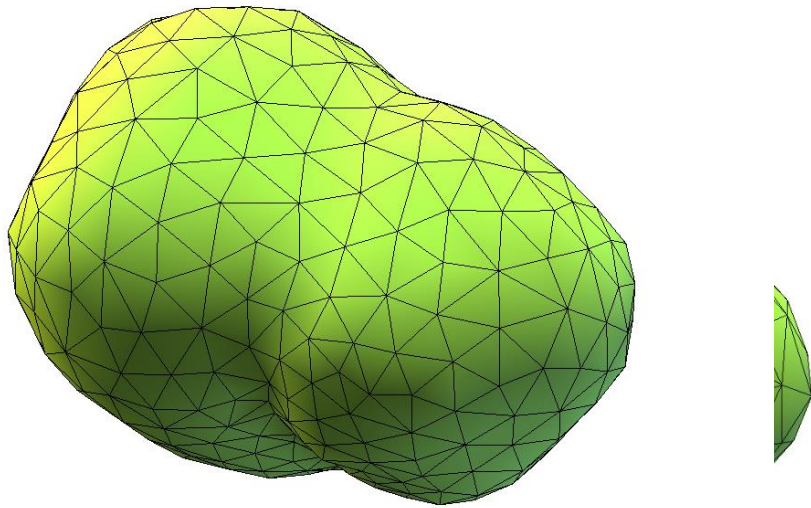
Voxel grid visualization

Voxel rendering

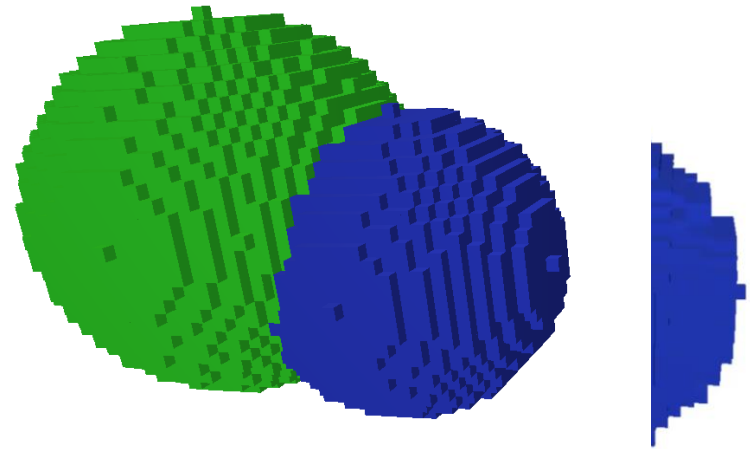
- GPU-based ray casting
- Time-varying dataset



Interactive visualization

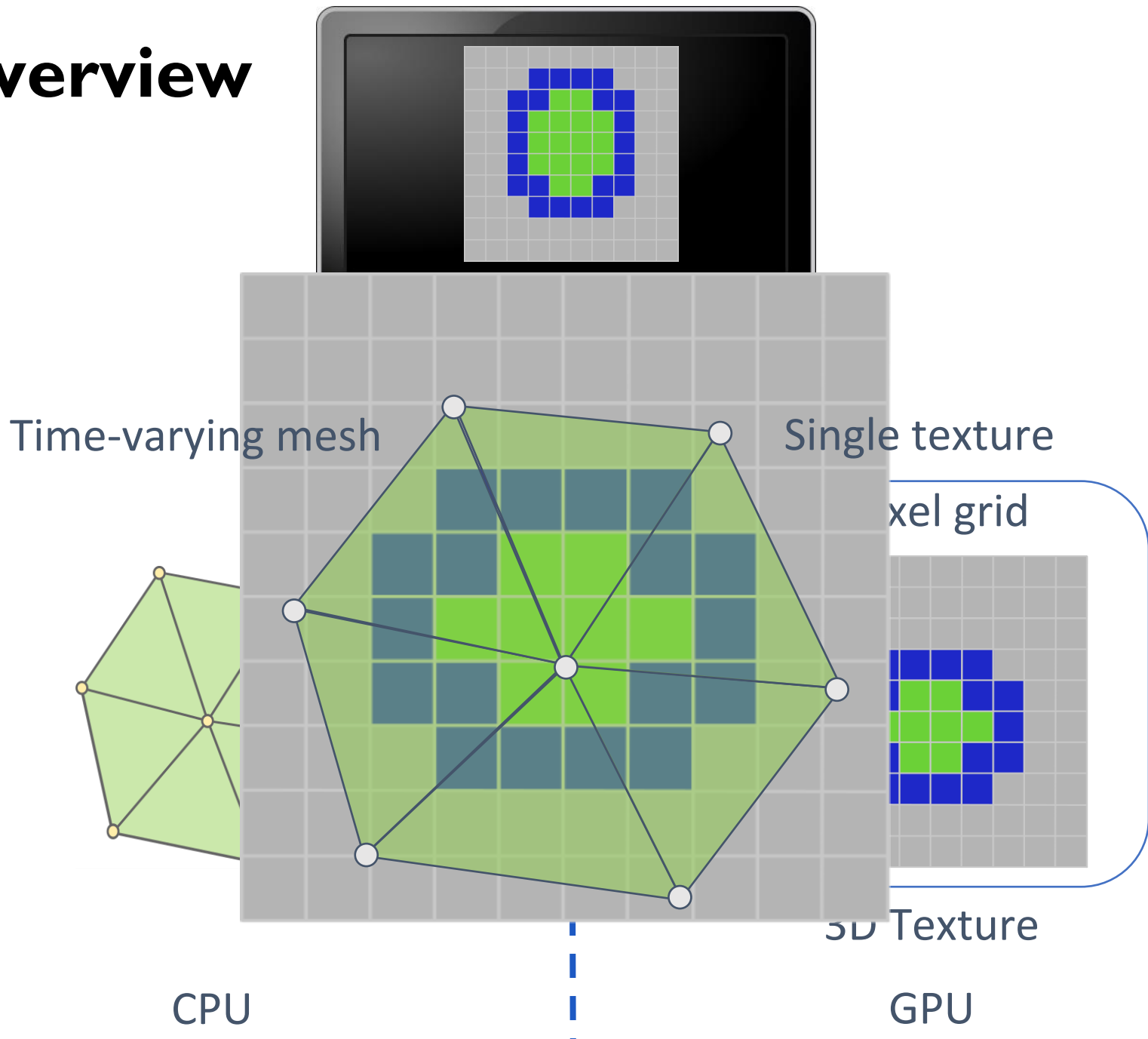


Embedding TM

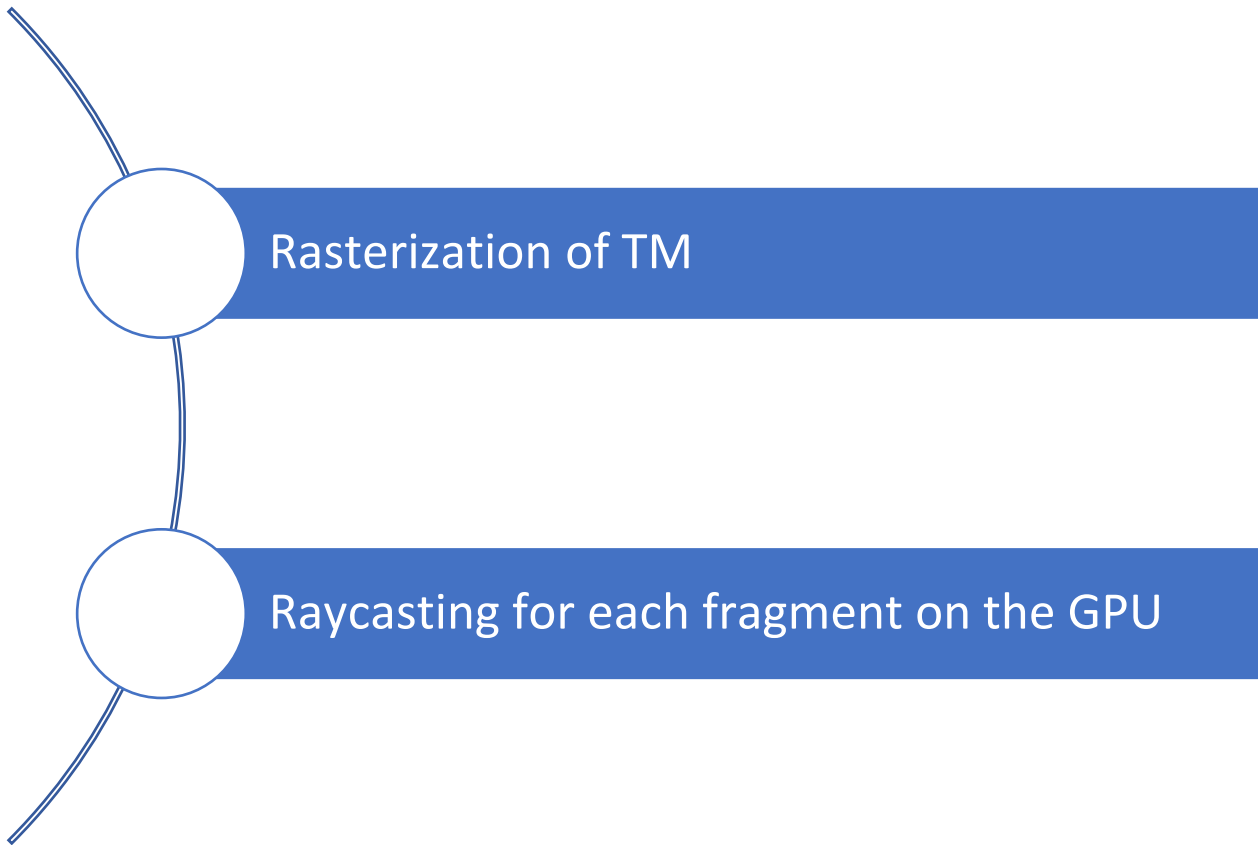


HD voxel grid

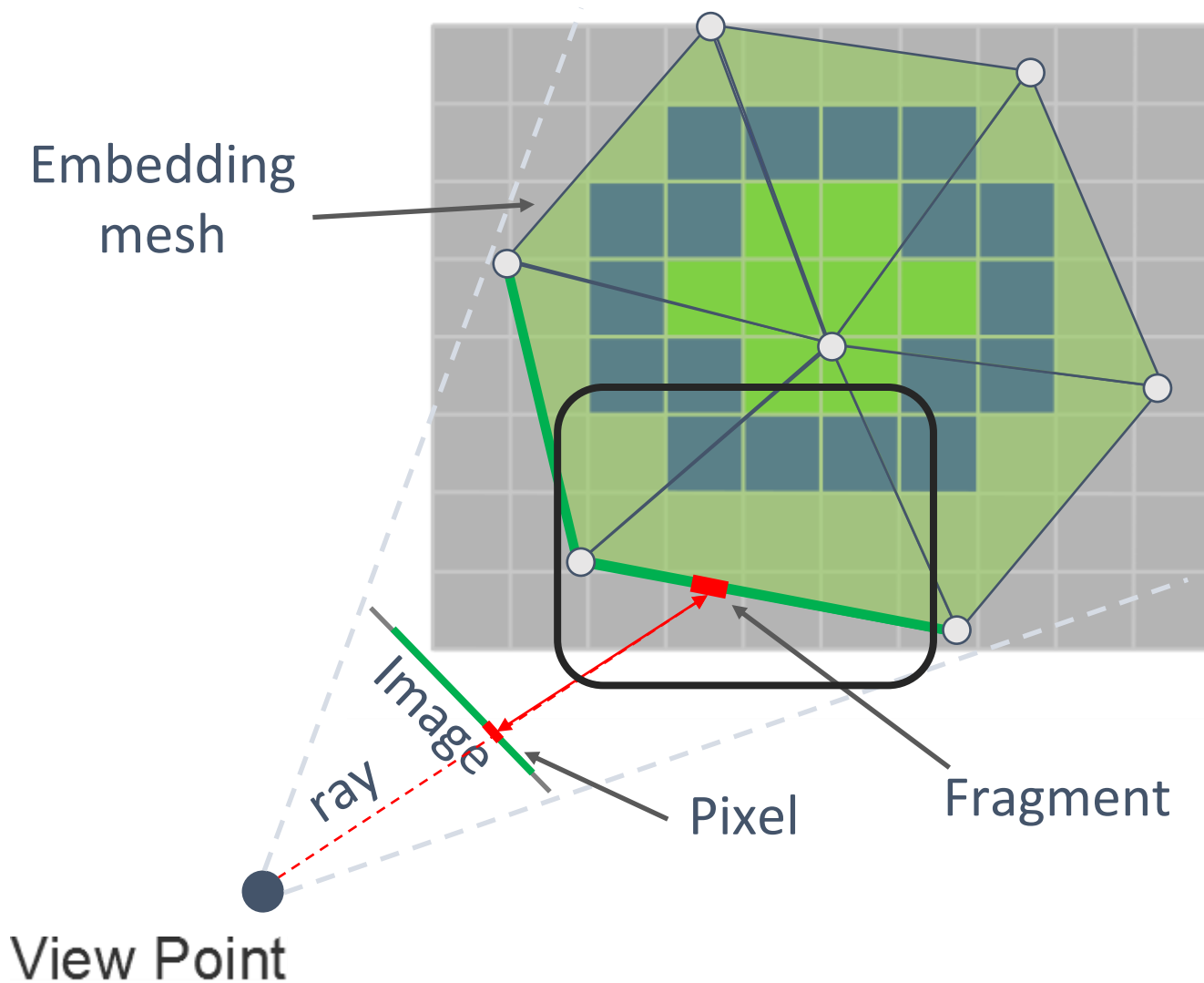
Overview



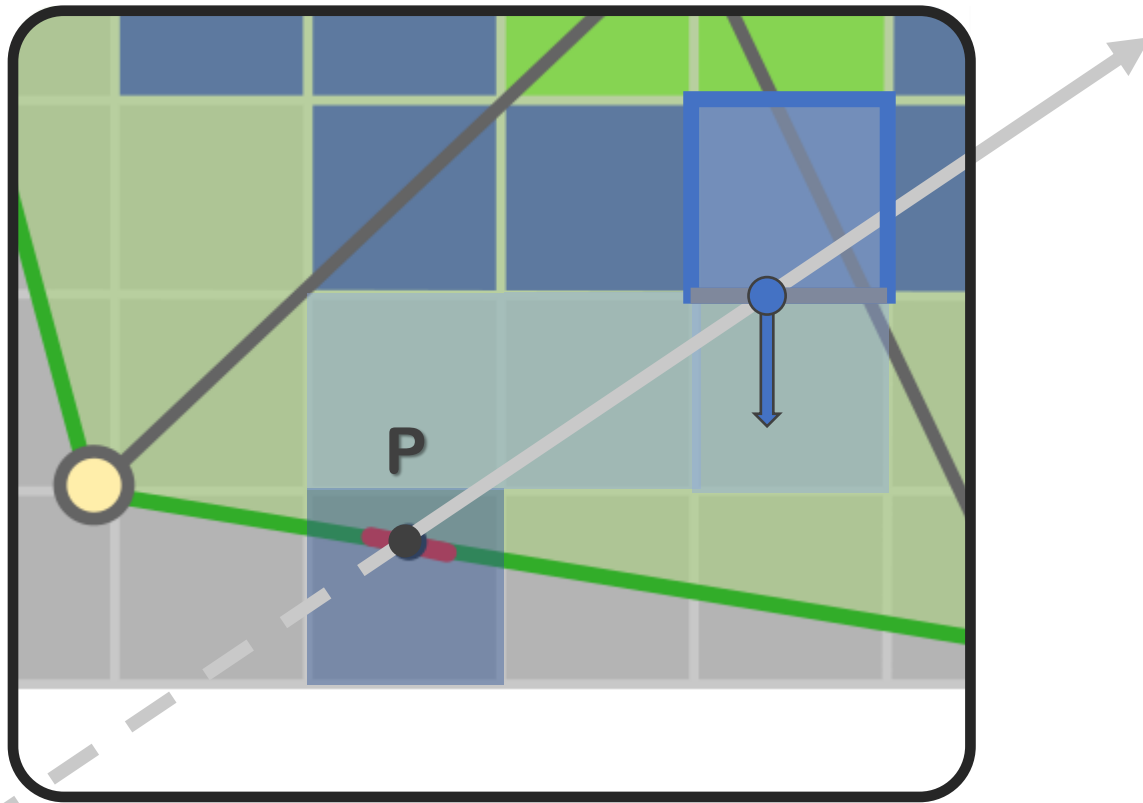
Overview



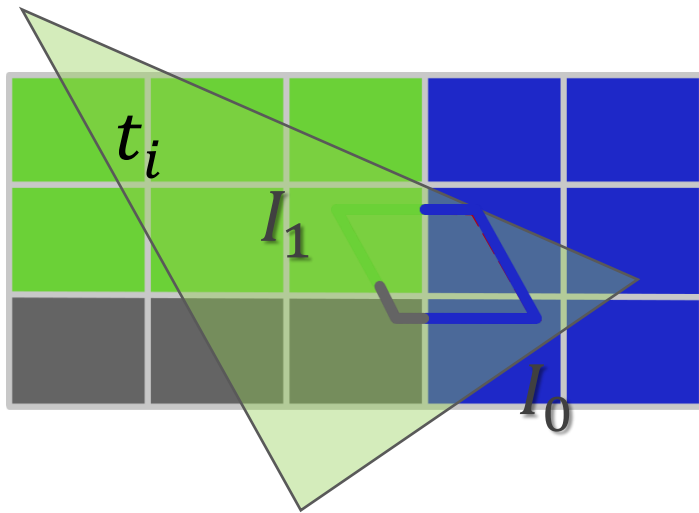
Rasterization



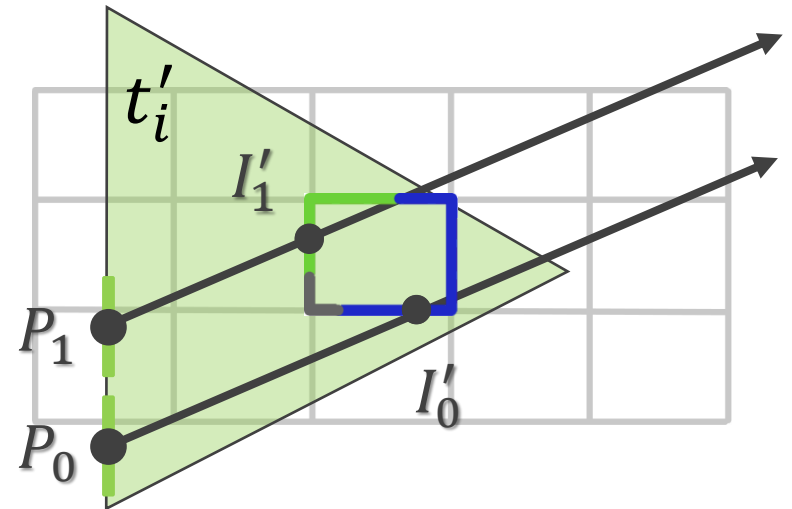
Ray casting



Color consistency problem

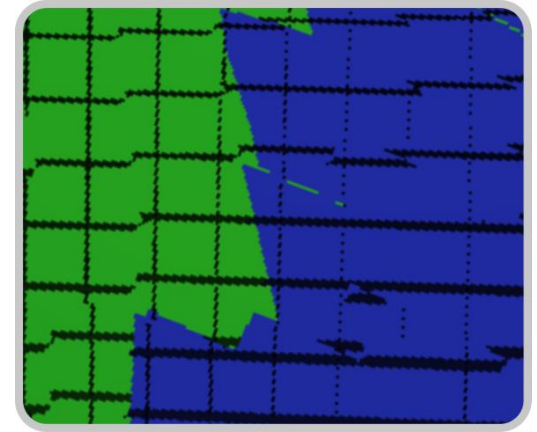


Input grid

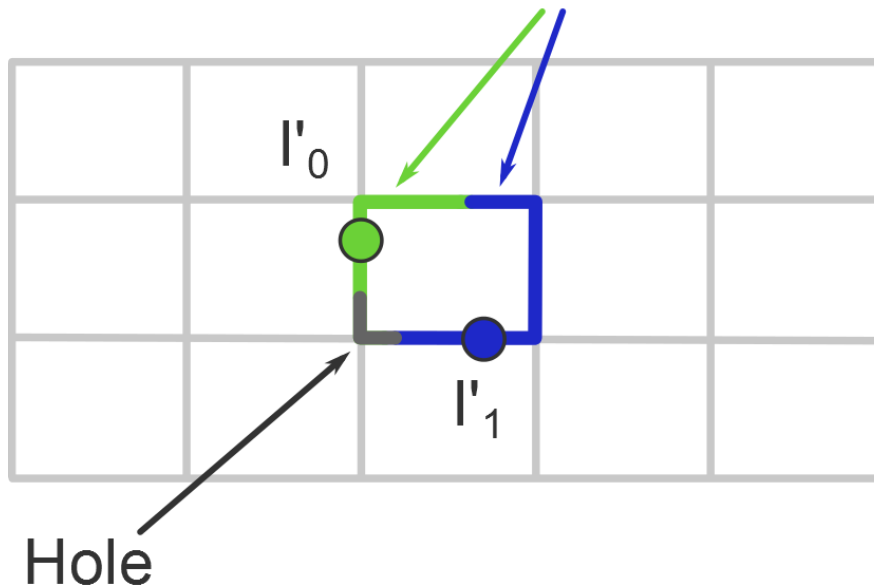


Deformed grid

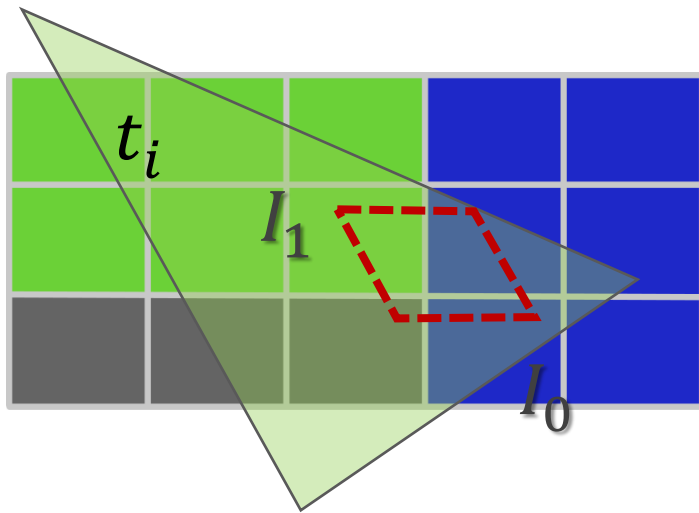
Color consistency problem



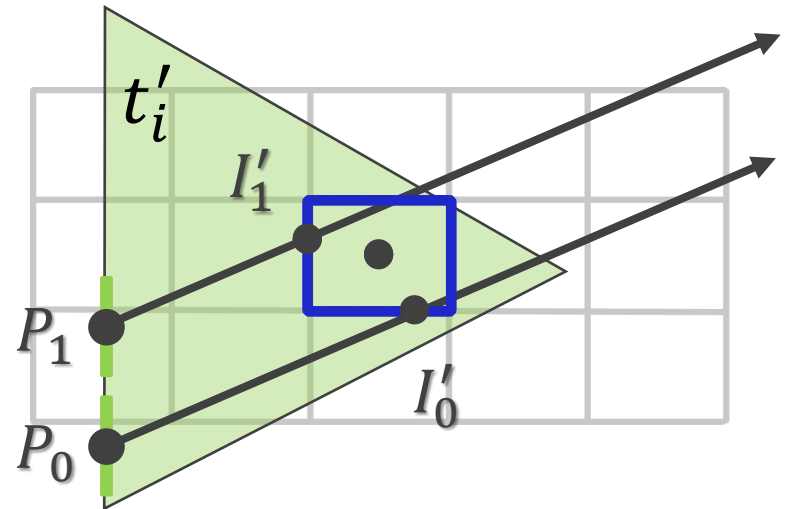
Two colors for a voxel



Color consistency problem



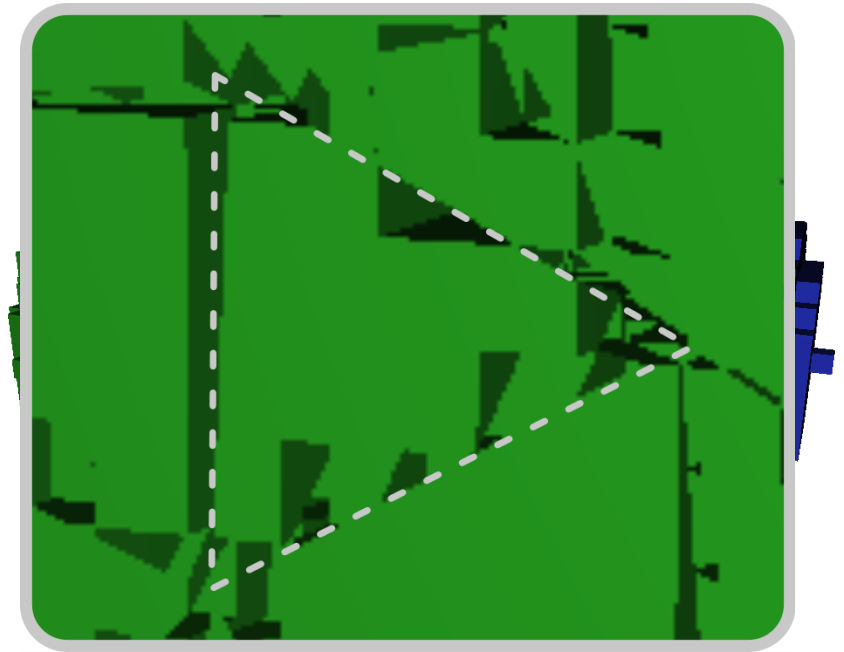
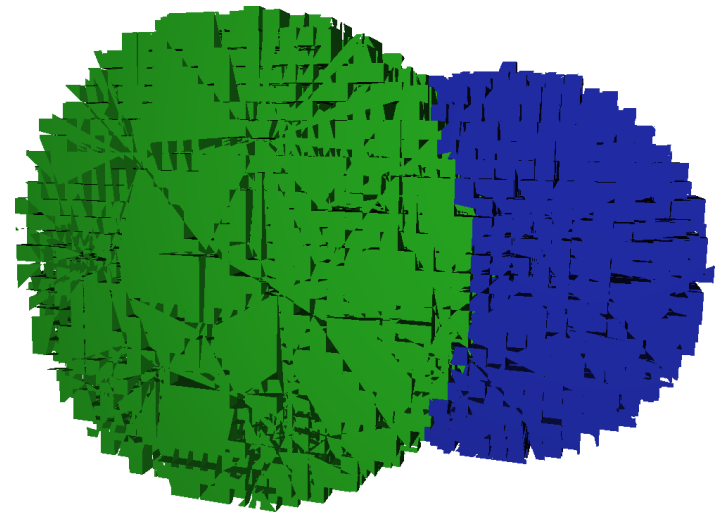
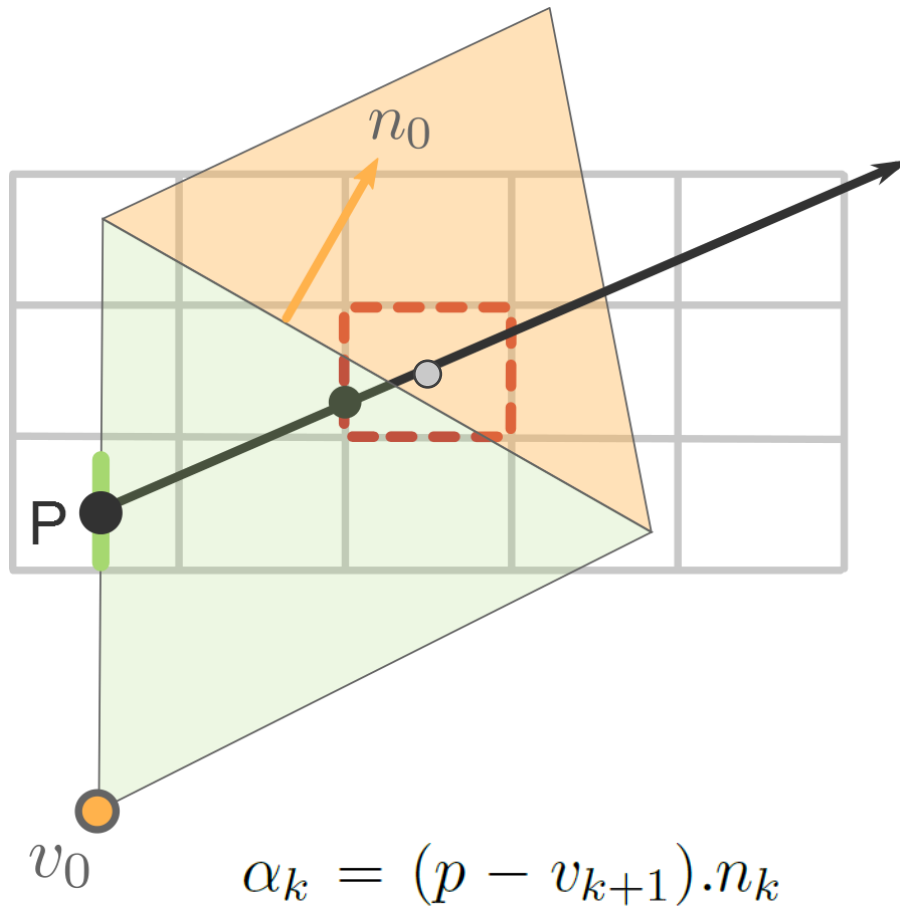
Input grid



Deformed grid

Neighborhood search

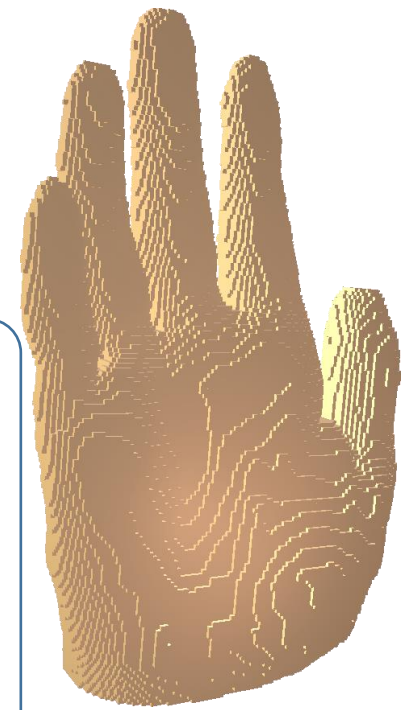
Depth-first search



Results



70 fps



1.14 M